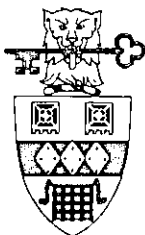


VECTOR



*The Journal of the
British APL Association*

GRAPHICS SPECIAL

- APL '84 in Helsinki — a photographic review.
- Find out about Mr. Babbage's secret.
- An APL framework for Expert Systems
- GKS and Graphic Design Standards
- ASCII notation and spotlight on statistics.
- Latest daring revelations from the ISO
- Once again we bring you 100 + pages of sizzling APL adventure.

A Specialist Group of the British Computer Society

Vol.1 No.2 October 1984

CONTRIBUTIONS

All contributions to VECTOR should be sent to the Editor at the address given on the inside back cover. Letters and articles are welcomed on any topic of interest to the APL community. These do not need to be limited to APL themes nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible, including a photograph of the author. Unless otherwise specified each item will be considered for publication as a personal statement by its author, who accepts legal responsibility that its publication is not restricted by copyright. The provision of camera-ready or machine-readable copy is encouraged: please contact the Editor beforehand. Program listings should indicate the computer system on which they have been run. APL symbols should be displayed on a separate line and not embedded in narrative. Except where indicated, items published in VECTOR may be freely reprinted with appropriate acknowledgement.

MEMBERSHIP

<i>Category</i>	<i>Fee p.a.</i>	<i>VECTOR copies</i>	<i>Passes</i>
Nonvoting student membership	Free (1984)	1	1
UK Private membership	£ 6	1	1
Overseas private membership	£ 10	1	1
Corporate membership	£ 50	10	5
Sustaining membership	£250	100	5

The membership year runs from 1st May to 30th April. Applications for membership should be made on the form at the end of the journal. Passes are required for entry to some Association events and for voting at Annual General Meetings. Applications for student membership will be accepted without charge at the discretion of the Committee on the production of educational bona fides and a recommendation from a course supervisor. Overseas membership rates include VECTOR airmail postage and should be paid in UK £.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive multiple copies of VECTOR and are offered group attendance of Association meetings. Partaking individuals need not be identified but a contact person should be nominated for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in the editorial section of the journal and opportunities to inform APL users of their products via seminars and articles.

ADVERTISING

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20 mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £200 per page. A6 and A7 sizes are offered pro-rata subject to layout constraints. Deadlines are:

Advertisement booking: 3rd Friday in April, July, October & January.
Camera-ready copy: 1 week later. Distribution: 2 weeks later.

Advertisements should be booked with and sent to David Preedy at the address on the inside back cover.

CONTENTS

	Page
EDITORIAL: Adopting the Missionary Position? Robert Bittlestone	3
APL NEWS	5
General correspondence	5
Quick-reference APL diary	8
British APL Association news	9
International APL news	12
British Computer Society news	21
News from sustaining members	25
APL product guide	29
Book reviews	41
Software review	44
RECENT MEETINGS	47
APL vs. packages and 4GL's	48
4GL's Killing APL?	49
APL84 in focus	52
An APL Compiler	54
Dyalog APL	54
The Ampere: A Portable APL Machine	55
APL2	56
The APL Machine	57
GENERAL ARTICLES	59
Mr. Babbages's Secret: A Presentation	61
XPL: An Expert Systems Framework in APL	65
Steps towards a Better Basic — part 2	73
APL and Graphics Standards	75
GKS: An Opportunity for APL	78
Graphics for Decision Makers	83
Case Study: Imperial Group plc	89
TECHNICAL ARTICLES	97
Technical Editorial	99
Technical Correspondence	101
Prize Competition: Matrix Numeric Validation	107
APLip: The Hard Seed of a Health and Wealth giving Fruit	109
Statistical computing with APL	113
Numeric integration & probability distribution	117
ISO Standards update	125
INDEX TO ADVERTISERS	127

The VECTOR Working Group

Jonathan Barman
Technical Section



David Preedy
Advertising & Reviews



Robert Bittlestone
Editor



Adrian Smith
Case Studies



David Ziemann
Technical Section



EDITORIAL: TIME TO ADOPT THE MISSIONARY POSITION?

By Robert Bittlestone

Response to the publication of VECTOR Volume 1 Number 1 in July 1984 has been most encouraging. Production technicalities apart, we've received nothing but praise for the new format of this journal. Our thanks to all those who have kindly written us letters; but we shall, however, start to become suspicious if the knocking copy doesn't arrive soon. No doubt this current issue will give plenty of opportunity for proffering the practised poison pen.

Enthusiasts of the APL computer language have long since resigned themselves to the fact of their minority role in the league of computer language usage. "Look at us", we exclaim when surrounded by a throng of like-minded converts at Heidelberg, Washington or Helsinki. "APL is the only language which hosts an international annual forum". "Ah yes", reply the cynics, "but that's because it's the only language whose international user base can fit into a single dining hall."

In fact this isn't true. The annual conventions tend to exhibit the same group of faces, *mutatis mutandis*, representing the inner sanctum, the mystics who are reverentially charged with the preservation and development of the language. There continues to be a much larger group of less fanatical but equally persuaded users who work in industry, banking, insurance, government and almost every other profession. These people are simply too busy to attend an APL conference — for all the right reasons.

But there is another set of people out there who code systems in APL who simply do not know that APL conventions, associations, publications — even this one — exist at all. David Preedy told me about an occasion on the Bristol-London train journey when he had been discussing matters of APLeque gravity with a colleague. They were finally interrupted at the approach to Paddington by the occupant of the adjacent seat, who had been unable to contain his amazement at hearing that a whole organisation — indeed several — existed to promote the language that he thought only he had discovered.

So please adopt the position of a missionary in the cause of APL. Think of some APL users you know who haven't heard of the British APL Association — or even some open-minded savages who are still sacrificing their dead in the names of their local deities BAY SICK, FOUR TRASH, KOBRAWL or P'SKULL. Take up this copy of VECTOR, walk up to them and stand on your soapbox (it may be your only tradeable commodity). There are at least 10,000 APL users in the United Kingdom alone and the current membership of the B.A.A. is less than 10% of that figure. Why not do it now? They have nothing to lose but their intellectual virginity — and as for you: well, you always wanted to be a Viking didn't you? After all, isn't that why APL is so popular in Scandinavia?

Give your Information Center users the world.

An Information Center should be a company-wide resource taking advantage of modern technologies through the integration of hardware, software and data communications.

There are many communications companies in the world. There are many software companies. But for full integration of communications and software, there is only one I.P. Sharp.

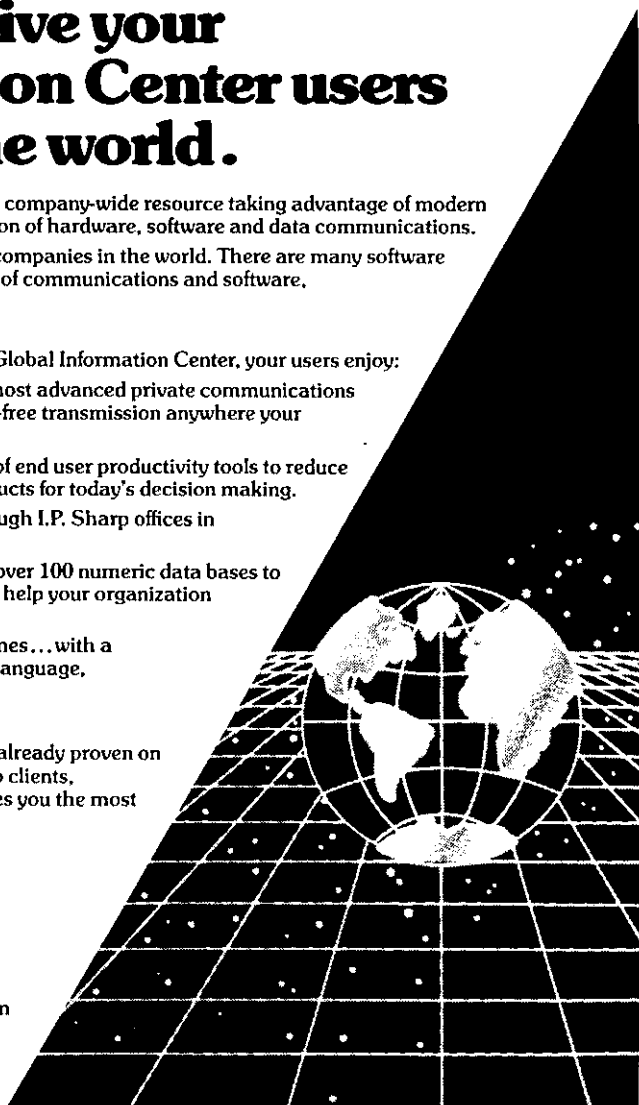
With I.P. Sharp's products for the Global Information Center, your users enjoy:

- IPSANET... one of the world's most advanced private communications networks providing secure error-free transmission anywhere your company operates.
- Integrated Software... a library of end user productivity tools to reduce your applications backlog. Products for today's decision making.
- Worldwide User Support... through I.P. Sharp offices in 22 countries.
- Online Public Data... access to over 100 numeric data bases to complement corporate data and help your organization make informed decisions.
- Integration of PC's and mainframes... with a single application development language, SHARP APL.

Ready to install, ready to run, and already proven on an international scale by I.P. Sharp clients, the Global Information Center gives you the most advanced implementation of the Information Center concept.

Why not let them have the world?

Call your nearest I.P. Sharp office in one of the cities listed below and we'll show you how your Information Center users can have the world.



I.P. Sharp Associates

**Products for the
Global Information Center**

London, England (European Headquarters)
I.P. Sharp Associates Limited,
132 Buckingham Palace Road, London SW1W 9SA,
England. (01) 730-4567. Telex: 8954178 SHARP

Aberdeen, Amsterdam, Atlanta, Boston, Brisbane, Brussels, Calgary, Canberra, Chicago, Copenhagen, Coventry, Dallas, Denver, Dublin, Düsseldorf, Edmonton, Frankfurt, Halifax, Helsinki, Hong Kong, Houston, London, Los Angeles, Madrid, Melbourne, Mexico City, Miami, Milan, Montreal, New York City, Newport Beach, Oslo, Ottawa, Palo Alto, Paris, Rochester, Rome, San Francisco, Savannah, Seattle, Seoul, Singapore, Stockholm, Sydney, Tokyo, Toronto, Vancouver, Victoria, Vienna, Warrington, Washington, Wayne, White Plains, Winnipeg, Zurich

GENERAL CORRESPONDENCE

We have been inundated with letters following the publication of the inaugural issue of VECTOR. One consequence is that we have split the letters section into general letters, which appear here, and technical letters, which are in the Technical section. It would be helpful if correspondents would indicate which section their letter is intended for. Letters containing APL symbols normally appear in the Technical section: please note the requirements on the inclusion of APL code stated on the inside cover. The Editor reserves the right to edit letters unless a writer states that a letter is to be published in full or not at all.

Unenlightened

From Mr. Witcomb

26th June 1984

Sir: I Was at 'APL in Focus' on 22nd June, and I am writing to say how much I enjoyed reading VECTOR, particularly the AGM notes, during the more erudite sessions. Also, I expect 'Why APL?' to be helpful in explaining APL to some unenlightened colleagues.

Yours sincerely

Simon Witcomb,
Nabisco Brands Ltd.,
121 Kings Road,
Reading,
Berks.

Auspicious

From Mr. Andrew Tarr

28th June 1984

Sir: First let me congratulate you on a most impressively auspicious inaugural issue of VECTOR. It appears to contain something for anyone with an interest in APL — my only worry is that any attempt to follow it must be an anti-climax!

As the introduction to the APL competition gave no name to receive entries, I am enclosing mine with a brief related communication which you might consider for the letters column. The 'APL84 in focus' meeting showed that some people are concerned with accelerating the execution of APL, and until we all get 'compilers' we should make our own attempts to balance efficient coding, quick execution and not filling workspace.

Yours sincerely,

Andrew Tarr
ICI plc (Mond Division)
Finance and Info Systems Dept.,
The Heath,
Runcorn,
Cheshire.

Editor: Mr. Tarr's comments will be included in our presentation of the "This is Your Life" competition results in the next issue.

Apology

From Professor Alan Hawkes

4th July 1984

Sir: As VECTOR is published by the British APL Association, may I be allowed to use its pages to make a public apology to Norman Thomson, and to correct a false impression which people may get from reading my article in "The Role of APL in Statistics Teaching" which was also published by the APL User Group (as it was then called).

Thomson's paper on multivariate analysis, which appeared in the proceedings of the APL83 Washington conference, and which I refer to on p.131 of my article (another apology for spelling his name wrong!), is a very good example of the way in which APL can be used to enhance a student's understanding of statistics. That is what I **intended** to convey. Unfortunately, the wording of my article can be interpreted as giving rather the opposite impression. If the last two sentences of that paragraph are interchanged, it then conveys unambiguously the meaning which I had intended.

Sorry Norman!

Yours sincerely,

Alan G. Hawkes.
Dept. of Management Science & Statistics,
University College of Swansea,
Singleton Park,
Swansea.

Enthusiasm

From Mr. Philip Goacher

5th July 1984

Sir: Can I say how pleased we were to have such a well produced and meaty journal to take to Helsinki. It certainly had great impact and I was sufficiently carried away to announce that it was now the international APL journal. This was greeted enthusiastically by most, and many offers of contributions were forthcoming — let's hope they materialise. Competition to Quote Quad was taken more philosophically by the SIGAPL team!

Yours sincerely,

Philip Goacher,
Chairman, British APL Association,
The British Computer Society
13 Mansfield Street,
London W1.

Sensible

From Mr. G.N. Lance

9th July 1984

Sir: May I first, in my capacity as Chairman of the BCS Publications Committee, congratulate the APL Specialist Group on its production of the first issue of the new periodical 'VECTOR'. First impressions are extremely favourable and the BCS should be proud to have this added to their list of regular publications. Maybe more Specialist Groups will follow the example.

The first "Letter to the Editor" was from Mr. R. Bowman on the subject of Character Representation in APL; may I please comment on this letter, although I do not pretend to be an APL expert.

In reality most computer languages have encountered the kind of difficulty which Mr. Bowman highlights; however, the authors of these languages have tackled the problem in a different way. Rather than invent new symbols which do not exist on standard terminals they use a combination of standard ASCII characters to represent special symbols. Obvious examples which come to mind are the ".GT." in Fortran to represent ">". Now it occurs to me that if APL users are anxious to promote the exchange of programs using standard protocols, which accept only ASCII characters, then they too could invent equivalents **simply for the purpose of communication.** (N.B. II in ASCII means Information Interchange.) Clearly text converters would be needed but they should be quite simple to produce. Whether this helps the **publication** of APL programs is another question, a lot would depend on how "sensible" the equivalences were.

I am sure that this idea is not original but hopefully it will keep this correspondence alive.

Yours sincerely,

G.N. Lance,
Chairman, BCS Publications Committee,
University of Bristol,
Computer Science Department,
School of Mathematics,
University Walk,
Bristol.

Brevity

From Mr. Llewellyn-Jones

12th July, 1984

Sir: I am very much enjoying reading VECTOR and would like to offer my congratulation to all the members of its working group for putting together such a readable, bumper, inaugural issue. A success such as this will be difficult to follow so I wish all of them great good fortune in the tasks that lie ahead. My one and only criticism of the publication is its "perfect" binding which, in the case of my copy, is turning out otherwise.

I was very interested in the matter of an ASCII-based character set for APL which was the subject of Mr Bowman's letter as well as being a topic of numerous articles elsewhere in the journal. I became so moved by the force of the arguments in its favour that I felt impelled to sit down and try to design one which would lose less in brevity than FGL, because brevity is not only the soul of wit, it is also one of APL's great strengths.

Yours sincerely,

L. Llewellyn-Jones
Reuters Ltd,
85 Fleet Street,
London EC4.

Editor: Mr. Llewellyn-Jones appended a further nineteen pages to his letter, which we have included later in this issue — see "APLpip — The Hard Seed of a Health and Wealth Giving Fruit". The covers of the first 500 copies of Vol. 1. No. 1. were not laminated in order to meet the Helsinki deadline: some of these have deteriorated prematurely.

QUICK-REFERENCE APL DIARY

Compiled by David Preedy

Efforts are made to ensure the accuracy of these listings before publication, but readers should in all cases verify details nearer the indicated date. Association members will be posted details of all meetings.

Unless otherwise indicated all Association events are free, open to non-members as well as members, and start at 2 p.m. at:

The Read Lecture Theatre,
 Sherfield Building,
 Imperial College of Science and Technology,
 South Kensington,
 London SW7 2BT.
 Tel. 01-589 5111

Underground: South Kensington, then subway to Science Museum.

The "APL Publications" bookstall will be a regular feature of all London events, giving delegates a chance to purchase hard-to-find APL literature.

Date	Venue	Event
September 21	London	Association meeting: "APL and knowledge-based systems"
October 15-17	Toronto	I.P. Sharp APL User Meeting: "The Information Centre and Changing Technology"
October 19	London	Association Workshop: "How to survive in Version XX of APL"
October 31	Lisbon	Portuguese APL Association (APL*2) Minicongress
November 15-17	Mexico	Mexican APL Society Annual Conference
November 16	London	Association meeting "Information Centres Revisited"
December	Midlands	Association Special Event: "What's New for APL in 1985"
1985 May 12-16	Seattle	APL'85 Conference: "APL and the Future"
1986 July 1-4	Cambridge, England	APL'86 Conference.

Further details of events organised by the British APL Association are given in the British APL Association News section of VECTOR.

Overseas events are detailed in the International News section

BRITISH APL ASSOCIATION NEWS

Compiled by Robert Bittlestone

Situations Vacant

Mr. Christopher Beatty, formerly Education Officer, has indicated his enthusiasm to transfer his activities to the Publicity Officer role, and has been confirmed in that role by your Committee, which anticipates with pleasure the missionary zeal that he is sure to lend to that position. As a consequence the role of Education Officer becomes vacant.

The Association's Education Officer is expected to spearhead the exposure of APL to schools and universities by involving them in the Association's activities and encouraging them to sample the language. Duties include (a) acting as a reference point for inquiries from the academic world, (b) promoting the use of APL in schools and universities, (c) liaising with the Activities Officer in arranging educational events and seminars, and (d) contributing material to VECTOR to assist the APL novice.

Various candidates for the Technical Officer's role are under consideration following the announcement of the vacancy in the previous issue of VECTOR. In the event that no appointment has been made by the date of publication, we are reprinting the details that we published then.

The role of Technical Officer requires a history of exposure to APL, ideally on several different types of computer, and an acknowledged expertise in the technical aspects of the language. Duties include (a) acting as a reference point for members of the Association who have technical inquiries, (b) coordinating the activities of the Association in relation to APL standards, (c) supporting the Journal Editor by identifying potential authors, and (d) supporting the Activities Officer by identifying speakers and panellists.

Situations Filled

Christopher Beatty, our new Publicity Officer, writes as follows:

Having eschewed Higher Education on leaving school, I spent 5 years in Chemical Engineering before taking a Physics degree at Essex University as a mature student (sic).

I then spent 10 years teaching in Uxbridge Mathematics which culminated in my appointment as Head of Computer Studies at Westminster City School — largely due to the seminal experience of reading the 'Ladybird Book of Computing' prior to my interview.

During my 4 years of Golden Enchantment at Westminster, APL was introduced to the pupils and Westminster Computer Systems Ltd. was formed by myself and a group of sixth formers. I have now severed my links with both the School and the above company and I now intend seeking my fortune in Computing and Communications.

Christopher Beatty, the Association's new Publicity Officer.



APL86

International APL Conferences take place alternately in the USA and elsewhere. The 1982 Conference was in Heidelberg; the 1983 in Washington; 1984 took place in Helsinki; 1985 will be held in Seattle, and 1986 has been allotted by EuroAPL to the United Kingdom to take place in Cambridge from July 1st-4th. Initial planning is currently under way by your Committee, and in due course it is likely that a special secretariat will emerge to run the affair, with an identity distinct from the Association Committee itself, although existing officers are by no means disbarred.

Offers of assistance in organising APL86 will be gratefully received by the Secretary. There is also a requirement for referees to scan through papers that are submitted to the conference. Although such appointments are normally by invitation, any individual who considers himself or herself qualified and enthusiastic for such involvement is also invited to contact the Secretary: address on the inside back cover of VECTOR.

Activities Programme 1984

Here are further details of forthcoming Association activities:

- September 21 — APL and Knowledge-based Systems
(Do we really have to learn PROLOG; or what should we do to make APL attractive in this growing area?)
- October 19 — Workshop — How to Survive in Version xx of APL (Some APLs are more equal than others; what do you need to do if you covet that which your vendor doesn't supply?)
- November 16 — Information Centres Revisited. (The British APL Association has set up some working groups, one of which is in the Information Centre area; here they report on some case studies)
- December 10 — What's New for 1985
This event will be a revival of the long-popular "Roadshow" in which we present an exhibition of all that's new in the APL world both in hardware and software. Product forums will accompany the exhibition. A new feature this year will be a "software village" to which we invite anyone with interesting APL software (both implementations and applications) to exhibit. Please contact either Dick Bowman or David Preedy as quickly as possible if you want to book exhibition, village or forum space.
- 1985 The technical meetings programme for 1985 is being prepared; among the prospective topics are:
- Graphics
Generalised Arrays Workshop
Tree-structured datasets and/or user interfaces
Text and numeric editors
Interpreters and Compilers
- More complete details will appear in due course; meanwhile if there are any topics on which you have a burning desire to know more (or indeed a compulsion to tell all) please contact the Activities Organiser.

The programme is subject to revision; further details of each meeting will be sent to B.A.A. members nearer the date.

The recent Activities Questionnaire reveals some interest in our holding meetings in venues other than London. We are actively considering this and would like to hear from anyone willing to act as a local organiser.

INTERNATIONAL APL NEWS

Compiled by Robert Bittlestone

This column is open to other APL Associations outside the UK; please write to us with your news and we shall be pleased to broadcast extracts for you. A substantial number of VECTOR subscriptions are from overseas and we hope in this way to keep APL users in touch with each other internationally.

The International News section in this issue offers a truly transatlantic tour from APL'84 in Helsinki to APL'85 in Seattle, via Toronto, Lisbon and Mexico.

APL'84 — Helsinki

All reports from Helsinki suggest that APL'84 turned out to be as successful a conference as everyone expected. Rather than attempting the gargantuan task of summarising the proceedings of the Conference, we are grateful to David Ziemann for preparing a photographic review of some of the memorable moments.

I.P. Sharp 1984 APL User Meeting

"The Information Centre and Changing Technology" is the theme of this conference, which will be held on October 15-17 in Toronto, Canada.

Keynote speakers include H. Mitchell Watson Jr., IBM Vice President and President, System Products Division, IBM Corporation, White Plains, New York; and James Cunnie, Director of Office System Planning, ITT Corporation, Stratford, Connecticut. There will be a special half-day presentation by James Martin, who is according to Computer Weekly the computer industry's most widely read author and best attended lecturer.

The registration fee for the 3-day Conference is \$320(US). A brochure and registration details are available by contacting:

Rosanne Wild, IPSA Ltd., Suite 1900, 2 First Canadian Place, Toronto, Ontario, Canada

or I.P. Sharp's London office.

More complete details of this conference were given in the previous issue of VECTOR.

APL*2 Minicongress

As you may already know, there is now an APL association in Portugal, called Associacao Portuguesa da Linguagem APL, or APL*2 for short.

APL*2 is holding its second minicongress on October 31st, 1984 at the Faculty of Human and Social Sciences, New University of Lisbon. It has been arranged to coincide with the National Informatics Conference organized by the Portuguese Computer Society.

The guest speaker will be Professor Wolfgang Janko, from the University of Karlsruhe, Germany, and several papers will be presented by local APLers. A detailed programme will be available by October 1st and will be mailed to anyone who requests it.

For further information, please contact the Chairman of APL*2:
Jose Marques Henriques, Rua Pedro de Barcelos 14,
Encosta do Restelo, 1400 Lisbon, Portugal.

Mexican APL Society Conference

The Mexican APL Society is holding its annual 3-day conference on November 15-17. Topics will include Microcomputers, Decision Support Systems, Documentation and Technical items.

At the time of going to print, we do not know the venue for the Conference, but further details can be obtained by contacting the organisers:

Adolfo Garcia, Presidente Sociedad Mexicana di APL,
Arenal 40, San Angel, Mexico 20, D.F.

APL'85 — APL and the Future

Looking slightly further ahead than the coming 3 months, it is not too early to be making arrangements for APL'85 in Seattle, Washington — the Emerald City. The following extract comes from the Conference Announcement:

"APL'85, the 1985 Conference of the international APL community will be held in Seattle, Washington, USA — May 12-16, 1985. The Conference is co-sponsored by ACM SIGAPL and by the Puget Sound Chapter of ACM. The Conference theme is: APL And The Future.

"The Conference Committee welcomes you to the scenic Pacific Northwest, home of beautiful mountains, forests, lakes and Puget Sound. May is customarily a warm and sunny time in Seattle.

"The conference site is the Seattle Westin Hotel. Its twin towers contain complete conference facilities, over 800 sleeping rooms, swimming, health and dining facilities. Located in downtown Seattle, the Westin is in easy walking distance of many of Seattle's fine tourist attractions. You can also ride the Monorail to the Seattle Center, then eat at the top of the 600-foot Space Needle, and visit the Center's fountains, shops and cultural attractions. Seattle's FREE downtown bus service will take you to other attractions, such as the waterfront and restored Pioneer Square. For more extensive touring, you can take a Gray Line boat and bus tour of Seattle, or a one- or two-day side trip to Vancouver or Victoria BC.

"The Conference starts Sunday afternoon, and ends Thursday noon. This is a day longer than previous North American APL Conferences, allowing fewer papers at one time so you can hear more papers of your choice. There will be the traditional fine banquet. The extra evening will feature a boat ride on beautiful Puget Sound, with an Indian-style salmon dinner, and entertainment by a Pacific Northwest specialty — African music from Zimbabwe played by a Marimba band.

"An excellent technical exhibit will feature the latest in APL Hardware, Software, Applications and Literature, with an emphasis on Personal Computers."

To obtain a registration packet, please send you name and address to:

Mike Metzger, 1800 N 46th Street, Seattle, WA 98103

or use IPSA and STSC Mailbox Code — APL85, attention Registrar.

The call for papers closed on September 14th., but other questions about the technical program should be submitted to the Program Chair:

Robert C. Metzger, I.P. Sharp Associates Inc.,
1200 First Federal Plaza, Rochester, NY 14614,
USA (IPSA & STSC Mailbox Code — RCM).

Questions about any other Conference matters should be directed to the Conference Chair:

Bob Gailer, Boeing Computer Services 9A-90,
Education & Training Divn., PO Box 24346, Seattle,
WA 98124, USA (IPSA & STSC Mailbox Code — APL85).

APL'86 — Cambridge, England.

If you think May 1985 is far enough to plan ahead, spare a thought for the British APL Association committee who are already planning for APL'86, which as you probably know is going to be held in Cambridge. Inevitably few details are available to date other than the venue and the planned date which is July 1-4, 1986. Further details will be announced in future issues of VECTOR.

APL Quote Quad Subscriptions

As an information service to VECTOR readers and as a courtesy to the editors of "APL Quote Quad" for allowing us to reprint some of their news material, we are relaying the details of how to subscribe to their journal.

APL Quote Quad is an informal quarterly publication containing articles, algorithms, announcements, book reviews, correspondence, problems, and other materials of interest to users of the programming language APL. Subscription is a benefit of SIGAPL membership, but is also available to non-members. Address inquiries to:

Fred Aronson
Coordinator of SIGs
Association for Computing Machinery
11 West 42nd Street
New York, New York
USA 10036

Telephone: (212)869-7440
Telex: 421686 ACMUI

The price of annual subscriptions to 'APL Quote Quad' is US\$30.00.

APL Quote Quad Back Issues

Back issues of **APL Quote Quad** are available, while supplies last, to members for \$7.50 US per copy (except Proceedings issues). Proceedings of some APL conferences are also available, with a discount for members. Request a Publications Catalog and/or Order Form, or mail order with payment or purchase order to

ACM Order Department,
Waverley Press,
P.O. Box 64145,
Baltimore, Maryland,
USA 21264.

Order Number	Description	Price \$
SIGAPL	APL Quote Quad	7.50
	1973-74 Vol. 4 Nos. 2 3 4	
	1974-75 Vol. 5 Nos. 1&3 2 4	
	1975-76 Vol. 6 Nos. 1 3 4	
	1976-77 Vol. 7 Nos. 1 2 3	
	1977-78 Vol. 8 Nos. 1 2 3 4	
	1978-79 Vol. 9 Nos. 1 2 3	
	1979-80 Vol. 10 Nos. 1 2 3 4	
	1980-81 Vol. 11 Nos. 1 2 3 4	
	1981-82 Vol. 12 Nos. 2 3 4	
	1982-83 Vol. 13 Nos. 2 4	
	1983-84 Vol. 14 Nos. 1 3	
554760	APL76 Conference Proceedings	23.00
554790	APL79 Conference Proceedings Vol. 9 No. 4 Parts 1&2	22.00
554810	APL81 Conference Proceedings Vol. 12 No. 1	28.00
554820	APL82 Conference Proceedings Vol. 13 No. 1	29.00
554830	APL83 Conference Proceedings Vol. 13 No. 3	24.00

International APL Organisations

We are publishing with permission an updated version of the list of organisations appearing in APL Quote Quad, Vol 14 No. 1. The VECTOR Editor would be pleased to receive information from any of these groups for publication.

Austria

APL-Club Austria
Dr. Siegfried Streit, Chair
Wiener Stadtwerke
Mariannengasse 4-6
A-1095 Vienna
Austria

Belgium BACUS

An affiliate of the Computational and
Applied Mathematics Group

Belgian APL-CAM Users Society
Joseph De Kerf, Chair
Roelenberg 72
B-2570 Duffel
Belgium

Telephone: 015/314724
IPSA mailbox: KERF

Canada: Toronto CIPS APL SIG
An affiliate of the Canadian Information
Processing Society

Toronto Special Interest Group on APL
Richard Levine, APL Co-ordinator
Xerox Canada Inc.
703 Don Mills Road
Don Mills, Ontario M3C 1S2
Canada

Telephone: (416)429-6750, Ext. 2684
(after 4 p.m.)
IPSA mailbox: RLEV; group box: TSIG

Finland FinnAPL

Finnish APL Association
Gustav Tallet, Chair
P.O. Box 1005
SF-00101 Helsinki 10
Finland

France

A subsidiary of the Association Française
de Cybernétique, Économique, et
Technique
Group APL, AFCET
Gilles A. Martin, Chair
CISI
35 Boulevard Brune
F-75014 Paris
France

Germany APLCG

APL-Club of Germany e.V.
Prof. Dr. Wolfrid Stucky
Postfach 6380
D-7500 Karlsruhe 1
Federal Republic of Germany

Italy

A subsidiary of the Associazione Italiana di
Calcolo Automatico
APL Club Italia
Giorgio Faconti, President
CNUCE
Via S. Maria 36
I-56100 Pisa
Italy

Telephone: (50)45245

Mexico

Sociedad Mexicana de APL, A.C.
Erick Alvarado, President
Teleinformática de México, S.A.
Arenal 40
México 20, Dist. Federal
Mexico

Telephone: (905)550 80 33

Netherlands APLWG

A subsidiary of the System-Programming
Section of the Netherlands Society for
Informatics (NGI)

Werkgroep APL
Dhr. Bram Kornaat, Secretary
Rijks Computer Centrum,
Fauststraat 1
NL-7323 BA Apeldoorn
Netherlands

Telephone: (55)778822

Portugal

Associacao Portuguesa da Linguagem
APL
José M. Henriques
Rua Pedro de Barcelos 14
Encosta do Restelo
P-1400 Lisboa
Portugal

Sweden SIG-APL

A subsidiary of the Swedish Society for
Informatics

APL Special Interest Group
Bo Lundmark
Sperry AB
P.O. Box 1138
S-171 22 Solna
Sweden

Switzerland SAUG

Swiss APL User Group
Herr Th. Formanek, President
Postfach
CH-3000 Bern 29
Switzerland

Telephone: (31)624980

United Kingdom

A Specialist Group of the British Computer
Society

British APL Association
Philip Goacher, Chairman
British Computer Society
13 Mansfield Street
London W1M 0BP, England

Telephone: 01 637 0471

USA: Dallas-Fort Worth SWAPL

Southwest APL Users' Group
Don Hatfield, Secy/Treas.
2809 Apple Valley
Garland, Texas
USA 75043

USA: Denver DIGAPL

Denver Interest Group for APL
P.O. Box 3931
Englewood, Colorado
USA 80155

USA: Houston

Houston APL Special Interest Group
Mary Hall, President
Charter Oil Company
P.O. Box 87535
Houston, Texas
USA 77287/7535

Telephone: (713)640-4724

USA: New York City NY/SIGAPL

New York Chapter, SIGAPL
P.O. Box 524
New York City, New York
USA 10025

USA: Rochester

Rochester Chapter, ACM SIGAPL
Robert Pullman, President
P.O. Box 92487
Rochester, New York
USA 14692

Telephone: (716)244-4732

USA: San Francisco APLBUG

APL Bay Area Users' Group
Dick Foss, Treasurer
Crocker National Corporation
West Tower, 29th Floor
One Montgomery Street
San Francisco, California
USA 94104

USA: Seattle

UW APL Users Group / SIGAPL
J.R. Atkins
University of Washington DH-05
Seattle, Washington
USA 98155

USA: Washington

Washington Chapter, ACM SIGAPL
Lawrence Dusold
Dept. of Health, Education and
Welfare HFF-146
200 C Street SW
Washington, District of Columbia
USA 20204

Telephone: (202)245-1413

What's in a Name

Whilst savouring the delights of Finnish vodka recently a discussion arose about the choice of the name VECTOR for our journal.

This lead to a suggestion from Jim Ryan, subsequently extended by Jim Brown, that an annual volume of VECTOR should be called an ARRAY.

To take this further, perhaps we should give the name SCALAR to all single sheet meeting notices and announcements and call future conference proceedings MATRIX.

Needless to say, a collection of such publications on your bookshelf would be a NESTED ARRAY.

Eugene MacDonnell collects his award from APL84 conference chairman Timo Seppala.



Phil Goacher, the chairman of the British APL Association, tells an expectant audience that APL86 will be held in Cambridge, England.



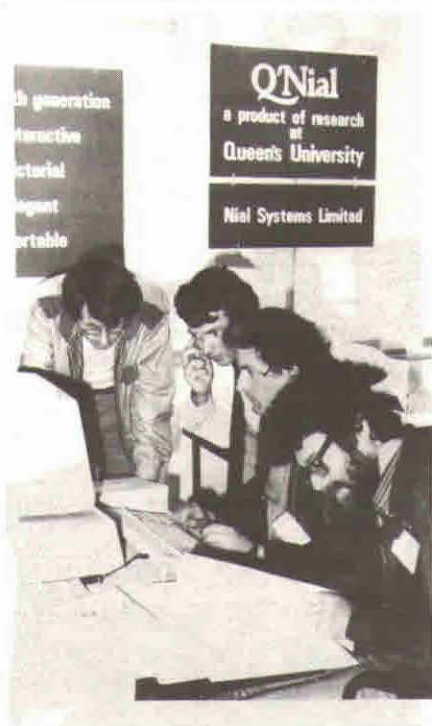
Gerald Langlet, a French delegate, tries out a problem on an Ampere micro.



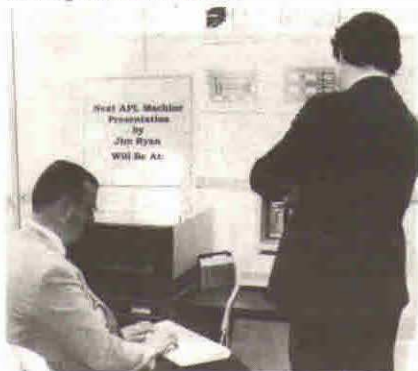
A vector of VECTORS.



"It says here: 'Rub the lamp and walk North'"



Jim Ryan demonstrating his APL on the Analogic APL machine.



Brian Drury of Cocking and Drury tries out some benchmarks on the Wang PC.



Timo Seppala, the conference chairman, deep in conversation



BRITISH COMPUTER SOCIETY NEWS

Compiled by Philip Goacher

Following my article concerning the BCS in the last issue of VECTOR, a number of readers have contacted me to enquire about membership. I have therefore put together the following notes on membership requirements, facilities and services. If you have any queries please do not hesitate to contact Pat Harris at BCS headquarters; she will be only too pleased to help.

Membership of the BCS

Each individual member belongs to an appropriate grade of membership, which reflects his or her experience and qualifications. Members pay a subscription which varies according to their membership grade.

The professional grades are:

Fellow (FBCS)

Member (MBCS)

Associate Member (AMBCS)

Requirements for Membership

Fellow (FBCS)

This grade is for persons who have been members of the Society for at least one year, and who are recognised as authorities amongst their peers in their sphere of computing practice.

Member (MBCS)

The requirements for applicants for this grade are:

Pass at, or exemption from, both parts of the Society Examination, plus at least four years' full-time working experience, **OR**

Exemption from Part II of the Society Examination, plus at least six years' full-time working experience, **OR**

An honours degree (or equivalent qualification) in a subject other than computing, plus at least six years' full-time working experience (NB Unless the degree contained a substantial computing element, more than six years' practical experience is likely to be required), **OR**

No formal academic qualifications, but at least ten years' full-time working experience; the applicant should be at least 30 years of age.

Applicants should normally be sponsored by two Fellows or Members of the Society.

Associate Member:
(AMBCS)

The requirement for applications for this grade are:

Pass at, or exemption from, Part I or Part II of the Society Examination, plus at least three years' full-time working experience, **OR**

Pass at, or exemption from, both parts of the Society Examination, plus at least one year's full-time working experience, **OR**

An honours degree (or equivalent qualification) in a subject other than computing plus at least three years' full-time working experience (NB Unless the degree contained a substantial computing element, more than three years' practical experience is likely to be required).

In addition there are the following non-professional grades:

Student: For those studying to take, or gain exemption from, Parts I and/or II of the Society's Examination.

Affiliate: For those wishing to take part in the Society's activities but who are not qualified for other grades of membership. The categories of Affiliate Membership are: Individual Affiliate and Institutional Affiliate (corporate bodies, institutions, companies, educational establishments and student societies).

Facilities and Services

All members of the Society benefit from:

- Opportunities to meet others from the scientific, commercial and social sectors of the computing industry
- Membership of the local branch of their choice
- Advice from, and participation in, the 46 specialist groups
- Choice of The Computer Journal or Computer Bulletin, plus the Society's newsletter, free of charge
- A copy of "Computing" each week
- Notification of conferences, exhibitions and seminars, many of which are run by the Society, and discounts on registration fees
- Discounts on computing publications from major publishers
- Use of the Society's library, housed at the Institution of Electrical Engineers, which welcomes personal callers as well as operating a postal lending service.

In addition, a wide range of advice and services connected with professional indemnity and legal charges are available to members of the professional grades.

Further details of the entry requirements can be found in the "Guide to Membership Regulations", available free of charge from the Information Department of the BCS. Detailed enquiries regarding eligibility should be made to

Ms P Harris
Assistant Registrar
The British Computer Society
13 Mansfield Street
London W1M 0BP
Tel: 01-637 0471

B.C.S. Appoints Cambridge University Press as publishers

As you may have noticed in the press recently, Cambridge University Press (CUP) are to produce the Society's periodicals and books in future, although specialist groups will of course continue to have complete freedom to use the publisher of their choice.

The Computer Journal is the leading publication of the BCS. It is produced quarterly and comprises papers that:

- (a) describe original research or new applications, or original contributions to standards or working practice, or
- (b) review the current position in some branch of computer technology which is not adequately described elsewhere, or
- (c) are of general interest to readers, for example, describing the difficulties encountered in introducing new working methods in an established situation, or
- (d) provide instruction on some aspect of computing of a standard at least equal to Part I of the BCS' membership examinations.

All papers are expected to have high computer content or, if not directly related to computer theory or application, must show how the results obtained are applied in real computing situation.

The Computer Bulletin is being completely restyled in 1985, and will present a authoritative voice in many areas of computing. The Computer Bulletin will have a very practical outlook and should be essential reading to a wide range of computer users and the computer managers that serve them.

The Computer Newsletter will appear ten times a year from next January and will keep members up to date with BCS activities, particularly those in the Branches and Specialist Groups (such as ours).

In addition to these publications the BCS publish several series of books including the Workshop Series (for conference proceedings, workshop reports etc) and Computer Monographs

There are the various ways in which our APL users can make use of the 'BCS connection' to publish papers, articles or books on their subject of interest, to supplement the pages of VECTOR.

If any of you are budding authors wishing to follow in Adrian Smith's footsteps then I would be pleased to offer any help to get your manuscripts into print in the most appropriate way.

Philip Goadar, BCS Business Manager (left)
and Michael Black, University Publisher of CUP, signing the publishing contract.



NEW FROM SUSTAINING MEMBERS

Compiled by David Pready

The category of "Sustaining Membership" of the British APL Association is available to any company trading in APL products and services; it is seen as a formal statement that these companies regard APL as more than simply another programming language, and is intended to promote increased interest and activity in APL and in the Association.

As well as receiving public acknowledgement for their sponsorship of the Association, sustaining members receive bulk copies of VECTOR to distribute to their customers, and are offered the opportunity to submit news material for this section of the Journal. They are also able to submit announcements for Association meetings (subject to the approval of the Activities Officer) and are invited to inform APL users of their products via seminars and articles.

The Committee of the British APL association would like to acknowledge the generous financial support of the following organisations that have become Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at no cost.

APL*PLUS Ltd.	Aston Science Park, Love Lane, Birmingham, B7 4BJ. Tel. 021-359 5096
Cocking & Drury Ltd.	16 Berkely Street, London, W1X 5AE Tel. 01-493 6172
Dyadic Systems Ltd.	30 Camp Road, Farnborough, Hants. Tel. 0252-547222
Inner Product Ltd.	Eagle House, 73 Clapham Common South Side, London, SW4 9DG. Tel. 01-673 3354
MicroAPL Ltd.	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP. Tel. 01-622 0395
I.P. Sharp Associates	132 Buckingham Palace Road, London, SW1W 9SA. Tel. 01-730 4567

Despite the inevitable pressures of work and the competing attractions of their summer holidays, almost all of these companies have managed to get their news items to us in time for the print deadline.

APL*PLUS Limited

APL*PLUS have been extending the list of IBM PC compatible machines on which APL*PLUS/PC is available. At the latest count it includes Wang PC, Olivetti M24, Eagle, Compaq, AJ Passport, Corona, Columbia, Ericson/Step One and ITT XTRA/OC. They have also announced Volume 1 of a tool kit as an optional addition to APL*PLUS/PC. This package of utility systems covers IRMA support for 3278 communications, full-screen management, RAM disk, a report formatter and various development aids.

APL*PLUS/UNIX is a new full-featured APL for the UNIX operating environment. It has virtual workspaces (if the hardware supports them), plus additional features such as shared file, full-screen editor, windowing, error-handling, nested arrays, etc. APL*PLUS/UNIX also does partial compilation of functions, which results in a 2-3 fold speed-up over tokenised code.

STSC's APL compiler will be available on their timesharing service from September. In addition customers will be able to purchase the compiler support package enabling them to run compiled code in-house; this will be available by November for APL*PLUS/1000 customers, and in the New Year for VS-APL sites.

Cocking & Drury Limited

Cocking & Drury have recently installed the first two copies of their AFM file system in the U.K. The first company wanted to provide integrated databases using AFM's shared file capability, whilst the second chose AFM primarily for its improvements in access time and storage requirements.

The company has been involved in A.I. work for some years, but has not been able to talk much about the projects they have worked on. However they will shortly be publishing a management overview to A.I., including an appraisal of current work on expert systems.

Cocking & Drury continues to grow. Recent arrivals include Caroline Hewson, Minaxi Patel and Bernard Perkins — making 18 full-time APL consultants.

Dyadic Systems Limited

In collaboration with its US distributors, Dyadic has recently announced implementations of Dyalog APL for the Convergent Technology Miniframe and Megaframe, AT & T's new 3B2, Sun 68000 and the very fast Pyramid Minicomputer. These versions are also available in the UK and Europe. Back at home, Dyadic has completed two new MC68000 implementations — the NCR Tower and Hewlett-Packard's HP9000-200. This brings the total number of implementations to 18.

Dyadic has signed an OEM deal with Modern Business Technology enabling MBT to market an enhanced version of Dyalog APL on its newly announced MC68000-based processor, the MBT 10/40. MBTAPL is also available on Torch computers. In addition, MBT is marketing Dyalog APL on Fortune 32-16 systems.

MicroAPL Limited

MicroAPL's SPECTRUM supermicro has been enhanced with a hardware change which speeds up CPU-bound jobs by about 25%. Basic SPECTRUM prices have remain unchanged since its launch in 1981. During that time CPU speeds have doubled, the number of RS232 ports has trebled and the basic memory has been quadrupled. At the same time more free software is included with the machine.

Testing of the GDDM emulation software (see previous VECTOR) is going well, and it will shortly be made available commercially. The emulation allows APL users to access graphics devices using a protocol compatible with IBM's AP126, with all the low-level GDDM calls fully implemented. It will run on all MicroAPL's Mirage-based machines, including the Sage IV and SPECTRUM.

The widely-used statistical packages GLIM, GENSTAT and the NAG statistical subroutines will shortly be available on MicroAPL equipment. There will be an interface from APL to these packages, allowing users to combine APL's easy manipulation of data with the use of standard, well-tested statistical software.

The latest recruit on the sales side is Paul Thornton, formerly of Burroughs and of Hart Computers. Paul will concentrate on U.K. end-user sales and deal particularly with new customers. Luan Thompson has been promoted to Director of Research & Development, and will be taking responsibility for both hardware and software development.

I.P. Sharp Associates Limited

The price of SHARP APL in-house has been reduced to approximately £1000 per month or £25000 for a long term licence. The system provides significant extensions over other IBM mainframe APL's. In addition a range of optional system software and application products allows users to acquire a system geared directly to their individual requirements.

In response to demand from their in-house customers, I.P. Sharp have developed a 4GL product called "VIEWPOINT". Directed at the same market area as products such as FOCUS and NOMAD, it requires no APL knowledge and makes sophisticated use of full-screen techniques. Of course, being written in APL it has very significant advantages in terms of applications development and customisation. VIEWPOINT is available in September and prices start at £1000 per month or £25000 for a long-term licence.

James Martin will be a key speaker at this year's User Conference in Toronto on 15-17 October. The meeting will concentrate on the opportunities provided by combining APL with the Information Centre. A number of Sharp's inhouse customers have also agreed to host visits to their data-centres to explain their current and planned use of APL. This will provide a unique opportunity to see a number of large and sophisticated APL users and to discuss how they have approached the problems inherent in running large Information Centres.

On the personnel front, Theo Sahlsberg has been appointed European Manager of Distributed Software in recognition of the increasing importance of in-house software sales to I.P. Sharp's product range. Theo was previously manager of I.P. Sharp's Stockholm office.

dyalog APL

...Probably the most powerful APL in the world

- Second Generation Interpreter
- Runs under UNIX™
- Nested Arrays
- Full-Screen Editor
- Full-Screen Data Manager
- Portable
- Component Files
- □FMT
- Event trapping
- External Variables
- Full implementation to ISO Standard
- Auxiliary Processes in high-level languages
- Complete access to UNIX™ facilities
- Full ASCII support

...and runs on all these computer systems

BLEASDALE BDC680	GOULD SEL
CADMUS 9000	HP9000
FORTUNE 32:16	PERKIN-ELMER
ICL PERQ	VAX
ZILOG S8000	(With many to follow)

TEST DYALOG APL FOR
YOURSELF BY MAKING USE
OF OUR **FREE TRIAL OFFER**

FREE TRIAL OFFER

dyalog APL

For further information fill in the coupon and post today to
Sales Department, Dyadic Systems Limited, 30 Camp Road,
Farnborough Hants Telephone (0252) 547222 Telex 858811

NAME _____

COMPANY _____

POSITION _____

ADDRESS _____

TELEPHONE _____

TM UNIX is a trade mark
of Bell Laboratories

APL PRODUCT GUIDE

Compiled by David Preedy

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

The second release of the Guide contains several additions of new products and suppliers who slipped through our net at the first pass. We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address on inside back cover.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to VECTOR readers, the product list has been divided into the following groups:

- Complete APL Systems (Hardware & software)
- APL Interpreters
- APL Visual Display Units
- APL character set printers
- APL-based packages
- APL consultancy
- Other services
- Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the VECTOR working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'

APL PRODUCT GUIDE

Company	Product	Prices £	Details
COMPLETE APL SYSTEMS			
Alan Pearman	IBM PC & XT	1995 – 4425	Authorised IBM PC dealer supplying complete systems including peripherals based on IBM PC.
C.A.C.I.	APL Machine	poa	APL-dedicated array processor from Analogic. Many configurations.
Cocking/Drury	MicroAPL SPECTRUM SAGE II SAGE IV	6000 – 35000	Supplied as part of a turnkey system. For product details see MicroAPL entry.
Inner Product	IBM PC	2000 – 6000	IBM PCs supplied for turnkey applications
MBS Rentals	IBM PC & PC/XT	2200 – 5000	For purchase or rental
	COMPAQ	poa	Portable IBMPC compatible
M.B.T.	MBT 10/40	poa	UNIX/68000 based multi-user APL system
	FORTUNE 32:16	poa	FORTUNE 68000 system enhanced for APL
	TORCH	poa	68000/Z80 multiprocessor
Metatechnics	COMPAQ with APL*PLUS/PC	3000 +	Portable IBMPC compatible with 3 utility packages
MicroAPL	SPECTRUM	11000 – 15000	Expandable multi-user APL computer using Motorola 68000. Standard configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	SAGE IV	8500	Multi-user APL computer, 1 Mb RAM, 12/18 Mb disc.
APL INTERPRETERS			
Alan Pearman	APL*PLUS/PC	420 – 600	See APL*PLUS entry
APL*PLUS	APL*PLUS/PC	600	Full feature interpreter for IBM PC and PC/XT and compatibles: CORONA, COMPAQ COLUMBIA, WANG, OLIVETTI, EAGLE, AJ, ERICSON, ITT

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL INTERPRETERS (continued)			
APL*PLUS	APL*PLUS/UNIX	poa	Interpreter for Unix systems: WICAT, CADMUS, CALLAN, FORTUNE 32.16, HP9000/500
	APL*PLUS/1000	poa	Enhancements to IBM's VSAPL. Runs under VM/CMS or MVS/TSO.
Burroughs	APL/700	3150	Runs on Burroughs B5000, B6000, B7000 and A9 mainframes.
Cocking/Drury	APL*PLUS/PC	600	Discount on multiple orders.
Dyadic	Dyalog APL	1000 – 8000	2nd generation portable APL for UNIX systems eg. VAX, HP9000, Fortune, MC68000, Zilog, Perkin-Elmer, Perq etc.
Inner Product	VIZ::APL	250 – 350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See APL*PLUS entry
IBM UK Product Sales	IBM PC APL	poa	With event-handling, & APs for full-screen I/O, disks disketts, asynch. comms.
I.P. Sharp	Sharp APL Sharp APL/PC	25000 55	For IBM mainframes For IBM PC or PC/XT
M.B.T.	Dyalog APL MBTAPL	poa poa	See Dyadic Systems entry Enhanced Dyalog APL for MBT hardware
	VIZ::APL	poa	Customized for TORCH hardware
Metatechnics	APL*PLUS/PC	600	With free utility package Discount on multiple orders.
MicroAPL	APL.68000	1000 +	Full implementation with component files, error trapping etc. for SPECTRUM and SAGE machines or other popular MC68000-based computers.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL VISUAL DISPLAY UNITS			
Brent-Cybernex	GR1014 Graphics Terminal	1970	Combined APL/ANSI/4010; 1024x1024., 2 ports, anti-glare, swivel and tilt.
	APL100	1065	Selectable APL/ASCII; hires chars; non-glare, swivel & tilt.
Farnell	Tandberg TDV2221	1498	Ergonomic design APL terminal, 50-19200 baud, 15" anti reflex screen, low profile keyboard.
	Tandberg TDV2271	1685	Combined APL/ANSI ergonomic terminal as above.
Lynwood	Alpha	1995	Full APL character set, 16-bit microprocessor with pixel addressable monochrome graphics.
	Alpha Colour	2985	As above but 8 colour graphics.
MBS Rentals	Lynwood Alpha	1995	See Lynwood entry.
	Lynwood Alpha Col.	2985	See Lynwood entry.
	Concept/APL	1491 - 1663	See Shandell entry.
	Concept GVT/APL +	2220 - 2393	See Shandell entry
	(All above for purchase or rental)		
M.B.T.	various		contact MBT for details
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1450	With monochrome graphics
	Tektronix 4105	2995	High resolution colour graphics supporting APL character set on MicroAPL hardware.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL VISUAL DISPLAY UNITS (continued)			
Shandell	Concept AVT/APL +	896 – 1195	Conforms to ANSI 3.64 standard; 80/132 columns, 4/8 pages of memory, windowing, two comms. ports, low profile keyboard.
	Concept GVT/APL +	1370 – 1825	As above with vector graphics.
APL PRINTERS			
Alan Pearman	Epson FX series NEC Spinwriter		Printer prices range from 289 to 1995 over all models.
Datatrade	Datasouth DS180	1395	180 cps matrix printer with 2000 byte buffer, 9x7 dot matrix and APL option.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 col.
	Anadex 9620	1150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1180	40 cps., letter quality
MBS Rentals	Epson FX series Datasouth DS180 Philips GP300	438-1100 1395 2251	See Datatrade entry. Matrix with letter & draft quality & APL.
	TI745 + APL DEC LA120 DEC LA12C AJ833 (All above for purchase or rental)	1390 2516 1444 2595	Portable dot-matrix. 120 cps dot-matrix. Portable dot-matrix. Daisy-wheel
M.B.T.	Facit 4565 Facit 4510/11/12	poa poa	40 cps letter-quality Matrix printers
MicroAPL	Datasouth DS180 Philips GP300	1545 2251	See Datatrade,entry Matrix printer with letter and draft quality and APL.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PACKAGES			
APL*PLUS	Info Centre	12000 – 20000	Tools for full-screen entry, display and analysis. Interfaces to other proprietary IC products
	CMCS	30000 – 100000	Complete Materials Management, Factory Planning & Forecasting package.
	Finance/Statistics	275	Comprehensive package for IBM PC and compatibles.
	Tools Vol. 1	poa	Utilities for IBM PC;IRMA 3278 comms, full-screen, RAM disk, report generator
C.A.C.I.	PILGRIM	20000	Language-driven data extraction, and foil preparation. VM/CMS VSAPL, AP 126.
	PIM	3000	PF-key driven personal information manager VM/CMS VSAPL, AP124
Cocking/Drury	ARMS 2	3000 – 15000	Applications Generator
	AFM	10000	High performance shared file system.
	CALL/AP	3000	Non-APL program execution.
	FORMAT	2250	Enhanced report formatting
	WSAIDS	795	Workspace documentation and development aids.
E&S	PROTOPAK		APL packages for prototyping on IBM PC & mainframes.
	consisting of:		Relational databases.
	RMS	550-2500	Multi-dimensional arrays.
	AMS	550-2500	Combined RMS & AMS.
	RAMS	750-3500	Financial modelling & forecasting
	BMS	750-3500	Full-screen editing for IBM PC.
	FMS	350	
	CMS	poa	Communications package.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
Holtech	CASH	3500 – 10000	Accounting package and hotel management system implemented on MicroAPL SPECTRUM & SAGE CPUs.
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/DBASE II	150	Interface APL with dBase II
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3500	EEC monetary and agrimonetary analysis.
I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	record maintenance/ reporting
	MAGIC	poa	Time series analysis/ reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MICROCOM	poa	Mainframe to micro link
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL — Infor centre product
	X TABS	poa	Survey Analysis
M.B.T.	RHOMBUS	poa	Integrated Office System
	HASLEMERE	poa	Hotel Accounting System
Metatechnics	MetaScreen	195	Full-screen handler for APL*PLUS/PC, based on VSAPL APL124
	MetaWS	195	Program Filing System for large APL systems on APL*PLUS/PC
	Toolbox	195	Utilities for APL*PLUS/PC.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL PACKAGES (continued)			
Metatechnics	DataLog.APL	poa	Interface A/Ds & D/As to APL on IBM PC.
	ADAPTA DLS	poa	Production & purchasing scheduling for continuous processes.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Project development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc.
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	500	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL *PLUS/PC product
APL CONSULTANCY (prices quoted are per day unless otherwise marked)			
Alan Pearman	Consultancy & Courses	25	Per hour, onsite and offsite
APL People	Consultancy	poa	All levels, most APL systems
	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short or long term projects, supervised.
Attikale	Consultancy	160	Management reporting systems & Information Centres.
C.A.C.I.	Consultancy	poa	All aspects, specializing in Decision Support Systems.
Cocking/Drury	Consultancy	120-150	Junior consultant
		140-200	Consultant
		185-300	Senior consultant
		275-400	Managing consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.

APL PRODUCT GUIDE (continued)

Company	Product	Prices £	Details
APL CONSULTANCY (continued)			
Dyadic	Consultancy & Training	poa	APL system design, consultancy, programming and training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping
H.M.W.	Consultancy	100-250	System design consultancy, programming.
Inner Product	Consultancy & Training	200	On-site micro/mainframe APL, PC/DOS & Assembler
I.P. Sharp	Consultancy	poa	Consultancy and support service world-wide.
M.B.T.	Consultancy & courses	poa	
Metatechnics	Consultancy	poa	Management Information & Production Management Systems & Process Control.
MicroAPL	Consultancy	150 - 250	Technical & applications consultancy.

OTHER PRODUCTS

I.P. Sharp	Productivity Tools	.poa	System utilities, operations utilities, system administration utilities, analyst utilities, auxiliary processors, comms. software, int. comms. network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

APL PRODUCT GUIDE (continued)

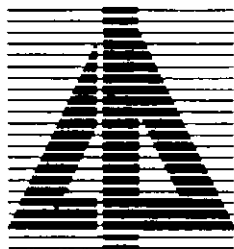
VENDOR ADDRESSES

Company	Contact	Address & Telephone
Alan Pearman Ltd.	Alan Pearman Maria Pearman	96-98 Chester Road, Huntington, Chester CH3 6BT. 0244-46024
APL People	Valerie Lusmore	3 Cross Lane Close, Orwell, Royston, Herts SG8 5QW. 0223-207530
APL *PLUS Ltd.	Barrie Webster John Ward	Aston Science Park, Love Lane, Birmingham B7 4BJ, W. Midlands. 021-359 5096
Attikale Ltd.	Peter Hurdley	51 Brookside, Paulton, Bristol BS18 5YR. 0761-415427
Brent-Cybernex Ltd.	Ken Rumsie	Sovereign House, Dallow Road, Luton, LU1 1TP. 0582-452020
Burroughs Machines Ltd.	M.J. Fennell	Heathrow House, Bath Road, Hounslow, Middlesex TW5 9QL 01-750 1400
CACI Inc.-International	Paul Thornett	Oriel House, 26 The Quadrant, Richmond, Surrey, TW9 1DL. 01-940-3606
Cocking & Drury Ltd.	Romilly Cocking	16 Berkeley Street, London W1X 5AE. 01-493 6172
Datatrade Ltd.	Tony Checksfield	38 Billing Road, Northampton, NN1 5DQ. 0604-22289
Delphi Consultation Ltd.	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. 03677-384
Dyadic Systems Ltd.	Peter Donnelly	30 Camp Road, Farnborough, Hants. GU14 6EW. 0252-547222
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn or Roger Attard	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. 0937-61961 Davenport House, Bowers Way, Harpenden, Herts. 05827-69071

APL PRODUCT GUIDE (continued)

VENDOR ADDRESSES (continued)

Company	Contact	Address & Telephone
H.M.W. Programming Consultants Ltd	Stan Wilkinson	142 Feltham Hill Road, Ashford, Middlesex. TW15 1HN. Ashford 41232
Holtech Ltd.	Jan Bateman	'O' Block 4th Floor, Metropolitan Wharf, Wapping Wall, London E1 9SS. 01-481-3207
IBM UK Product Sales Ltd	Stephen Voller	PO Box 32, Alencon Link, Basingstoke, Hants., RG21 1EJ. 0256-56144
Inner Product Ltd.	James Manning	Eagle House, 73 Clapham Common Southside, London SW4 9DG. 01-673 3354
I.P. Sharp Associates Ltd.	David Weatherby Elizabeth Penrice	132 Buckingham Palace Road, London SW1W 9SA. 01-730 4567
Lynwood Scientific Developments Ltd.	Gareth Wokes	Park House, The High Street, Alton, Hants GU34 1EN. 0420-87024
M.B.S. Rentals	Robert Notley	119/120 High Street, Eton, Windsor, Berkshire. 075-35-68171
Modern Business Technology Ltd. (M.B.T.)	Michael Bramson	P.O. Box 87, Guildford, Surrey GU4 8BB. 04868-23956
Metatechnics Systems Ltd.	Steve Margolis Dave Toop John Stembridge	73b Pelham Road, Wimbledon, SW19 1NX. 01-707-5469 or 01-543-3953
MicroAPL Ltd.	Richard Nabavi	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP. 01-622 0395
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. 02407-2027



COCKING & DRURY LTD.
THE APL PROFESSIONALS

Enhance your APL Information Centre

An enhanced APL will improve programmer productivity — but what if you can't afford to be cut off from future IBM Information Centre products?

Cocking & Drury have a range of powerful enhancements to VS APL that will not commit your company to a non-IBM strategy.

- * Multi-user APL filing system
- * Enhanced report formatting
- * High speed printing
- * Mailbox facilities
- * Execution of non-APL programs from APL

For further information call us on 01-493 6172 or write to
16 Berkeley Street, London, W1X 5AE

BOOK REVIEWS

APL — An Interactive Approach (3rd. Edition)
 by Leonard Gilman and Allen J. Rose
 Published by John Wiley & Sons

Reviewed by David Preedy

I must confess to having set out with some trepidation on the path of reviewing this book, feeling rather like a Sunday-school pupil being asked to review the Sermon on the Mount. After all, Gilman & Rose has been the standard work on APL for a decade now.

However my dilemma about where to begin was eased somewhat by the slogan "Includes APL2 and APL for PC's", which adorns the cover of the new edition. So I decided to see how well the 'Revised Version' copes with these new aspects of the language, and looked up 'APL2' in the index. Finding no entry, I turned hastily to the list of contents, to find the first mention was "More APL2 functions" in Chapter 14. It dawned on me that I would have to browse through the bulk of the book to pick out the new features offered by APL2.

When I did find the descriptions of the new facilities, they were generally understandable but not quite up to the high standards set by the previous edition for 'traditional' APL. I did like the side-by-side presentation of the new function with an alternative way to achieve the same effect; it frequently helped me confirm that I understood the derivation of the printed result.

However the overall effect was of a somewhat hurried incorporation of the new material, with several typographical errors; for instance the 'squad' symbol appeared everywhere as $\overline{R}$ except in the index where it is given as $\downarrow$. Similarly on page 265 the match symbol is described as " $\overline{=}$ overstruck with $_$ " but is displayed as $\overline{=}$ in the subsequent examples. Some of the type-setting could also be improved, as on p209 where we read:

"For say, $+ /$, $+$ is the left operand of the reduce operator $/$, ..."

A few spaces would be invaluable.

The coverage of APL for PC's is minimal; I did find a couple of references to the IBM PC, but the bias of the book remains firmly oriented towards mainframes and printing terminals. For instance, chapter 10 spends 8 pages describing the ∇ function editor including additional variants available under APL2, before a small section on full-screen editors contains a recommendation to use them in preference to ∇ .

The assumptions about the reader's starting level are also inconsistent. The original material (i.e. from previous editions) assumes minimal knowledge, so Chapter 1 ("Getting Started") is an ideal introduction. The new material, however, is overlaid without any warning. On page 13, APL2 is mentioned for the first time without any explanation. Surely a brief review of the main versions of APL available, or at least those explicitly mentioned in the book, is essential for the novice reader.

In summary, I fear that the new material in this third edition fails to live up to the superlative standard set by previous editions, and seems to have been hurriedly tagged onto the existing edition rather than being carefully integrated. Having said that, the new book covers a vast amount of material and is still enjoyable to read. It covers a lot of the less mundane parts of APL, such as operations on multi-dimensional arrays, better than any other book I have come across. It must remain an essential part of every serious APLer's library both as a reference book and for the casual browse through less familiar parts of the language.

Mr. Babbage's Secret
by Ole Immanuel Franksen
Published by Strandbergs Forlag, Denmark

Reviewed by the Author

Charles Babbage (1791-1871), originator of most of the principles behind the modern computer, had an exciting hobby. He broke enciphered messages in the so-called "agony columns" in the Times and other London newspapers. In the British Museum, he studied the correspondence of Charles I and his Queen, Henrietta Maria in order to establish their secret ciphers. For the counsel for the defendant in a lawsuit, he broke an enciphered correspondence, and by his testimony as an expert witness, he decided the outcome of the lawsuit. Under the pseudonym "C", in a public exchange of letters in the periodical "Journal of the Society of Arts", he took up a challenge about an "unbreakable" cipher, discovered the keyword of the challenger, and finally set him checkmate by confronting him, in his own system, with a counter-challenge the solution of which is given here, more than 125 years later.

Some of the coding principles used by Victorian Englishmen for their messages in the agony columns would also have been easily recognized by the Vikings and the Norsemen of early mediaeval times. They were used in the old runic inscriptions, but at that time in order to invoke the Gods and the Evil Powers. Other, and more difficult ciphers can be referred to the times of Frederik II and Christian IV. Especially, this applies to the "unbreakable" cipher in the "Journal of the Society of Arts" which is called "Vigenère's Cipher" after a French diplomat who described it in 1587. The Vigenère Cipher is probably the most famous and most frequently described cipher in the literature. Nevertheless, American authorities emphasize that the only historically correct description was given by I.B. Lindenfels, a major in the army of Frederik VI. This description appeared in the book, *Den hemmelige Skrivekonst* (The Art of Secret Writing), published in Copenhagen in 1819 under the motto: *Ei blot til Lyst* (Not only for pleasure). This book also describes a predecessor of the Vigenère Cipher which was used by one of Christian IV's ambassadors, and at a glimpse, we get from an eye-witness a description of H.C. Oersted's contribution to the subject. A later Danish contribution is the invention in 1875 of a new enciphering system by Julius Petersen, professor of mathematics at the Copenhagen University.

Many famous scientists, and especially mathematicians, have occupied themselves with the cryptographic subject area, either as a hobby or in a professional capacity. The study at the British Library (British Museum) of the papers left by Babbage throws a new light on the motive power behind this interest. Throughout the ages, secret ciphers and codes have constituted the most purified material for the manner in which people communicate. Some ten years before Babbage took up the challenge in the "Journal of the Society of Arts", he succeeded in giving a mathematical formulation of the law, or the principle, on which the Vigenère Cipher was based. This accomplishment made him look upon his hobby as a mathematical laboratory for the experimental investigation of the fundamental properties of data and data operations. This led him into a search for laws for data corresponding to the known laws of nature. He imagined that data connections might be described by means of a new kind of geometry in which quantitative measures, such as lengths and angles, were replaced by qualitative concepts, such as relative position and mutual order. Assisted by his cryptographic investigations, he tried to produce a corresponding algebra, and he planned to write a book on this subject. Like so many of his other ideas of genius, this plan was never realised.

The present book takes up Babbage's idea and challenge. With the secret ciphers as its arena and illustrative material, it extracts over the space of time from the Viking age to the time of the most recent research the interdisciplinarily accepted data concept. Data are facts, and must therefore be discussed in consideration of their procurement by observation or by measurement in the broadest sense. Through the achievements of Einstein and other great scientists, this conceptual area has long ago been tested and prepared for transmission to datamatic application areas, such as industrial production, or administrative systems. The necessary mathematical ideas and tools have been ready for use for more than half a century. What is now needed for an integration of the computer in our educational systems, is not, therefore, new "inventions" but changed choice of subjects, and a new mutual balancing of the subjects. Which subjects, and why they are of importance in the formulation of a geometry for data and data operations, are discussed in a historically inspiring perspective.

The education around the computer not only requires an operational theoretical formulation: to make it clear how the processes must be run, it also requires the creation of illustrative areas of application. Here, secret ciphers would be a good starting point. They can be made just as exciting as computer games; the only prerequisite for their use is that you are able to write and read, and they allow the gradual introduction of a number of mathematical tools. In addition, they present many data theoretical problems of a general nature. How to proceed and build up one's datamatic laboratory is illustrated in the programming language APL. All examples have been taken from the historical source material, and the interested reader will be able to try his or her strength against number of problems in the form of enciphered personal messages from the agony columns of the Times.

SOFTWARE REVIEW

APAT: First Impressions

by Bernard King

IBM has attempted to provide a set of tools to increase the efficiency and ease the maintenance of APL code. The documentation for these tools is adequate but there are errors in the code which need correction before use.

The tools are mostly flexible. All functions are unlocked so the logic used for each tool can be examined and tailored to meet particular requirements. Results are either returned as an APL variable or the name of a print function is provided which can be amended to redirect the output.

Efficiency

Execution times of functions, lines or expressions can be measured using 'PARETO'. The CPU time used in each function is calculated; the analysis is more difficult on large systems as one extra function is added for every function monitored. Our own approach to split CPU usage between functions was to count the number of times various lines were executed. This is less informative than timing each function but takes less CPU time itself, so it is more suitable for a long term check. 'PARETO' would therefore only be useful for a short term check.

Proposed amendments can be compared using 'EXTIMER' to evaluate the saving made; each function is repeated until the variation in CPU usage is within acceptable limits. Our 'TIME' function is a simple version of 'EXTIMER' — it executes the function only once and does not compare the execution times of different functions.

Once the areas where most CPU time is used have been ascertained, further effort can be concentrated on those functions. If necessary the CPU usage can be calculated for each line separately. (An extra function is created for every monitored line so it must be restricted to a few functions only; this could be rewritten more efficiently to avoid excessive function calls). CPU time required by functions called during execution is not included unless the called function is locked or the modifications made to the called functions are reversed. Our 'CLOCKIN' function can be used to monitor only the CPU usage generated by each line.

In addition to execution time, the available workspace required by a function or expression will be calculated by 'IRONMAID'. The point at which maximum usage occurs is identified. In a programme running with restricted workspace, this highlights the worst areas, which will then allow the best balance between CPU time and workspace to be found.

Maintenance

There are several tools which ease maintenance of APL code

'CONSTRUE' executes an expression or line of a function in parts. Complicated lines of code are simplified by showing the order of execution of each line and intermediary results. Understanding code written by someone else is much easier using this tool.

'MONITOR' shows the number of time each line of a function has been executed. A function is added to the end of each line in a way that does not affect execution. This function could be rewritten to record any aspect of the monitored function that is useful. The option to stop monitoring does not return it to its original form so it would be safer to copy the function from another workspace and erase the monitor variables separately.

'BMONITOR' shows every branch within specified functions in order.

'VMONITOR' displays all values assigned to specified variables within specified functions.

The last three of these would be particularly useful for identifying the point at which an error occurs.

In the past, efficiency has been checked using a mixture of formal and informal methods. The IBM set of tools extends the range of checks and makes them easier to use and interpret. Whilst there has not been sufficient need to create these tools for ourselves, now that they are available they will prove useful.

APL TITLES FROM WILEY . . .

LEARNING AND APPLYING APL

by **B. Legrand**, *Ingenieur de l'Ecole Centrale des Arts et
Manufacturers*

Translated by **J.G. Matthews**

A complete reference on APL, this book can be used either as a manual for the beginner or as an aid and source of ideas by the more experienced user. Those wishing to learn APL will find the treatment simple and easy to follow, with useful examples of common situations and without boring mathematical computation.

0471 90243 8

414 pages

April '84

£14.75/\$29.95

APL MANAGEMENT PROBLEMS WITH ANSWERS AND "KIT OF TOOLS"

by **B. Legrand**, *Ingenieur de l'Ecole Centrale des Arts et
Manufacturers*

Translated by **J.G. Matthews**

Complementary to Bernard Legrand's previous work "*Learning and Applying APL*", this clear straightforward book covers all the essential aspects of APL. The problems have been designed from teaching experience and give much practical advice in programming via a tool-kit of answers.

0471 90334 5 approx. 168 pages September '84 approx. £14.50/\$29.00

APL: A Design Handbook for Commercial Systems

by **A. Smith**, *Rowntree Mackintosh, York*

This book describes the use of APL in commercial computing. It aims to show where APL can be most profitably applied, and introduces the tools and techniques to build successful APL systems.

Wiley Series in Information Processing

0471 10092 7

190 pages

March '82

£10.25/\$23.50

All books available from your usual book seller or in case of difficulty from Wiley. If ordering from Wiley please make cheques payable to John Wiley and Sons Limited. You may also telephone your credit card order - dial 100 and ask for FREEPHONE 3477 (UK only)



John Wiley & Sons Limited

Baffins Lane · Chichester · Sussex PO19 1UD · England

RECENT MEETINGS

This section of VECTOR is intended to document the seminars delivered at recent meetings of the Association, particularly for those members who work outside London and often find it hard to spare the time to attend.

We are dependent on speakers for their willingness to provide us with a written version of their seminars, and we would remind them that "a picture's worth 1000 words". Copies of slides and transparencies will enhance their articles.

The Activities officer (details on inside back cover) will respond enthusiastically to offers from individuals to contribute seminars and supporting papers.

APL vs Packages and 4GLs**May 18th 1984, Imperial College, London***Notes compiled by Adrian Smith*

In all 48 people signed the register for this meeting, which consisted of two talks and a panel discussion. The speakers were Dominic Murphy and Des Booth, and the panellists were:

Des Booth (APL*PLUS)

Sylvia Camacho (Tesco)

Dominic Murphy (IPL)

Chris Miles (Independent Consultant)

Dave Wetherby (IPSA)

I have in my possession a tape of the entire discussion, but unfortunately the only audible word is an unattributed comment of 'Knickers!'. However Des Booth has come up trumps with a full text, so the meeting will not go entirely unrecorded for posterity.

E&S
ASSOCIATES**APL****THE
PROTOTYPING
EXPERTS**

Provide a unique service
-software and consultancy-
in this most rapid, flexible
and economical method of
system development.

Contact:**Frank Evans****19 Homesdale Road****Orpington****Kent BR5 1JS****Phone:****0689-24741**

"Are 4th Generation Languages Killing APL?"*by Desmond Booth*

Before answering this question, it is necessary to consider two related questions:

Who uses 4th generation languages?

What is a 4th generation language?

The first question is perhaps easier to answer. The concept of the Information Centre has grown over the last few years, and it is clearly in the Information Centre that 4th generation products will find their home. This gets us no closer to resolving our main topic, however, as we must still ask:

What is an Information Centre?

Why does it exist?

This discussion will first describe Information Centres, in order to understand the growth in demand for 4th generation languages. It will then use a 4th generation product, DATAPORT, to demonstrate 4th generation characteristics by example. Finally, it will show why the use of APL in 4th generation products is an advantage. It will come as no surprise, therefore, that the answer to "are 4th generation languages killing APL?" will be "NO."

It has been suggested by sceptics (*Ed: see Romilly Cocking's article in Vol. 1. No. 1.*) that Information Centres have arisen because IBM have seen the IC concept as a means of selling blue boxes. Whilst it is true that IBM have put enormous resources into promoting IC's, it is now clear that, as usual, they have been backing a winner. During the last twenty years organisations have moved more and more of their corporate information to the data processing environment, and potential users have found it increasingly difficult to cope with the problems of accessing this information. Sometimes this has been because of the sheer volume of data available, at other times the technical difficulties of access have made the time scale of data interpretation too long. Hence the expression "application backlog" became an emotive term in many DP departments.

The Information Centre concept arose, therefore, in order to formalise methods by which non DP professionals (now often referred to as "End Users") could access and interpret this wealth of computerised data. Professionals would be made available to advise and assist where necessary, but the users would use advanced computing tools to achieve results at terminals without the need for intervention by the DP department.

APL was introduced to the Information Centre not, as has been suggested, to fill machines, but because IBM had used two APL products — APL/DI and ADRS — internally to great effect themselves. In fact, considering IBM's vast internal commitment to APL it is surprising that APL is not pushed more into the limelight. Nevertheless, both APL/DI and ADRS found great following in emergent Information Centres, and still remain the most frequently used End User tools.

At the same time that Information Centres using mainframes as the computing vehicle started to emerge, the micro computer revolution occurred. Suddenly people found themselves confronted with computing tools which were easy to use! And productive! Visicalc changed the face of End User Computing, but this was only the beginning. Command driven systems, once considered to be at the forefront of technology, were replaced by sophisticated menu-driven systems, with on-line help facilities giving real power to the users' fingertips, without the need for heavy commitment to learning how to use the systems offered. Mainframe systems fought back, and the term "4th generation" was spawned.

"4th generation" is a term widely used, but rarely defined, since it embodies such a diversity of concepts. Rather than attempt a definition, I will use a new Information Centre product, DATAPORT, to achieve a definition by example.

DATAPOINT is a 4th generation Information Centre tool providing full-screen data entry, display and analysis capabilities together with interfaces to other proprietary Information Centre products. DATAPOINT uses APL as a "core technology", draws heavily on spreadsheet experience to provide a fast, consistent environment for End User applications.

The 4th generation aspects of DATAPOINT can be divided into three main areas:

- user-friendliness;
- functionality;
- interface capabilities.

Looking first at that over-worked expression "user-friendly," what does it mean? The experience of micro computer users is clear. It is possible to "throw" a micro and a good spreadsheet at a user and never hear from them again. Expect for a stream of converts asking for their own PC! Clearly this is a result of user-friendliness, but how is it achieved? Certainly aspects such as menu options and on-line help are part of the answer, but not by any means all. This is why older generation software does not necessarily respond to being tarted up with menus and help — a new, consistent approach is called for. This is achieved in DATAPOINT by using the following:

- combination of command and menu driven design;
- two level help facility;
- clear, consistent error messages;
- full support for 3270 technology;
- selection of options by cursor control;

all of which are incorporated into a truly elegant design. The result is a product which not only encourages beginners to learn without "painful" experiences, but also enables experts to work quickly and efficiently.

Friendliness is fine, but ultimately it is the results which matter. DATAPOINT functionality extends the well developed concept of windowing to the multiple dimensions which are so important for modelling the real world. In combination with calculation facilities which enable formulae to be expressed in terms of rows, columns, or even pages at a time, the multi-dimensionality enables users to rapidly develop applications. It will come as no surprise to experienced APLers that the calculation facilities take full advantage of the APL product core to provide unparalleled power of expression.

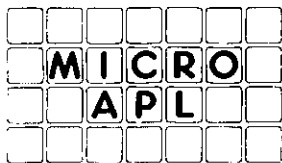
It is in the interface capability, though, that DATAPOINT is furthest ahead. Friendliness and functionality have developed gradually, but here is something new. The capability of "talking" to other software, whether mainframe or micro based, without the user knowing anything about the relevant data structures, is revolutionary. Current interfaces include links to database products including FOCUS and RAMIS; core Information Centre products APL/DI and ADRS; other mainframe based products including SAS and Tel-a-Graf; and simple interfacing to the popular micro spreadsheets including LOTUS and Visicalc.

Having now achieved some clarification of the implications of "4th generation" the advantages of APL in this environment are clear. Whilst the use of APL is of no direct benefit to the End User (in fact it is important that the user be unaware of APL, in terms of upper/lower case requirements, APL characters, or such "peculiarities" as right to left evaluation) the benefits for those supporting the software are tremendous. Even the simplicity of installation of APL, merely requiring the loading of a workspace, is a benefit in terms of the avoidance of the need for that all-too-scarce resource, the systems programmer. But it is in the ease of customisation that the main benefit lies. Frequently this can be achieved with minimal awareness of product internals. Even where the relevant system is not APL based, the use of APL to provide a front

end, or interface to other products in the manner of DATAPOINT, is frequently because of the ease of transforming data structures in APL.

The above assumes that APL expertise is available. But the natural environment for 4th generation products is in the Information Centre, with its historical use of APL. It may well be that APL is the only language supported by an IC; in any case IC's will be well equipped to work with APL products.

It is seen then, that rather than killing APL, the emergence of 4th generation languages, whether APL based or not, will stimulate the continuing growth of APL as the vehicle for the best of End User software.



When it Comes to Consultancy
Experience Matters.

MicroAPL Consultancy with Confidence

MicroAPL Ltd. Unit 1F, Nine Elms Industrial Estate, 87 Kirtling St. London SW8 5BP. Tel: 01-622 0395

APL84 IN FOCUS

June 22nd 1984, Waldorf Hotel, Aldwych

Notes compiled by Adrian Smith

This was very much a 'special event'; there was an admission charge and delegates were asked to register in advance. In spite of (or because of!) this the meeting was very well attended, and a total of 98 bodies were counted.

All 5 talks were repeat performances of papers given at APL84 in Helsinki. I would like to thank Steve Lyus (Imperial Group) and Derek Wilson (Rowntree Mackintosh) for providing me with very comprehensive notes. I have used Steve's comments on the papers by Roy Sykes and John Scholes, and Derek's for the papers of Phil van Cleave, Jim Brown and Jim Ryan.

Before these notes we have included an overview of the event by Dick Bowman — the Association's Activities Officer — who is to be congratulated for bringing such a good representation of the APL'84 Conference to London.

APL84 IN FOCUS

A Commentary by Dick Bowman

The APL84 conference was a splendid occasion, but one which could be attended by only a minority of Association members. It seemed right that we should attempt to do something to bring back some flavour of what went on, whatever news there was, and maybe even catch one or two of the seminal figures of APL in transit.

As it turned out we were fortunate enough to be able to attract four speakers from the USA, one of our very own, and a number of hardware exhibitors.

The detailed notes by Steve Lyus cover the ground of what was said by our speakers and I shall comment briefly instead on the exhibitors and the audience.

Among the hardware exhibits was the Analogic APL Machine; we finally managed to get a full working demonstration of this most impressive piece of hardware, and are grateful to CACI, the UK distributors, for their involvement. Also present was the Ampere lap-top APL micro (MicroAPL can tell you more), which goes a long way towards cutting the entry cost of APL hardware and provides not only APL and ASCII characters, but also Katakana. We were also pleased to have a working NIAL system. NIAL derives from the array theory of Trenchard More; it could be summarised as an English keyword version of generalised APL arrays, although I suspect that it is biased more toward array manipulation and omits a few APL excesses. Dyadic Systems brought along one of their more transportable UNIX machines, and I.P. Sharp generated some interest in their new bargain-basement prices. IBM, unfortunately, did not respond to my suggestion that they'd like to demonstrate APL2 running on an XT/370.

At the end of the day the 100+ attendees had received the word direct from some of the innovators of the APL world and had seen some of the more stimulating new kit in action. On our part, we'd put on a meeting of greater length than our norm, and the first for many years in which we'd stepped outside the environment of Imperial College (Loughborough excepted). We even included a decent lunch. As an event additional to the usual schedule (which is financed by your subscriptions) we were obliged to charge a fee, but one which was considerably lower than the going rate for commercial events. After considerable nail-biting we made a small surplus, which is being ploughed back into further improvements in the Association's services to the membership.

Finally, I'd like to record my thanks to the participating companies. Most of the speakers had delayed their return to the USA; Jim Brown interrupted a holiday in France and flew in for the day specially.

AN APL COMPILER

by Roy Sykes (STSC)

RS started off with a brief history of attempts to speed up the processing of APL Code (both hardware and software). STSC thought that there was much greater potential available from a proper compiler and initiated a research project. They had proved it was possible by June 1983, and have since speeded it up. They aim to release the product from August onwards with a goal of 3 x faster running (the range is likely to be 100x to 0.5x!) for a restricted set of codes.

To the user, a compiled function will be like a locked function and as such can be copied and filed (quad-CR and quad-VR will return the source code). RS made several interesting comments about the different ways they are using to speed it up.

STSC aim to have two products available in the near future:

A compiler support pack (APL * PLUS in Aug 84, VS APL in Jan 85) which will include a set of utility compiled functions.

APL Compilation Service: the user will submit functions to be compiled and they will be returned the following day. (It would appear that the greatest benefit will come to those who code in inefficient loops!)

They hope to have proper software products available around January 86 (When it arrives, will the mainstream D.P. establishment welcome us back as the prodigal son?)

DYALOG APL

by John Scholes

JS went through in some detail all the extra goodies available on their Unix-based system.

e.g. System Utilities (arbitrary Input/Output).

The use of a hybrid system — APL plus compiled bits; these compiled bits being auxiliary processors. They would appear to the user as a locked function and would occupy negligible space in the W.S., but perform their tasks very quickly. Examples given were of a full screen editor and formatted screen I/O.

Bridge systems — connections allowed between the APL W.S. and a relational databases or graphics routines written in other languages.

THE AMPERE: A PORTABLE APL MACHINE

by Philip van Cleave

"If I have to think for 4 seconds, that is equivalent to using all the processing power of all the computing resources in the world!"

Philip van Cleave started his presentation by giving some background to 16 and 32 bit micros and portable machines. He said that 16/32 bit micros had given opportunities to try ideas and approaches which would not have been possible before. One can now have massive memory and real storage (which makes processing much quicker) which in turn has opened up new applications.

He then proceeded to quote various buzz-words and desirable features (access to other people's data, "user-friendly", "integrated office") and the implication was that the product he went on to describe had all these attributes. "APL is one place where we can really hit the (portable computer) market."

The portable APL machine has a MC68000-based interpreter providing 1 megabyte of storage. It also has a built-in modem which gives it the capability to work with other devices.

Philip van Cleave stressed the power of the machine, which resulted from the fact that there are two APL processes running simultaneously internally. Data and commands could be passed backwards and forwards between them.

The machine included many useful features, such as the ability to record and pass back voice patterns, being able to see the operating system from within APL, and restarting from the point of interrupt in the event of a power drop.

All in all it was a somewhat loosely structured presentation, but the claims made were interesting. His main theme was that the portable APL machine allows APL users to develop software for the mass market.

APL2

by Jim Brown

Jim Brown's talk consisted of an introduction to APL2 and a brief discussion of some of the features. He was very enthusiastic and had a naturally attentive audience. APL2 had been announced in Germany the week before and this, he said, was an indication of IBM's regard for the European market.

APL2 on IBM would run under TSO and CMS and utilizes 32-bit architecture and therefore allows up to 128 Mbyte workspaces. IBM's intention was to use APL2 rather than VS APL for any further enhancements of APL and he said "It is our opinion that APL will remain the base of the Information Centre products."

He commenced by talking about general Relational Data and Structured Query Languages (SQL). Relational Data tables have columns with data of a common identity and rows are different entries. SQL allows data to be retrieved from one or more tables, the ability to create new tables, new columns, and temporary tables. SQL is a subset of an APL2 array. He also talked about ISPF under APL and features such as a menu with APL options; calling other applications from within APL (e.g. PL/I programs); and general recursive routines.

The rest of the talk was a brief summary of APL2 and the new features. There are lots of new primitives and the existing ones can be combined in a number of extra ways. The inner and outer products have been extended to general functions; and the nested arrays and "Each" operator virtually do away with any looping (but also lead to some very contorted code). APL2 naturally leads to recursive functions.

APL2 can accommodate a "LISP"-type structure and has other arithmetic which is very useful for graphics applications.

There are a few minor differences which makes APL2 not quite upwards compatible, e.g. the display of nested arrays has meant that conventional arrays of many dimensions are displayed differently. Another change is that many of the errors now return results, which should make error-trapping easier.

The talk was interesting, but obviously the new features, and the pros and cons, would have to be looked at in great detail to assess the real benefits.

The APL Machine

by Jim Ryan

Jim Ryan's talk and demonstrations rather stole the show and everyone was very impressed by the power and versatility of the product. It was, he said, "a processor of arrays integrated with a language of arrays".

The hardware configuration is best understood by reading the Analogic product literature. Basically there are three active systems: a work station, a peripheral system and an array system; and the way these work together gives very powerful processing and data recall. He said the operating times were comparable with an IBM 3081.

There is a virtual workspace of 850 Mbyte (which can go up to 1 Gbyte), i.e. up to the memory limits of the whole machine. There is no filing system (but does there need to be?).

Many of the software features are very impressive. The APL complies with ISO APL but there are layers of extensions. The workspaces are organised in a hierarchy, and this threaded workspace allows you to make components of a workspace available to others. The overall result is that it is possible to have a complete data-base of functions with the ability to make universal changes but with local versions and various security options also available.

There are also many exception management options which make validation and error-control easy. The overall result is a powerful, versatile machine which can be multi-user and allows multi-tasking (screen windowing is a key feature).

Analogic hope to have the lowest level of the system (ISO APL) available in September 1984.



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

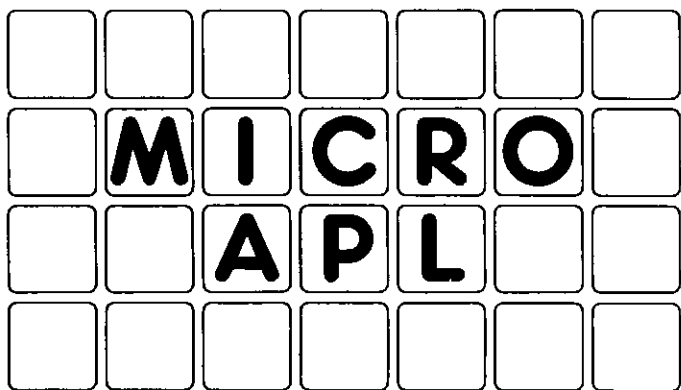
Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL nor may be interested in learning it. However, we hope that you are curious about why, under the right conditions, such impressive results can emerge so quickly from APL programmers.



where APL means business

You know what's good about micros. Fast response; no monthly bills; software designed for users, not programmers; access whenever you like. In a word, independence.

You know what's bad about some micros. No networking; no access to your mainframe data; fewer features than you're used to; no technical and service backup. In a word, isolation.

Wouldn't you like to see a full range of APL micros, from the IBM PC to large multi-user machines? With local area networking, and IBM and ICL terminal emulation? With full lookalikes of features like IBM's AP124 and AP126? All from a company with the commercial and technical muscle to support everything it sells?

MicroAPL Limited

Unit 1F, Nine Elms Industrial Estate
87 Kirtling Street, London SW8 5BP
Telephone: 01-622 0395 Telex: 896885 IOTA

MR. BABBAGE'S SECRET: A PRESENTATION

by Ole Immanuel Franksen

When Charles Babbage died in 1871, the 'Times' remarked in his obituary that he had lived to be almost 80 "in spite of organ-grinding persecutions". A prolonged opponent of street musicians and other noise pollution was how society remembered him — the genius who declared, looking back upon his life: "The great object of all my inquiries has ever been to endeavour to ascertain those laws of thought by which man makes discoveries".

Today, Babbage is recognized as the great pioneer of the digital computer. Modern electronics has made his dream come true. To appreciate the significance of this evolution, let me draw a historical analogy with the introduction of logarithms, one of the first practical means for facilitating more advanced computations.

In 1627, after years in the treadmill of arithmetic, Kepler finally published the 'Rudolphine Tables'. Based on the life-long labour of Tycho Brahe, these tables also contained logarithms, for the first time put to astronomical uses. Kepler became acquainted with Napier's logarithms some ten years earlier, and he wrote enthusiastically: "A Scottish baron has appeared on the scene (his name I have forgotten) who has done an excellent thing by transforming all multiplication and division into addition and subtraction...". This enthusiasm did not please Maestlin, Kepler's old professor, who rebuked him: "It is not fitting for a professor of mathematics to manifest childish joy just because reckoning is made easier."

Since Napier did not at first explain the underlying principle, Maestlin's remark was not without merit. The potential of technological advances is in the novel principles they uncover. Babbage's perhaps most successful book: 'On the Economy of Machinery and Manufactures', support this thesis. Published in 1832, in the midst of the industrial revolution, it is hailed today as a scientific landmark in economics as well as in manufacturing engineering. In it, Babbage carried the concepts and abstract models of description from the world of natural phenomena into the realm of man-made systems. He said in its preface:

"Having been induced during the last ten years, to visit a considerable number of workshops and factories, both in England and on the Continent, for the purpose of endeavouring to make myself acquainted with the various resources of mechanical art, I was insensibly led to apply to them those principles of generalization to which my other pursuits had naturally given rise".

We are also in the midst of an industrial revolution, brought about by the modern computer. Within a few decades, we have witnessed a rapid change from electromechanical relays to vacuum-tubes, transistors and complex chips. Yet Ohm's and Kirchhoff's laws have not changed. Independent of the technological stage, they have ruled the electrical behaviour of the hardware. The means and the emphasis have changed, but the basic principles still hold.

What a contrast to the development on the software side! From the very beginning, we have identified the aim of our teaching with programming — the dull drilling of current technology. Over the years, the adherents of each individual programming language have fought, in the words of Oehlenschläger, the nineteenth century Danish poet, for each their own "little Hassan with the crooked legs".* Notwithstanding that education is simpler than re-education, it is re-education which has carried the day. For we have taken no heed of Babbage's advice to apply "those principles of generalization to which my other pursuits had naturally given rise".

These other pursuits suggest that we are facing a two-fold problem in our teaching for the future. We need to distinguish between:

- (1) A theory, giving the principles or laws of data and data-operations.

*Ed.- The cited work will be too well-known to our readers to merit formal reference.

- (2) An art, establishing the empirical guide-lines and preferred standards for systems design.

To recognize this dichotomy between laws of data and guidelines of systems design is of vital importance; for laws of data and data-operations are independent of the changing technology, whereas the art of systems design is not. Both problem areas present formidable challenges which must be met here and now, since each in its way demands adjustments, changes, or even reorientations in our curricula. Note, though, that I am not talking about the development or invention of entirely new ideas or principles. Because, as I see it, the inventions are made, tested and documented in the traditional sciences. What we need to do is to collect them, adapt them and bring them down from the level of the highly-trained specialists.

Piet Hein,* my fellow countryman who pins down truth by humour in his "Grooks", once said that humour cannot be defined: — for those who were likely to define it, had no sense of humour. Data is another familiar concept which escapes a rigoristic definition, at least in the scientific sense. The mathematician Gödel proved that in any logically consistent system, there had to be some undefined terms basic to the system. The notion of data is one such basic term common to the various sciences. However, since Galileo the important thing has been, not definition nor explanation, but measurement. For something to classify as a measurement, it must be verifiable, and hence repeatable. In other words, we must be able to identify certain invariant properties or relationships.

The notion of data is closely related to the scientific concept of measurement, since observation and the recording of observations are the only link between the real world and a corresponding theoretical model. Yet, the concept of measurement is strangely absent from our discussion of data and their handling by computers. The fact remains, though, that some of the most important properties of data derive from the invariant scale-form underlying the associated measurements. Common scale-forms are defined across the disciplines, from psychology and sociology at one end to physics and engineering at the other. It lies in the scale-form that it is meaningful to subtract calendar years, but not telephone numbers, and that it is meaningless to take the ratio of calendar years. In fact, scale-forms help us to classify data-operations.

But data are more than mere recordings of individual measurements. I am thinking here of the relationships between data, the so-called data-structures, which always depict certain invariant properties of the system under consideration. The computer is merely a tool whose justification lies in its application for solving problems. The data-structure "measures" the interconnections or interrelations in the applications. Lists, tables, boxes and nested arrays are geometrical representations of these systems properties. Geometry is the fundamental model of description in any scientific discipline, so it would be natural to expect that there should also exist a geometry of data. Indeed, there does; and APL illustrates part of it.

The geometry of data differs from the classical geometries on two accounts. Its spaces are finite, and its binding property is not distance but well-ordering. That is, every non-empty subset has a first element. Certain properties of these subsets, that I might perhaps call data-spaces appear strange at first, because common-sense notions of the conventional metric space break down, such as the usual conception of dimensionality. However, since the geometry of data satisfies Felix Klein's definition of a geometry, put forward in 1872 in his famous Erlanger Program, this formulation paves the way for the enunciation of laws of data, as we use the APL notation to push this geometry into an associated algebra.

The laws of data are concerned with preservation of properties under various operations.

*Ed. — Here is an example of one of Piet Hein's Grooks:

"It is sad to lose a glove, but not nearly so sad as losing one glove, throwing away the other, then finding the first again"

They do not compare with, and they are essentially different from the well-known natural laws of Newton, Hooke, Ohm, Kirchhoff, and all the other classical physicists. Yet, analogies to the data laws, classified mathematically as algebraic invariants, are abundant in modern physics and chemistry. The power of adopting this approach is that we are now able to introduce laws in various areas of application: production engineering, management, administrative systems, and modern control theory, for which it has not been possible to formulate laws in the axiomatic sense of classical physics. To illustrate, the bill of materials, the cost aggregation of an assembly, the scheduling of processes, and the state-equations of a control system, may now be explained as similar, so-called isomorphic, data-structures.

Thus, as I have tried to indicate here, a new perspective has opened up towards a geometrical theory about laws of data and data-operations. This theory is in the mainstream of all the other sciences and their areas of application. It is teachable and testable in the conventional scientific sense, and it is essentially independent of the stage of current technology.

Let me now turn to the other problem area we have to face in our education, namely the art of systems design. The building-up of systems is encountered in many disciplines. Still, it has been cultivated in its purest form in engineering, for the essence of this topic area is design. Consequently, many of the basic ideas and approaches to be used in connection with the computer may be introduced by emulation of the time-honoured engineering approach.

However, for our universities and high-schools to adopt the spirit and attitude of design is perhaps the most difficult challenge in our time. The training of our teachers has been geared towards analysis of existing systems emphasizing problems with a unique solution. Now, the computer will force us to complement this type of education with the experimental or cut-and-try approach so characteristic of the design process. We have to emphasize the scaffolding, how to solve multiple-answer problems, and to establish criteria for evaluating the feasible solutions.

Of course, what is important here, is not the actual design, but rather the development of an attitude in the mind of the student: an intellectual courage and certain rather formal working habits. What makes this such a perplexing task, however, is the fact that the student does not command any discipline well enough to actually solve a meaningful design problem. Faced with a series of often trivial and usually uncorrelated problems, the attention is focused on all kinds of side issues, so that when the first excitement of using a computer is over, the desire to learn has become a desire to leave.

What is needed, therefore, is an area of application requiring no or very little prerequisite knowledge, yet complex enough to admit of meaningful or even exciting problems. Also, it should be possible gradually to introduce a rich and inspiring variety of mathematical solution techniques emphasizing the geometrical view on data and data-operations.

This problem as well as the other challenges I have mentioned in relation to the computer education in our universities and high-schools, have much occupied my colleagues in FinnAPL. The excellent 'FinnAPL Idiom Library' which they published in 1981, is a tangible result of their thoroughness in this endeavour. So, naturally, during a visit to Helsinki a few years ago, I found myself in the midst of a heated discussion on this vitally important educational issue. My Finnish colleagues were so enthusiastic that, in the spirit of the moment, I somewhat rashly accepted their challenge, later politely formulated as an invitation, to write a book showing how Babbage's cryptographical hobby could be brought to bear on the teaching of computers. This is the book 'Mr. Babbage's Secret' which, thanks to the gracious support of IBM in the Nordic countries, has been published on the occasion of this conference.

Newton said that if he had seen further than others, it was because he had stood on the shoulders of giants. He immersed himself in the past for the very same reason that the engineering constructor digs into the ground — to secure the future. The more powerful the tool, the greater its potential possibilities, the more we need to have perspective and inspiration. Consideration of the great scientists of the past provides this longer view. From their work on the kind of problems confronting us now, we can learn, to quote Alfred Whitehead, "to see what is general in what is particular and what is permanent in what is transitory". It is this aspect that makes history fascinating. It is here we may ascertain, as Babbage expressed it, "those laws of thought by which man makes discoveries".

This historical perspective brings us round a full circle to the starting point of Charles Babbage's early work on computers. What he and his contemporaries wanted, and what the modern end-users want, is to use the computer as a problem-solving tool. Their interest and effort are in the application. They have no desire to become computer specialists in order to transform their problem formulation into a form acceptable to the computer. Instead, they want the communication with the computer to take place in a form which is natural relative to the problem at hand.

For two reasons in particular, APL represents a breakthrough in this direction. One feature is the reintroduction into computer programming of the fundamental fact that, basically, people perceive data as geometrical objects. The other feature is APL's elegant and simple mode of interactive communication, offering the user the opportunity of gradually improving his design by experiments. It is noteworthy how well these two features meet the two basic educational needs — in the area of a geometrical theory of data, and in the art of systems design.

I do not intend here to go into the actual contents of my book, except to say that it is centered around the notion of laws of data and the use of the computer as a workbench. APL is applied as a mathematical notation as well as a programming tool to make the approach operational. In fact, some thirty user-defined APL functions are documented in the book and, with a little patience, everything I have done, can be done in a 64 K workspace. Half of the figures are in APL, so together with the terminal sessions APL occupies slightly more than one quarter of the total number of pages. Yet it is no textbook nor an introduction to APL. It is a tale based entirely on historical facts, many of which are published here for the first time.

When Hans Christian Andersen had written his first tales, he turned to the physicist Hans Christian Oersted for advice, whether or not he should publish. Oersted was enthusiastic, and thus became directly responsible for launching Andersen onto his life's work. What a strange combination: that the discoverer of electromagnetism should counsel the poet on tales for children. Perhaps it was not so strange after all; for Andersen's tales have that special quality, often lost in translation, that they are ambivalent. Submerged in each tale for children there is also one for adults. Undoubtedly, this was what fascinated Oersted.

Babbage's preoccupation with secret writing exhibits a similar trait: an exciting hobby and a fascinating scientific work. This similarity made any other form than the tale inconceivable. In Babbage's endeavours there would be something to inspire teacher and student alike. For he had fun.

In my introduction, I touched upon Babbage's battle against street-musicians. Let me therefore conclude by recounting a true episode from one of the times Babbage had hauled an offender before a magistrate. The latter asked Babbage whether a man's brain would be injured by listening to a hand organ, to which Babbage retorted: "Certainly not, for the obvious reason that no man having a brain ever listened to street-musicians".

Are we listening, or are we just playing it by ear?

XPL: AN EXPERT SYSTEMS FRAMEWORK IN APL

by Robert Bittlestone

It is hardly possible to pick up a computer journal today without seeing references to the Alvey project in the UK, the Esprit project run by the EEC, the Japanese "Fifth generation" initiative, the proliferation of knowledge-based computer companies around Stanford in California and MIT in Massachusetts, and the large amounts of money that are being poured into the development of expert systems. The references abound: as I write this article the September 1984 issue of "International Management" (1) arrives on my desk with a cover depicting a digitised human brain image on a computer screen and a caption saying "Artificial intelligence: the race to make it work for managers". I like the use of the word "race" here; it seems to imply that this is our last chance now that the prospect of human intelligence in managers has declined asymptotically to zero.

Among the savants cited in the International Management article is Sir Clive Sinclair, no less, who has become notorious with the British Computer Society's Expert Systems Specialist Group: notorious for arriving unexpectedly at meetings and giving the rest of the audience something to think about. Sir Clive apparently said in the interview:

"Once 'machines of silicon' surpass us, they will be capable of their own design. In a real sense, they will be reproductive, and silicon will have ended carbon's long monopoly. And ours too, I suppose, for we will no longer be able to deem ourselves the finest intelligence in the known universe."

Maybe expert systems will dominate us after all — it's just that there'll be a ten-month delay between quoting your Visa number and becoming an inferior intelligence.

All this is quite fascinating since the phenomenon is very recent: before 1983 the phrase "expert systems" was largely unheard of and certainly not in itself viewed as the potential salvation of the Western world (there are as yet few hints of Eastern bloc development apart from in Hungary, where the University of Budapest, along with those of Marseilles, Edinburgh and London's Imperial College, claims to have invented PROLOG). However, by now there has been "Horizon" television coverage of the genre and there seems little doubt that expert systems are here to stay. But before I take up the theme of my title, perhaps I might be allowed to state my position on one or two related matters. First of all:

"Expert systems are not the same thing as artificial intelligence"

We can see the beginnings of a bandwagon effect here in which technical namedroppers, bored of stating that "UNIX is the coming thing" or "The future of computing lies in networking" are catching up with this latest trend: "Life is just another form of domain specialism". Expert systems are in fact a very narrowly defined class of computer program with certain in-built inference-drawing capabilities. Artificial intelligence, on the other hand, is a much wider field involving the generic search for synthetic rational life. Much of the most interesting work in artificial intelligence does not involve digital computers at all but concerns the possibility of producing artificial models of brain process, using special fabrication techniques to emulate neurons, axons and synapses. Which brings me to my second definition:

"Artificial intelligence will not be implemented on digital computers"

It turns out that the use of a digital clocked device, irrespective of speed, imposes a fatal limitation on the sophistication of a genuine search for artificial intelligence, and parallel processing makes no difference. The reason is that in the brain it is the dynamic connections between at least 10 to the power 10 asynchronous independent processing elements that provide the kind of complexity that is required. The processors in the brain are called neurons and the interconnections are called axons, and there is substantial evidence to indicate that to a first approximation, almost any neuron can grow an axon to connect to almost any other neuron.

Now the number of connections between 'N' elements increases in proportion to the square of 'N'. If you have three neurons you can connect A to B, A to C, B to C, and the same in the other direction, which is valid since it corresponds to a negative signal (inhibition vs. excitation). Consequently we have 6 possibilities for the network topology: ie 3×2 . For 4 elements we have 12 options: 4×3 . If we had a mere 100 neurons the result of 100×99 is still only about 10,000. However, in the brain we are considering about 10,000,000,000 neurons. So the number of possible connections is 100,000,000,000,000,000,000 — a number of literally astronomic proportions. We haven't yet introduced the complexity of neurons which connect to more than one other neuron (the majority do), nor signal processing within the neurons themselves (estimated as a seventh order differential equation), nor chemical effects at the synapses, which are retransmission points along the axons. When you drink too much alcohol, it is primarily the synaptic effects which alter your responses. I am unconvinced that there yet exists an artificially intelligent program capable of intoxication.

If you now consider the implications of asynchronous (but not parallel) operations taking place between arbitrarily connectable elements on this scale, you begin to see that even a billion 64-bit microprocessors would make no difference at all. The emphasis in the brain is on analogue circuits in which feedback and frequency of signal are the determining factor and where there is no single location at which processing can be said to be taking place; the whole of the brain is in this sense intelligent.

When I asked the Alvey representative at a recent meeting of the Expert Systems Specialist Group (yes, Sir Clive was there too) why there was so little emphasis on neurophysiology in the Alvey programme (I was uncharacteristically diplomatic as in fact there is none at all) his response was that the programme concerned the "hard" sciences and engineering: the bridge to biology was too narrow for engineers to cross. I believe he was genuinely rueful. However, if your appetite is in any way whetted for "real" artificial intelligence, you had better go out and buy two rather extraordinary books. One of them is, of course, Hofstadter's "Godel Escher Bach: An Eternal Golden Braid" (2). The other is less well-known: William T. Powers's "Behaviour: The Control of Perception" (3).

In Hofstadter's book the notion of recursion is explored in many delightful ways. For those who have missed it, here is a sampler. Our heroes, Achilles and the Tortoise, have found a lamp. They rub it and a Genie appears. The Genie will grant three wishes. This prospect is greeted with glee by Achilles, who has read the 'Arabian Night' and knows the form. He makes his first wish:

Achilles: "I wish that I had 100 wishes instead of just three."

Genie: "I am sorry, Achilles, but I don't grant meta-wishes."

Achilles: "I wish you'd tell me what a 'meta-wish' is."

Genie: "But that is a meta-meta-wish, Achilles, and I don't grant them either."

But after some pleading, the Genie agrees to try (meta-wishes are his favourite wishes too) and from inside his cloak he produces a lamp. He rubs it and out pops a MetaGenie:

Genie: "I wish for permission for temporary suspension of all type-restrictions on wishes, for the duration of one typeless wish."

MetaGenie: "Half a moment please (produces a lamp, rubs it, etc.)."

This continues for a short infinite period until we get to the top-level MetaGenie, also known as GOD, who grants permission for the typeless wish:

Achilles: "I wish my wish would not be granted!"

(There follows a SYSTEM CRASH: or as Hofstadter puts it, "the land of dead hiccoughs and extinguished lightbulbs").

Earlier, when quizzed on what the word GOD means, the Genie explains that it's an acronym — G.O.D. — standing for "GOD OVER DJINN" (Djinn being a generic type of intermediate Eastern Deity). Does this help? Ah yes, because we can expand it:

GOD = GOD OVER DJINN
 = (GOD OVER DJINN) OVER DJINN
 = ((GOD OVER DJINN) OVER DJINN) OVER DJINN

.....

If this is starting to look familiar, it's because it is. A deep knowledge of the relation of a system to its metasystem and of the closely connected notion of recursion is an essential pre-requisite for serious work in artificial intelligence. In fact I suspect that:

"Artificial intelligence = recursion + superficial incompetence"

What about the other book I referred to: "Behaviour: The Control of Perception" (or as they write it in the USA, "Behavior: The Control of Perception")? In my estimation this book ranks with Norbert Wiener's "Cybernetics" (4) and Stafford Beer's "Brain of the Firm" (5): together they form the twentieth century triumvirate in this field. Powers starts off with some deceptively simple experiments and ends up with a necessary but not sufficient model of human intelligence based on first principles and an appeal to reason. The argument is convincing and is closer than any other I have read to the true core of artificial intelligence. Significantly, I don't recall in the book a single serious mention of digital computer programming.

Powers explains that the basic circuit elements in organic brains are not digital at all. They are analogue, but they use frequency not voltage as the yardstick. If 10 neural impulses per second flow in an axon, that may cause (at the lowest level) a certain amount of contraction to be applied to a single muscle fibre. 20 impulses per second might double the contraction force. However, the key point is the perception of all this. Feedback from local sensors is compared against a reference level for the desired state, and the output signal is adjusted accordingly. This compensates for unexpected resistance to the muscle's work — and if you think about it, you've never lifted a weight by applying linear pressure in your life. Literally millions of low-level frequency sensitive feedback circuits create what Powers calls Level 1 — the interface to the outside world at the sensory/motor level.

Then we start to encounter hierarchy. Where do the reference levels for the desired states come from? Well, these are the outputs from Level 2, but this time they correspond to a higher-level notion: like that of vertical vs. horizontal movement. By positing a hierarchy of feedback circuits whose outputs become the reference levels (i.e. control signals) for lower level circuits, Powers will lead you to agree that when you are walking at 2 miles per hour, there is an identifiable, traceable neural frequency in your brain which doubles in frequency when you walk at 4 mph. And if you find that hard to swallow, by the time you reach higher levels of hierarchy we are talking about frequencies which correspond to degrees of emotion. Of hate. Of love. To sleep, perchance to dream... "Do Androids Dream of Electric Sheep?" (6).

I suspect I have strayed somewhat from the title of this article. But I do think it's important to set the context. If you want some fun with computers, start playing with expert systems. If you want to learn something useful about artificial intelligence, start reading some books. The two are not the same and I personally doubt that insights from one much help in the assimilation of the other.

Perhaps we should focus on the main theme. This gives me a small problem since the arbitrary reader of this article exists in four configurations: you know about APL (or you don't) and you know about expert systems (or you don't). Here I am going to pitch the discussion slightly in favour of someone who knows a bit about APL but nothing about expert systems, although I hope there will be some comfort for the others here and there.

Let's recap on APL's strengths in this area. We have a language which supports recursion with no limits apart from workspace size. Local variables and arguments to recursive functions are properly masked and pushed to a stack on each recursive invocation. Logical operations in APL are generally implemented at data word bit level and are very comprehensive and very fast. Arrays of arbitrary dimension are supported and most primitive operations apply element-wise without looping. APL is a threaded interpretive language in which individual building blocks are defined and can be invoked in a free-standing manner (like LISP). If LISP suffers from parentheses, APL suffers from silly specialist symbols, but fortunately this last aspect is starting to disappear in the more enlightened implementations. APL is very good indeed at numeric computation of high complexity in both the scientific and the commercial domains, and in addition it is fast and elegant at string handling.

Standard APL disallows non-orthogonal arrays and those of mixed character and numeric type. This is undoubtedly a disadvantage for expert systems work but is overcome in APL2. However, I haven't used APL2 seriously yet, like most other APLers I suspect, and shall confine my remarks to traditional (or as my American friends would put it, "vanilla") APL. So the question becomes:

"What has PROLOG got that APL hasn't?"

The answer, surprisingly to APL aficionados, is: quite a lot. However, perhaps surprisingly to PROLOG programmers, it turns out that one can code all the standard PROLOG backtracking and similar logic capability into APL user-defined functions (ie. into a subroutine library). You then end up with an APL workspace oriented towards the development of expert systems, but with the great advantage of carrying with it all the "hooks" and existing facilities which have been developed in APL over the last 20 years.

Evidently PROLOG users don't mind that they have no proper filing, no error control, no serious peripheral control, no multi-process synchronisation, no inter-user communication, no user or file security, no computational intrinsics beyond add, subtract, divide and multiply, no interpreter available on IBM mainframes, no links to COBOL and other languages, no screen control, no graphics capability, no links to devices like "mice"... It makes it fun to develop expert systems in PROLOG, but awfully hard to sell them. So far I keep seeing demonstrations of PROLOG expert systems telling me how to water my houseplants, but the florist down the road isn't selling the package yet.

Let's look at the framework of an expert system in APL terms. This path has been trodden before (7) but I'd like to look at the problem in a slightly different way. To make it more practical, let me tell you about an expert system I have been writing and playing with. It's called GENESIS 4. This sounds like a reasonable name for a package: GENESIS Release 4.0; but actually this is to do with the Bible (8). Not that I am particularly religious by nature; it's just that the Bible predates "Behaviour: The Control of Perception" as a functional specification of humanity by about 5000 years, so it's of interest to all artificial intelligence buffs.

All expert systems have three chunks.

The SITUATION MODEL is a whole load of information about (in this case) Adam's descendants, their marriages, their progeny and the main events in their lives.

The KNOWLEDGE BASE is a set of rules which indicate the laws of genealogy. For example, "A is offspring of B if B begat A" and so on.

The SYSTEM MANAGER is a set of programs that allows you to explore the situation model, ask it questions, and ask it to draw conclusions: for example, "Is Z descended from A?". Clever system managers sometimes try to propose rules by looking for patterns in large amounts of data in a situation model.

That's all.

What I will do now is re-write parts of Genesis Chapter 4 as if I were entering lines into an APL-based expert system, trying to keep it as much like PROLOG as possible. The numbers in the margins are the verses.

1. "And Adam knew Eve his wife; and she conceived, and bare Cain, and said 'I have gotten a man from the Lord'."

ADAM IS-MARRIED-TO EVE
CAIN IS-OFFSPRING-OF ADAM
CAIN IS-OFFSPRING-OF EVE
CAIN IS-SEX MALE

2. "And she again bare his brother Abel."

ABEL IS-OFFSPRING-OF EVE
ABEL IS-SEX MALE

8. "Cain rose up against Abel his brother, and slew him."

CAIN KILLS ABEL

16. "Cain went out from the presence of the Lord, and dwelt in the land of Nod, on the east of Eden."

CAIN INHABITS NOD
NOD IS-EAST-OF EDEN

17. "And Cain knew his wife, and she conceived, and bare Enoch: and he builded a city, and called the name of the city, after the name of his son, Enoch. And unto Enoch was born Irad: and Irad begat Mehujael: and Mehujael began Methusael: and Methusael begat Lamech."

CAIN IS-MARRIED-TO MRS.X
ENOC IS-OFFSPRING-OF CAIN
ENOC IS-OFFSPRING-OF MRS.X
CAIN BUILDS-CITY-CALLED ENOC
IRAD IS-OFFSPRING-OF ENOC
MEHUJAE IS-OFFSPRING-OF IRAD
METHUSAE IS-OFFSPRING-OF MEHUJAE
LAMECH IS-OFFSPRING-OF METHUSAE

There's a lot more of this sort of thing. I know that the Bible is now available in compressed text form on floppy discs as "THE WORD Processor" (9) but I'll wager that the logical inferences remain unencoded.

What can we do with APL so far? Well, we need to encode this information into APL data structures. There are many ways of doing this, of course. An obvious approach is to write a little parser which takes statements like LAMECH IS-OFFSPRING-OF METHUSAE and divide them into their different text strings (3 in this case). We start building a text matrix with a unique instance of each name. When a statement comes in, we look up the row number of each string or we add the new string to the end of the matrix. So the sequence:

MEHUJAE IS-OFFSPRING-OF IRAD
METHUSAE IS-OFFSPRING-OF MEHUJAE
LAMECH IS-OFFSPRING-OF METHUSAE

would be represented by these two matrices:

MEHUJAE	1	2	3
IS-OFFSPRING-OF	4	2	1
IRAD	5	2	4
METHUSAE			
LAMECH			

Actually it's slightly trickier than this as we shall need a flag bit to indicate which entry is a relationship and which is a noun, and they aren't always binary relationships, and the arguments aren't always on each side. But the basic idea's the same. A simple parsing function in APL is about 5 lines of code, and if we call it ADD it really does look like PROLOG.:

```
ADD 'LAMECH IS-OFFSPRING-OF METHUSAE'
```

So far, so trivial, but we've done something epoch-making: we've defined our first function in XPL, which is the name I have chosen for this set of functions, written in APL but using ASCII character set throughout. Now we want to interrogate the database:

```
"Is Lamech the offspring of Methusael?"
```

```
DOES 'LAMECH IS-OFFSPRING-OF METHUSAE'
```

```
TRUE
```

```
"Is Irad the offspring of Mehujael?"
```

```
DOES 'IRAD IS-OFFSPRING-OF MEHUJAE'
```

```
FALSE
```

Funny syntax, isn't it? I'm trying to stick to PROLOG though, and this is it. Now is it mindbogglingly hard to code this in XPL? Not really: function DOES looks to be at least one line long: it has to find a particular row of integers in the matrix. Of course things get tougher in PROLOG quite quickly:

```
"Who is the offspring of Methusael?"
```

```
WHICH 'X:X IS-OFFSPRING-OF METHUSAE'
```

```
LAMECH
```

```
"Who is the offspring of Lamech?"
```

```
WHICH 'X:X IS-OFFSPRING-OF LAMECH'
```

```
NO MATCH
```

This time not only do we have to find a particular row of integers in the numeric matrix, we also have to work out who X refers to and find him in the list of names. Fiendish stuff. Still, I think we can just about cope. Let's move onto the knowledge base. Here's some useful jargon. You and I are "knowledge engineers" in the context of our translation of the Bible into XPL format. Priests are "domain specialists": Heaven and Hell is their domain. To this I would add that people who create expert systems languages, as we are doing at this moment, are presumably "metadomain specialists" (I believe this is an original coinage).

It gets boring telling XPL everything twice. For example:

```
ADD 'LAMECH IS-OFFSPRING-OF METHUSAE'
```

```
ADD 'METHUSAE IS-PARENT-OF LAMECH'
```

How about defining a rule instead?

```
ADD 'X IS-PARENT OF Y IF Y IS-OFFSPRING-OF X'
```

and while we're about it:

```
ADD 'X IS-OFFSPRING-OF Y IF Y IS-PARENT-OF X'
```

Not too hard to handle either is:

```
ADD 'X IS-HUSBAND OF Y IF X IS-MARRIED-TO Y AND X IS-SEX MALE'
```

Now try:

```
ADD 'X IS-GRANDCHILD-OF Y IF X IS-OFFSPRING-OF Z AND Z IS-OFFSPRING-OF Y'
```

We've introduced the concept of "place markers", which are analogous to the local names of the arguments of functions, except that the functions are relations (data) rather than programs. There are many ways to incorporate these into XPL and your own ideas will undoubtedly be better than mine: I use negative numbers in the integer matrix to denote place markers:

MEHUJAEI	1	2	3
IS-OFFSPRING-OF	4	2	1
IRAD	5	2	4
METHUSAEL	-1	6	-2
LAMECH	-1	2	-3
IS-GRANDCHILD-OF	-3	2	-2

The functions DOES and WHICH now become rather more interesting. Suppose we ask the question:

DOES 'ENOCH IS-GRANDCHILD-OF ADAM'

DOES now has to go scouting around to see if this is a bit of known data. If not, it goes to see if it has a rule-based definition for the grandchild relation. If it does, it has to find the first match for Enoch based on the offspring relation, and then pause while it tests whether this match in turn relates to Enoch. If not, it can resume looking for further matches for Enoch. It goes without saying that an XPL function employing recursion is the elegant (possibly the only) way to implement this process. It isn't very difficult to do.

Here's a more interesting rule:

ADD 'X IS-DESCENDANT-OF Y IF X IS-CHILD-OF Y'

ADD 'X IS-DESCENDANT-OF Y IF X IS-CHILD-OF Z AND Z IS-DESCENDANT-OF Y'

Recursion groupies will see the classic syntax: the first line denotes the limiting condition, the second line forms the recursive relation. The extensive use of recursive tests based on goal-seeking requests from the user is termed "backtracking" for obvious reasons. The standard mechanism of the state indicator in APL handles such requests beautifully.

I hope that those with some APL experience can see that it is not inordinately tricky to implement these aspects of PROLOG in user-defined APL functions. After a while the imagination starts running riot:

ADD 'X PROBABLY IS-CHILD-OF Y IF X IS-CHILD-OF Z AND Y IS-MARRIED-TO Z'

I suppose there's always the milkman. Adverbs like DEFINITELY, PROBABLY, POSSIBLY, CONCEIVABLY (oops) and INCONCEIVABLY can conveniently carry probability values like 1.0, 0.75, 0.5, 0.25 and 0; if you want to change the climate of expert opinion you can tinker with the values. And don't forget NOT:

ADD 'X IS-SEX FEMALE IF X NOT IS-SEX MALE'

Well, we don't all live in San Francisco.

What about expert systems themselves? If you recall the three elements:

- situation model
- knowledge base
- system manager

then it turns out in languages like PROLOG that the preceding constructs are used on an intermixed basis to represent data in the situation model and rules in the knowledge base. In XPL one feels it may be cleaner to keep them separate, but that's a matter for personal taste.

More importantly, most PROLOG users don't seem to have caught on to the fact that much of the data in the situation model may well be a transform of, for example, data about patients in a medical file whose records have been painstakingly composed over many years. File access to other systems is imperative; most of the facts are already there.

Furthermore, although PROLOG is good at relations, it's very bad at numerical tests. An expert system for shop floor control needs to cope with quantified tests based on quasi-real-time input from process sensors: for example:

ADD 'TURBINE SWITCH-SETTING OFF IF RPM ABOVE 5000'

is as far as I know impossible to handle in PROLOG if RPM is intended to be directly linked to the real world, whereas in APL one simply makes RPM a variable shared with an auxiliary processor which handles direct data capture. For non-APLers this means: variable RPM in the APL workspace is dynamically coupled via a programmer-specifiable update protocol with an arbitrary machine code segment, which is itself either handling process input/output or is talking to some other process that is. The point is that a robust and very well established framework for this kind of dynamic linking already exists in APL.

I hope I have managed to whet two kinds of appetite. If you know APL already, you may be starting to think about building your own XPL functions (mine are for private consumption and you've had numerous hints already) and writing your own expert system. If you know about expert systems and are toiling with inadequate tools, you might just think about looking at APL instead. An unlikely marriage? I don't think so. No more than Cain's, anyway; the identity of Mrs X is the theological riddle of the millenium. Good luck with such experiments: may thy domain specialities be brought forth and multiply upon the interface of the digital computer.

References

- (1) Artificial intelligence: the race to make it work for managers.
International Management, September 1984, pp. 20-26.
- (2) Godel Escher Bach: An Eternal Golden Braid
Douglas Hofstadter: Harvester Books & Penguin Press.
- (3) Behaviour: The Control of Perception
William T. Powers: Wildwood House
- (4) Cybernetics
Norbert Wiener: MIT Press
- (5) Brain of the Firm (2nd edition)
Stafford Beer: John Wiley & Sons
- (6) Do Androids Dream of Electric Sheep?
Ray Bradbury (publisher untraced)
- (7) Logic Programming in APL
Robert Jernigan: APL84 Conference Proceeding
- (8) Book of Genesis Chapter 4
Holy Bible, King James version
- (9) "The Word" Processor: King James Bible on floppy discs
Bible Research Systems, 9415 Burnet Road, Austin, Texas.

STEPS TOWARDS A BETTER BASIC — PART 2

by Anthony Comacho

In this second extract from Anthony Comacho's series aimed at convincing BASIC users of the delights of APL, the notion of array processing is discussed and an intriguing example is selected from the Gregorian calendar.

The article is reproduced with the kind permission of DATALINK magazine.

A language which allows a single instruction or command to process all the members of an array "at once" so to speak, has the advantages that it saves the programmer thinking about unnecessary loops and it makes the programs compatible with parallel processors.

In any programming language it is necessary to have a method for referring to variables which allows the variable referred to to be changed under program control.

An easy way is to name all the fields in a record and then to step from the record to the next through a file. This approach avoids the need to vary the name of the variable that you are dealing with while at the same time achieving the object of dealing with a series of variables in the same way with the same piece of program. This is common in Cobol.

A second way — one familiar to people who have programmed in Basic — is to define an array of variables which have a name in common and a subscript to distinguish between the various members of the array, for example ARRAY (25) for 25 numbers.

The third method is to use the array name to refer to all the members of the array and to use a subscripting method when one or more of the members of the array is to be singled out for individual special treatment.

Of these three methods, only the third allows simple instructions to refer to all the members of an array.

The first advantage of this is that the programmer doesn't have to think of so many loops. All he needs to do is to ensure that all the variables on which he wants to perform a similar operation are members of the same array. Of course, this doesn't actually remove the loop from the program. The machine code still runs round processing each item of the array in turn until they are all done, but it is done in machine code by the interpreter program, not slowly as a FOR...NEXT, DO WHILE or REPEAT UNTIL loop and the programmer does not have to bother about start and end conditions of the loop.

The other main advantage is that this method explicitly allows for parallel processing. We will not benefit much from this until a new generation of computers is in common use. The problem with array processors or parallel processors of any kind is that they normally require programs to be specially rewritten for them. This is because an ordinary compiler cannot tell whether the sequence given is important.

A consequence of this is that most programs get no advantage from array processors and so very few array processors are in regular daily use.

The leap year function shown in Figure 1 demonstrates the great brevity of APL. Instead of explaining the use of the symbols, I will stick to the principles. It takes in YEARS — a one dimensional array (ie a row or list) of years — and outputs LEAPS — another one dimensional array of leap years. I have put in an extra pair of brackets to show the sequence of calculation. The first operation (inner bracket) is to find the remainder after dividing each year in the list by 4 and 100 and 400 and 4000.

The result of this is a table of remainders four deep and as long as the number of years in the

list. Then for each year (ie column in the table) this table is processed (outer brackets). If the remainder from division by four was zero then it is a leap year unless the remainder from division by 100 was zero, in which case it is not. But even then it would be if the remainder from the division by 400 was zero and the remainder from division by 4000 was not.

The result of the processing is an array of true or false or yes or no values — ones for years that passed the test and are leap years, and zeros for years that did not pass the test. The last thing to do is to select out of the original array all those years which correspond to ones in the array of true and false values and this is done outside all the brackets by the (true and false array)/YEARS operation. Notice how the operations on each year can be done in parallel. The line works equally well with two, 20 or 200 years in the YEARS array.

Figure 1: A Leap Year Solution in APL

YEARS								
1984	1985	1986	1987	1988	1989	1990	1991	1992
$LEAPS \leftarrow (\neq + 0 = (4\ 100\ 400\ 4000 \cdot \cdot YEARS)) / YEARS$								
LEAPS								
1984	1988	1992						

APL AND GRAPHICS STANDARDS

by Dick Bowman

Now that the bang of your head on the table (reaction to the word "standards"?), has woken you up again, there are some things to be said about linking APL to the computer graphics world and doing it sensibly, so that we can take advantage of what other people are doing, and they have less of an opportunity to think of us as a bunch of iconoclastic weirdos.

What's APL Got To Do With Graphics?

Quite a lot, I think; on two bases:

- a) Graphics offers the same sort of productivity to users that APL offers to program developers. Taking the view that one major attraction of APL for the program writer is the ability to see what's going on as it happens, there is a symmetry to also being able to provide visualisation to the end user at a similar level of immediacy. You just can't tackle this by filing data away and logging on to another system every time you want to draw a picture; graphics have got to be immediately accessible from within the APL environment (I recognize that there are recent developments on some implementations which do, in principle, offer very flexible access to non-APL facilities).
- b) More seriously, taking graphics at a more abstract level than piecharts and histograms, what is being manipulated by a graphics program is objects. Now what are objects but collections of coordinates in n-dimensional space? At a simple level, in 2D space we could have a square defined as a set of four pairs of x-y values. We would then like to be able to move it, rotate it and magnify it. The obvious approach is to handle the square as a 2x4 or 4x2 array. One would think that a language which handles arrays implicitly would be at something of an advantage over scalar languages.

What Has The Computer Graphics World Been Up To?

Once upon a time graphics were a hideously expensive business to get into (particularly interactive screen-based graphics) and you needed all the computational power you could get. Graphics programs were written to operate with specific hardware: inside your graphics program you would be specifically incrementing the pen of, say, a Calcomp pen plotter.

Gradually, an evolutionary process occurred in which falling entry costs and device diversity led to increasing recognition of the problems associated with customisation of application programs to specific items of hardware. For example:

- i) Calcomp software — many other manufacturers supplied software to permit their hardware to emulate Calcomp pen plotters.
- ii) Manufacturer packages — stemming from the wider introduction of hardware with more extensive features (e.g. graphics input); a prime example being the Tektronix PLOT10 package.
- iii) Independent standards — the SIGGRAPH Core proposal, the ISO GKS, or "Graphical Kernel System" standard. There are two immediate points of comparison between Core and GKS; Core is intrinsically a 3D standard, while GKS is 2D. GKS nevertheless looks closer to becoming an accepted standard in the ISO-BSI meaning of the word.

There is good reason to suppose that graphics standards which emerge from a wider base than individual manufacturers or users will achieve some stability and widespread recognition. Already

in the world of FORTRAN programming you acquire leper status if you persist in writing graphics applications without due regard for either Core or GKS. Equally, manufacturers are beginning to produce graphics terminals which are themselves bound to the standards (note this reversal of direction).

One other aspect of GKS which is worthy of note is that the standard introduces a concept of language binding; implementations of the standard which are invocable from a particular programming language must all use exactly the same calling protocol — even to the extent of using the same subroutine names.

Something else about GKS should strike a familiar note; the main line-drawing primitive is the "polyline" — it draws a line through a set of points. In other words, GKS treats single points as a special case of multiple points.

What's The APL World Been Up To?

So far as I can tell, the APL community has been its usual independent self; often invoking the cry "we don't need to worry about that, we're too busy being productive". I don't want to detract in any way from the excellence of some of the work that's been done in APL with graphics as its end-point; the IPSA graphics which lie behind packages like SUPERPLOT spring immediately to mind, as does the achievement of IBM in making the 3279 display screen combine graphics with text dialogue. There have already been proposals for implementing the Core standard in APL (1,2).

Nevertheless, I get a distinct flavour of low-tech when I look at the general approach to graphics in the APL world; devices like Anderson-Jacobson paper terminals seem to be the norm, with few applications stretching beyond the business graph.

Equally, I am interested to note that the "leader" among APL graphics software is probably IBM's GRAPHPAK (3), at least in terms of market penetration. If GRAPHPAK has a philosophical stance, it certainly isn't one of alignment with either Core or GKS.

Looking to the future, I predict that we'll be seeing a lot of the APL * PLUS/PC graphics functions (4); but you don't design a graphics package by copying Microsoft Basic: anyone intending to work seriously with this system is going to do themselves a long-term favour by creating a set of appropriately-designed cover functions.

Can We Do Anything About It?

The first step we have to take is to recognize that there is indeed a problem.

As a next step, we should begin to familiarize ourselves with what the rest of the world is doing. Both GKS and Core have complete (if dry) descriptive documents (5,6); there's also a quite readable textbook about GKS (7). Further background reading on computer graphics is plentiful, (8) being a recommended example.

I would suggest also that anyone planning to work in the graphics area makes a conscious decision to implement functions at the level defined within either Core or GKS, building up in classic fashion to the levels needed within the application. Even if you initially intend only to produce the usual business graphs you'll have laid a foundation for the more imaginative displays exemplified by (9).

One could do worse than adopt GKS as a standard for two reasons:

- a) GKS looks like beating Core to the status of being an accredited standard as opposed to a draft proposal;
- b) 2D systems have limitations, but they should be easier to produce and have a pragmatic advantage that the most useful (if not the prettiest) business graphics are 2D. Most graphics terminals also are 2D.

Having homed in on GKS, we then note it as concerning itself with language bindings; if we want to be taken seriously we'd better get down to at least agreeing these among ourselves. Multiple contradictory APL implementations of GKS, all with differing function names, are not going to go down well with the rest of the graphics world.

Is Anyone Out There Working on an APL GKS Implementation?*

APL has a lot to offer the graphics world. Non-APL graphics users are beginning to show some signs of recognising that they have needs for the sort of solutions that APL offers. It may be hard for you to accept, but the majority of them have never even heard of APL.

We can make ourselves known, but before we do this there has probably got to be some element of setting our own house in order. I suggest that we begin to dialogue with the GKS people to see whether a GKS language binding for APL is a viable proposition. The British APL Association could provide a platform on which we might base an approach, but there's no point doing anything unless we can agree amongst ourselves what we want to do. Any such proposal would in no way remove incentives for individuals to produce packages as they see fit.

References

- (1) Faconti and Palamides; Practicing on the Core Graphics System; APL82 Proceedings, pp105—109
- (2) Freiden; A Core Viewing System For APL; Computer Graphics, March 1979, pp55—77
- (3) VSAPL GRAPHPAK: User's Guide and Reference; IBM SH20—9199
- (4) APL *PLUS/PC Programmer's Reference Manual Release 3.0; STSC
- (5) Status Report of the Graphics Standard Planning Committee; Computer Graphics, August 1979
- (6) Functional Description of the Graphical Kernel System Version 7.2; ISO/DIS/7942
- (7) Hopgood, Duce, Gallop, Sutcliffe; Introduction To The Graphical Kernel System — GKS Academic Press, 1983
- (8) Foley and Van Dam; Fundamentals of Interactive Computer Graphics; Addison Wesley, 1983
- (9) Robertson; A Graphic Vision of APL; VECTOR, June 1984

*Ed: Yes — see next article.

GKS — AN OPPORTUNITY FOR APL

by Richard Nabavi

Introduction

Traditionally, APL has been out of the mainstream of computing. A lofty disregard for most of the things which DP professionals regard as important has characterised the world view of APLers, although in many cases this disregard has been justified by subsequent events. The result of this has been that APL has continued on its own course, with little recognition of the good things which the non-APL computing world has been doing. Even today, you come across APL 'experts' who seem oblivious to the power and simplicity of spreadsheets such as Visicalc and Lotus 1—2—3. It is illuminating to look through the proceedings of the annual APL conferences, where you find surprisingly few papers which look outside the confines of the APL workspace.

Probably the greatest step forward in overcoming the insularity of APL has been the introduction over the past few years of versions of APL for microcomputers. Many of the micro implementations of APL have included good 'hooks' to the outside world to allow easy integration of, for example, word processing with APL applications. APL users are becoming more aware of the advantages of this kind of synergy, and this is certainly progress. Another very significant move is the current work being done on the ISO standard for APL; most industry commentators now feel that the main concern of users is moving away from the performance of their computer systems, towards issues of portability, standardisation, and links between different software suites. The forthcoming ISO standard for APL seems to me to be strong evidence for the argument that the APL community is taking these trends seriously.

It is surprising, therefore, that there has been so little interest in the APL world in the GKS or "Graphical Kernel System" standard. This is even stranger when you consider that APL has for a long time been quite closely associated with graphical applications (admittedly usually of the business graph variety, rather than animated or CAD/CAM graphics). I feel that it is time that the APL community considered its response to GKS.

What Is GKS?

For many years, computer graphics has evolved in a very hardware-dependent way. Major manufacturers such as Tektronix and Calcomp have produced graphical devices with their own interface protocols and conventions, and of course with different capabilities. As usual, IBM has gone its own way, and has remained incompatible with any other supplier, whereas smaller manufacturers have often tended to emulate the protocols of the main vendors, by for example, selling equipment which is 'Tektronix 4010 compatible'. On the software side, the routines to access graphics devices have also been highly diverse in the way they have handled graphic instructions. The net result of all this has of course been that applications software which makes use of graphics has tended to be much more machine-specific than software which uses only text input/output.

In keeping with the overall trend in the computing industry towards a greater emphasis on international standards, various influential groups and individuals have been working since 1975 on a graphics standard which would remove this 'Tower of Babel' style of graphics development, and substitute a clear, well defined and above all flexible standard protocol for interfacing computer software to graphics devices. The initial work was done by the German standards institute (DIN), and later taken up by experts from 16 countries, including a strong UK team. The final result of all this is an ISO standard, which is now complete, and which has international support from most major suppliers of graphics devices. In addition to its International status, GKS is expected to be adopted as a national U.S. standard by ANSI, although another rival graphics standard ('Core') has its supporters in the States.

What does GKS look like?

Because of the need for the GKS standard to be comprehensive, as well as free of dependencies on particular hardware or programming languages, it is expressed as a rather abstract model, expressed in a number of levels. Some VECTOR readers may be familiar with one vendor's low-level graphics system, namely IBM's GDDM, and so I will illustrate the features of GKS by reference to GDDM.

GKS defines a number of logical graphics functions such as 'POLYLINE' (draw a series of lines) and 'FILL AREA' (fill arbitrary polygons with specified colours and patterns). These are low-level commands, and correspond roughly with GDDM's low-level concepts. The most basic GKS commands (which nonetheless are sufficient to implement very good graphics systems) are defined as Level 0a, and are intended to be suitable for business graphics, as well as providing the low-level support for higher level commands. Level 0a defines only graphics output, without any digitising or interactive graphics capability. An implementation of GKS might, therefore, restrict itself to being an implementation at Level 0a, and ignore the more sophisticated features.

Levels 0b and 0c are the same, except that they add input facilities for interactive graphics applications.

Moving up to Level 1 of GKS, further features are added, including support of multiple workstations, display of several pictures simultaneously, manipulation of parts of pictures, etc. Level 1 (again subdivided into 1a, 1b and 1c) would generally be used for simple Computer Aided Design or interactive simulation software.

Finally, Level 2 adds features such as copying whole graphs, or 'segments' of graphs, between workstations, use of pre-defined picture segments, etc. This level is aimed at sophisticated CAD and graphical editor applications.

Provision is made at each stage for the application to query whether or not a particular feature is supported by the software and/or the hardware in use.

Language Binding

The GKS concepts described above are defined in an abstract fashion. For example, POLYLINE is defined as a GKS function for drawing a series of straight lines, and it takes a set of parameters consisting of the number of points (needed for crude languages which have not heard of the APL 'shape' primitive), and the XY coordinates of the points through which the series of straight lines should be drawn. The line is drawn using a preselected linetype, width and colour.

The jargon 'language binding' is used to describe a standard way of interfacing a given computer language to the abstract GKS functions. Associated with the ISO standard is a 'language binding' for Fortran. This specifies, for example, that the GKS function POLYLINE should be accessed by a Fortran subroutine called GPL, using the syntax:

```
CALL GPL (N,PX,PY)
```

where N is the number of points, and PX and PY are two arrays of the coordinates, expressed as real numbers.

As well as the Fortran binding, there is also a semi-standard Pascal binding, although this has not been incorporated in the GKS ISO document.

To understand this more closely, consider GDDM again. GDDM defines a set of graphics functions which can be accessed from a number of languages. In APL, the interface ('binding' in GKS jargon) is AP126, whereas other languages have their own IBM-defined methods of accessing GDDM.

An APL binding for GKS

This is where we come to the crunch. Is it possible, or indeed even desirable, to propose a standard interface between APL and GKS?

My own view is that it is highly desirable that there should be a standard APL interface to GKS. The main reasons for this are:

- (a) It will increasingly be the case that mainframes, minicomputers and even microcomputers will be supplied with GKS implementations. It is highly desirable that APL should be able to access these, and it would be a great pity if each vendor came up with its own syntax (again!) for doing what other languages will be able to do in a standard way.
- (b) It will also be increasingly the case that manufacturers of graphics devices will design the interfaces to their hardware with GKS in mind. If APL is to get the best out of such hardware, access via GKS routines will be both simple and efficient.
- (c) It will add greatly to the international credibility of APL, and further the progress which the APL ISO standard has made, if the APL community is seen to be taking international standards like GKS seriously.

As to the question of whether it is possible to define an APL binding for GKS, my answer is yes.

How would an APL binding for GKS look?

It seems to me that there are three possible ways of defining a standard interface between APL and GKS.

Method 1

The interface to GKS could be defined by a Shared Variable Processor. This would look roughly like IBM's AP126 processor which links APL with GDDM. Most probably, there would be a Control variable (CTL) and a Data variable (DAT) which between them would govern the interface to GKS. CTL would be used to indicate the GKS function required, and DAT would be used to pass parameters (such as coordinates) to GKS and receive back data from enquiries to GKS. GKS functions would be indicated by (probably) numeric codes. For example, to draw a polyline, we might enter:

```
DAT ← LINEDATA  
CTL ← 11
```

In this example, LINEDATA would be a 2 by N numeric matrix of the points through which the polyline would pass, and 11 would be the code which the binding standard had assigned to the GKS function POLYLINE. (This is the number defined by the so-called Metafile format for GKS, so there would be a correlation between the APL codes and the GKS main standard).

If it were felt advantageous to avoid numeric codes, we could use an alternative form:

```
DAT ← LINEDATA  
CTL ← 'POLYLINE'
```

i.e. We could use text names or mnemonics to indicate the GKS call required (although this could reduce efficiency of execution).

The advantages of Method 1 are as follows:

1. It would be similar to IBM's AP126, which is already widely used, and it might even be possible to provide partial compatibility between applications for GDDM and applications for GKS, using table lookup to translate between the two.

2. It would involve no additions to the APL language.

The disadvantages of using Shared Variables to access GKS are:

1. Shared variables are defined in the ISO APL standard as an optional feature. This means that not all ISO-standard APL interpreters could use this method to access GKS.
2. The use of numeric codes to indicate the GKS function required might be considered rather opaque, although it would have the advantage of being independent of national language.

Method 2

A second method of defining the binding is to propose a list of APL function names and syntax definitions for those functions. In other words, access to the GKS system would be via (presumably locked) APL functions, which could internally use any method they liked to get at GKS, but which would have an agreed form. This same example shown above might be simply:

POLYLINE LINEDATA

where POLYLINE would be a locked function. The choice of the names to use for the functions could be as above, although it might be better to use the same names as the Fortran binding standard uses for its subroutines. In this case, the above example would be:

GPL LINEDATA

The advantages of this method are:

1. It could work with any APL implementation, irrespective of whether shared variables were available.
2. It would be simpler for less sophisticated APL users to take advantage of this method, since shared variables are something of a mystery to some APLers.
3. Again, the APL interpreter itself would not have to be changed.

The disadvantages are:

1. Using defined functions would be cumbersome, because there are something like 300 GKS functions defined.
2. It might reduce efficiency in critical applications.
3. It might be considered odd to have user-defined function names reserved for a low-level international standard interface.

Method 3

Rather than using defined functions, a more extreme course of action would be to use Quad functions to access GKS. My first reaction to such a suggestion is that it would not be practical to do this, because of the large number of Quad functions required, plus the fact that it would involve major additions to APL interpreters (which will not always be using GKS).

Case Study: one reader's approach to the GKS — APL connection

MicroAPL Ltd is keen to promote a standard interface between GKS and APL. The steps which we are proposing to take are as follows:

- (a) We will probably be implementing GKS on MicroAPL's Mirage-based range of APL microcomputers. This will be done by an operating-system add-on which implements GKS, at least at the low levels (up to Level 0a initially), and which will be accessible by APL and by other languages, including Fortran using the standard GKS-Fortran binding.

- (b) We will invite interested parties to send comments to us on the best overall method of interfacing APL and GKS, with the intention of being able to submit a paper for APL85 with a suggested language binding. At the same time, we will implement the suggested binding on our equipment, to give those who are interested the opportunity of trying it out. Although we have not reached any firm conclusion on the best form of the binding, our current thinking is that, overall, Method 2 represents the best compromise, with the function names identical to the equivalent Fortran subroutine names (with some minor exceptions in the inquiry functions, to reduce the number of distinct names).
- (c) If there is enough support, we will propose the binding to the various standards committees in the hope that an agreed APL-GKS binding may be incorporated as an international standard.
- (d) We would welcome contact with interested users, and other APL vendors, in making this initiative more widely-based.

Conclusion

GKS represents a major opportunity for APL to become associated with standard graphics applications. If a standard binding can be agreed, APL will be a natural choice for many of the graphics packages of the future. If GKS is ignored by the APL community, much of the benefit of international standardization will be lost to APL users. Action is needed to take advantage of this opportunity for APL.



APL People Ltd.

3.Croft Lane Clore,
Orwell,
Nr.Royton,
Herts. SG8 5QW
tel.0223-207530

The *only* employment agency run
by experienced APL professionals.

We specialise in supplying skilled
APL personnel to complement *your*
APL project team —
permanent or temporary.

To discuss your particular requirements:

Call Valerie Lusmore on 0223-207530

Registered employment agency licence SE6440

GRAPHICS FOR DECISION-MAKERS

by David Preedy

"A picture is worth a thousand words", we are repeatedly told by computer graphics salesmen. Yet, despite the increasing availability of graphics hardware and software, there is little evidence of the use of graphics at a senior level in corporate planning and decision-making. The typical board member is presented with a thick report of detailed accounting schedules for the latest period, but rarely is a graph included.

Several reasons are typically put forward for this:

- "graphics is too expensive"
- "graphs of all our accounting items would take up too much space"
- "graphs are misleading"
- "numerate people don't benefit from graphics".

Such excuses might be acceptable if the current alternative could be seen to be working effectively. But the traditional accounting statements — profit & loss, balance sheet, and the like — were developed long before the advent of computers and the resulting impact on management methods. Valuable as they are for some purposes, these traditional statements inevitably filter out much of the information needed to run a business in the modern competitive environment, and they provide formal documents rather than actionable information.

By ignoring graphics our decision-makers are failing to equip themselves with a powerful tool, and yet the computing profession seems unable to dispel the misconceptions that abound. Some blame for this failure can be attributed to the Media for whom layout considerations, visual impact and sensationalism are often more important than good graphic design (and who have performed a similar disservice to the cause of Statistics).

Management can also point with some justification to the inability of D.P. departments in the past to produce well-designed output of numeric data. Even the basic courtesy of providing easily legible text by using upper- and lower-case letters is a recent, and by no means universal, introduction (and a major failing of the APL character set). I have yet to see any computer output that incorporates the principles outlined by, for example, Ehrenberg (1). Against this background it is hardly surprising that the D.P. department is rarely consulted in the design of Board reports, be they in numeric or graphic form.

But the greatest harm to the cause of computer graphics is frequently inflicted by the graphics themselves. They are too often badly-designed, inappropriate or geared to showing off the technical capabilities of the equipment and the programmer's skill, rather than the messages contained in the data.

Verbal and non-verbal thinking

A prerequisite to the design of effective graphics is to understand the broad information needs of the audience and to identify which of these needs can best be met by a graphical approach. The key to this identification is to distinguish between two modes of thinking — 'verbal' and 'non-verbal' — which are well explained by Hewlett-Packard (2).

'Verbal' thinking is carried out by the left-hand hemisphere of the brain and includes the use of symbolism, logic and step-by-step reasoning; the right-hand hemisphere handles the 'non-verbal' activities of mental pictures, intuition and pattern-recognition. The most effective use of graphics of any sort is in these 'right-hemisphere' situations.

Contrary to one's expectations, the majority of Board-level decisions are of this 'non-verbal' nature. For despite the claims of some Operational Research workers that models can solve

problems, in practice the output of a model is just one of a range of inputs to the decision-makers. Even if the logic of the model is undisputed, there is likely to be debate about the assumptions. Almost by definition, a 'left-hemisphere' decision that can be made by a logical process is going to be delegated by senior management. It follows that graphics ought to be a particularly suitable way to present information at Board level.

Management uses of information.

The degree of suitability and the appropriate criteria for good design will depend on how the information is to be used. This can be suitably classified into four categories — reference, analysis, browsing and presentation.

Reference, in isolation, is a left-hemisphere activity for which graphics tend to be unsuitable. If the Managing Director wants to know today's share price, it is much easier for him to read the number than to derive the figure from a graph. So a user-friendly data retrieval system is likely to offer the best reference facility. However, such data on its own is of strictly limited value. Knowing today's share price tells a manager what he will have to pay for shares, but doesn't help him decide whether to buy them.

Analysis involves trying to uncover relationships in the data, for example when planning future activities, and is inherently an exercise in pattern-recognition. In many cases models can be used to help find and quantify relationships, but even then a graphical display is normally the best way to explain the results. Often the most revealing analytic charts do not use time as one of the axes. So, for instance, a plot of share price against dividend yield may allow the manager to compare share purchase at different prices with some alternative investments.

Browsing differs from analysis in that it tends to be a fairly unstructured search, normally to monitor performance, the aim being to check for under- (or over-) performance and to diagnose problem areas. For example, the manager might want to look at the historic trend of share price and compare it with market movements. Exception-reporting can play a key role by providing some structure to the browsing, but has to be backed up with an easy way to access more detailed information. Typically, a time-plot will highlight the sort of significant variations that are being sought.

Presentation is a significantly different type of activity, because the conclusions have already been drawn from the data, so the objective is to communicate them rather than discover them. In these circumstances, the value of a graph depends on the nature of those conclusions. A detailed step-by-step argument may well be best supported by non-graphic material, maybe using graphs to explain the background to the problem. In presentation it is essential that each display tells a message, and the required message should be the starting-point of the graphics design.

The traditional Board report.

Given this range of information needs, it is easy to see the shortcomings of the conventional Board report. It concentrates on the latest period at the expense of trend data and fails to discriminate between data of different importance; for example, the key profit lines are treated identically to the accounting adjustments. Moreover it presents the data in a numeric form that is inappropriate for most of the Board's requirements; it is a reference document rather than a true 'management report' and reflects the need of auditors and accountants rather than senior management. A better alternative would combine graphical and numeric displays, include trends and provide an easy way to search through the (typically) large volumes of data. For any large company this implies using a computer-based information system, offering well-designed graphs.

At this point it is worth mentioning the approach advocated by Irwin Jarret (3), who has proposed standard graphical displays of the main financial statements — Balance Sheet, Profit & Loss

and Source & Application of Funds. However, by accepting these existing reports as the starting-point, this approach does not differentiate the ways in which the information might be used. The result is merely to convert what are in practice reference documents from their traditional numeric form into graphs that are, as discussed earlier, inherently unsuitable for reference purposes.

Graphics Design.

Graphics design is a complex area in its own right, and an article like this can only highlight some salient points. (A more complete exposition is given by Edward Tufte (4). This excellent book is well worth obtaining if only for the superb examples of some early graphics that it presents.) Tufte pinpoints the importance of eliminating what he calls 'non-data ink', — the frills that distract attention from the message of the graph. Some of these are obvious — the scrolls and fancy decorations that adorn many graphs in the Press. These tend to be difficult to generate on computer systems and so are less of a problem in computer graphics. But there are other manifestations of 'non-data ink' that some graphics software seems designed to encourage.

Grid-lines, for instance, (a relic from the days of graph-paper) serve only to refer to the values of the points on the graph. Such reference is managed much better by providing a table of numbers that shows the actual values. Numbering the points on the graph itself is even worse because it confuses the left and right hemisphere activities of the brain. Similarly the gratuitous use of colour and/or shading patterns tends to distract from the message of the graph.

Tufte extends this elimination of 'non-data ink' a stage further and questions, for instance, the need for axes, whose prime function seems to be a washing-line on which to hang tick-marks. I cannot fault his argument, but must confess to not having implemented this idea and so have no experience of how it works in practice.

Of course, few graphics originators will be experts in graphic design, and so some guidelines are needed to direct them, at least until an Expert System is written to do the job. These guidelines should prevent the worst excesses of bad design, but their effect can be beneficial in a more subtle way. Presented with consistently-designed graphics, managers rapidly become adept at comprehending them, even when the formats themselves are fairly complicated. People turn out to be surprisingly skillful at learning how to 'read' the graphs.

Moreover most of the browsing and analytical graphs can be selected from a 'chart-book' of a few standard formats, e.g.

- current year by period
- current year cumulative
- moving annual total/average
- long-term trend
- scatter-plot
- map

each of which can be carefully designed to incorporate best current practice. The same format can then be used to display many different accounting items, such as sales, profit, return on capital.

However this 'chart-book' approach does not address the individual needs of presentation graphics. The rest of this article concentrates on some guidelines that are theoretically sound and have been found to work in practice. Other good references are the publications by Hewlett-Packard and ISSCO (2) and (5). Such guidelines should provide a good foundation for designing 'chart-book' formats and one-off presentations.

(a) Provide output in a variety of media. VDU output is essential if you want to cater for browsing or analysis for any but the smallest organisations without becoming 'snowed under' with paper.

Paper hardcopy is the easiest way for managers to take the system away from the office. 35-mm slides or foils for overhead projection are needed for presentations away from base, although video projection now offers a real alternative for in-house presentations.

(b) Where there is a choice, use landscape rather than portrait format. At the purely practical level, this will enable you to get a much closer match between output on different devices, and also allows more space per line of text. There are also good theoretical grounds for this choice; portrait format tends to exaggerate effects (normally shown on the Y-axis). Fortunately VDU manufacturers have been faced with the same problem and so make screens with landscape format. Occasionally you may need to insert graphs in documents designed as A4 portrait; normally you can use a reduced-size graph in landscape format.

(c) Decide on a size for hardcopy and stick to it. If, for example, your overhead projector handles only 8x10, then don't produce A4 output at all.

(d) Establish a colour-coding convention, like the example given in Figure 1. This can pose considerable problems depending on the kit you use — try matching plotter pen colours on a white background with VDU colours on a black background. Moreover the convention that you establish should depend on the major comparison being made in the graph, and you should beware of colours like red which not only has connotations of financial loss, but also tends to dominate other colours.

(e) Make careful choices of shading patterns; use shading intensity and line-types to complement any colour convention so that black-and-white photocopies do not lose the meaning of the graph (even though they reduce its impact). Where the patterns represent ranges of a variable, use different intensities of the same tone, ordered so that the intensity increases as the range values increase. (See Figure 2 for an example.) Avoid dazzling patterns.

(f) Use upper and lower case text and stick to one style of characters. Beware of software-generated characters on VDUs, as the interaction between the character definitions and the pixels on the VDU can produce distortions, such as asymmetric 'O's.

(g) Avoid numbering data-points on graphs (see earlier). If your message relies on having accurate numbers, then you should question whether it ought to be presented graphically anyway. Prepare a separate reference table outside the graphics area, such as below the X-axis or on a separate display.

Figure 1. Illustrative example of a colour-coding convention.

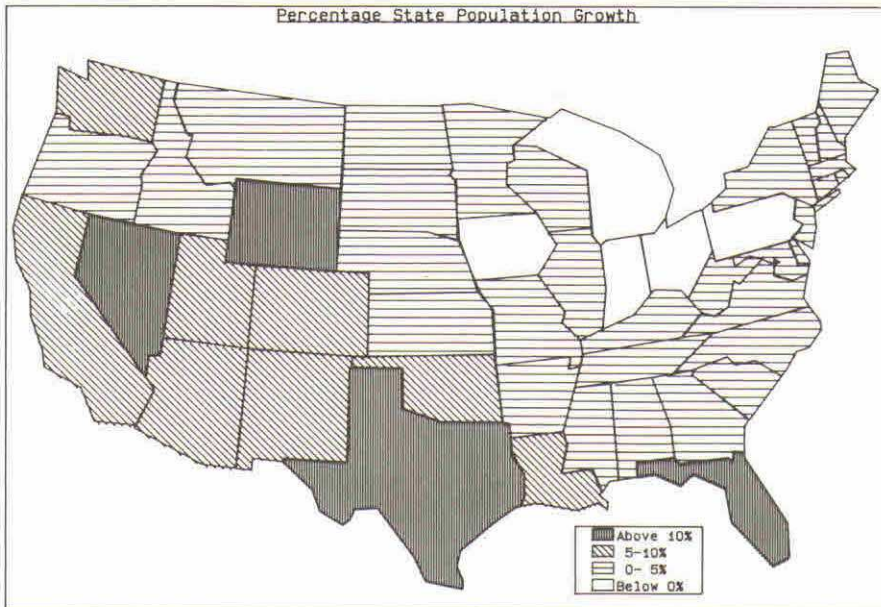
Colour	Major dimension of comparison			
	Data-mode	Variance	Division	Competitors
Red		Adverse		Us
Green	Budget		Divn. A	Comp. 1
Blue	Plan	Favourable	Divn. B	Comp. 2
Orange	Estimate		Divn. C	Comp. 3
Magenta	Actual		Divn. D	
Grey	Last Year			

(h) Avoid pie charts. A single pie chart gives insufficient information to draw any meaningful conclusions. Knowing that 40% of product sales were in the U.K., 35% in Europe and 25% in America merely prompts other questions. What about last year? What about profits? What about our competitors? Multiple pie charts should be replaced with a stacked bar plot of the same data which will be easier to understand. It also overcomes the visual conflict of mixing a polar coordinate system (within each pie) and a rectangular coordinate systems (the spatial arrangement of the pies). Multiple pies with different radii (or areas) aggravate the problems by raising issues of interpretation. Anyway, sooner or later you will find that you need to plot a negative pie angle.

(i) Conventionally time goes along the X-axis, and people are used to this, but for other bar charts use horizontal bars to allow easy vertical comparison. This compares with similar advice from Ehrenberg (1) when displaying tables of numbers.

(j) For time series, use line plots to display 'stock' items (physical stocks, balance-sheet items), averages over time, and ratios. To a first approximation they vary continuously, so it is meaningful to visualize them as moving from one level to another. Flow items (sales volumes, profit & loss entries) on the other hand, represent discrete quantities for each period and so may be represented by bars on a bar-chart.

Figure 2. Use of shading intensity to show ranges.



(k) Above all, for presentation graphs stick to one message per chart and state explicitly what that message is. In some cases you can even write the message on the chart itself. It is a false economy to reduce the number of charts by cramming several messages together, because the audience have to separate the messages as well as understand them individually. Remember too that one side-effect of a complicated graph (or a convoluted argument) can be to reduce the presenter's credibility in the eyes of his audience.

I have deliberately avoided describing these rules as 'standards', because they should not be applied indiscriminately. Standards can sensibly be applied to 'left-hemisphere', logical activities, but are less appropriate for the intuitive, 'right-hemisphere' areas where graphics are most valuable. Occasions do arise when the best way to convey the required message may entail some breach of these guidelines. However your chosen guidelines should be built into your graphics software at least as default settings, which the user has to take positive action to override.

Moreover few, if any, software packages are designed with considerations like these in mind. Instead they draw pie-charts to show off the high resolution of the VDU, and offer flamboyant character-styles and lurid shading-patterns, whose use will impair the effectiveness of the graphs. Perhaps the most important role of APL in selling graphics to the senior management of major corporations is as an interface to disable the use of such facilities.

References.

- | | | |
|---------------------|---|---|
| [1] A.S.C.Ehrenberg | Rudiments of Numeracy | J.R. Statist.Soc.A.
Vol 140.p277-297 |
| [2] B.S. Matkowski | Steps to Effective Business Graphics | Hewlett-Packard |
| [3] I.M. Jarett | Computer Graphics:
a Reporting Revolution | Jnl. of Accountancy
May 1981 |
| [4] Edward R. Tufte | The Visual Display of
Quantitative Information | Graphics Press |
| [5] ISSCO | Choosing the right chart | |

CASE STUDY*by Adrian Smith***Acknowledgements**

I would like to thank David Quas and Katie Williamson for inviting me down to Imperial Group HQ. It was a thoroughly enjoyable visit, which was interesting personally as well as in my 'professional' capacity as applications correspondent.

Plea

I would like to hear from anyone who has been using APL in the 'Expert Systems' field. This appears a likely theme for the next issue, and offers of material would be most welcome.

Graphics in Decision Support

by Adrian Smith

Introduction

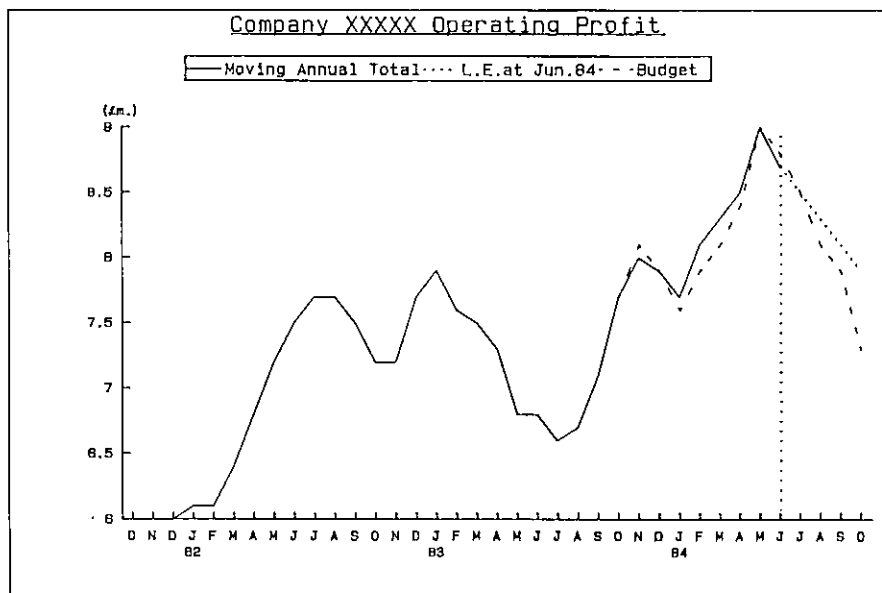
Imperial Group have consciously diversified from a tobacco company to a broader base. They have four divisions (tobacco/food/brewing/Howard Johnson — an American Hotel and Restaurant chain) within which are various operating units, e.g. Golden Wonder crisps. The group has a headquarters staff based in London and Bristol, whose job it is to support the chief executive in directing overall policy and in maintaining corporate control. Part of this HQ staff is a 5-man Operational Research team, which has developed and implemented the graphics system described below.

Current Hardware

Clearly the system is developing rapidly, and the hardware is in a constant state of flux. As I saw it, the computer centre was based round 2 MicroAPL 'Spectrum' machines, running a total of about 200 Mbyte of hard disk. The computers are linked via Ethernet, so that any user can run any program on either machine. There is also a (slow) link to a further Spectrum at Bristol.

Virtually the entire London HQ is cabled up with standard RS232 ports, and around 12 Tektronix 4105 graphics screens are scattered throughout the offices of the board members, financial planners, etc. Also there are two HP7221 plotters, which were used for the illustrative graphics included in this article. The Tektronix screens have been modified to output a standard 'Red/Green/Blue' signal, so that a whole chain of cheaper colour monitors can be slaved together for conferences. All the screens are on trolleys, so that re-location is very straightforward.

Figure 1. Graph No. 768



Outline of Current System

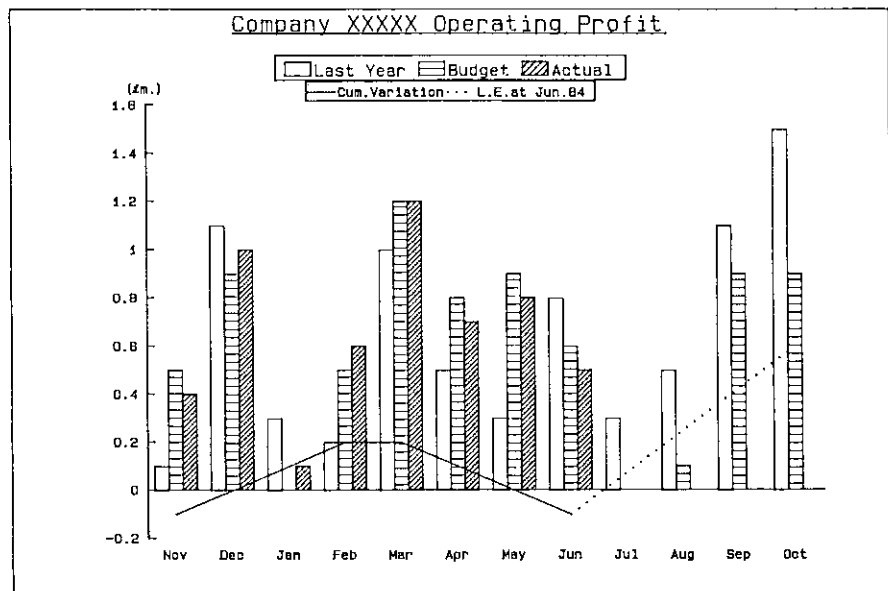
What I would like to do is to outline quite briefly the system as it now stands, and then to concentrate on the factors which governed its evolution and led to its undoubted success. First let me say that as a user of IBM 'mainframe' graphics I found the speed, quality and flexibility of the micro-based systems very impressive indeed. Many of the key features would be quite impossible to reproduce in a big-machine environment.

The nearest analogy I can find to the style of operation is PRESTEL. Each graph (e.g. Fig.1) is given a number, and can be brought up on the screen in one of 3 ways:

- the system starts with a master menu, below which are a series of sub-menus, which soon lead one to the right range of numbers, and thence to the graph.
- each user has a 'quick reference' card showing the range of numbers allocated to each type of graph. Thus he can get straight to the appropriate sub-menu, without working through the tree of options above.
- the user can look up the graph number (e.g. '748' for Figure 1) in a directory and key it in directly.

Note that only standard graphs are available; to include a new picture in the system involves a certain amount of programming and some changes to the menus. However there are 1825 graphs (and tables) to choose from, and in practice users undoubtedly prefer a range of known formats to a general 'gimme a graph' facility.

Figure 2.



It seemed to me that the presentation of the system was an important factor in its acceptance at board level. All selection was done entirely on the numeric keypad, and the 'function keys' were replaced by specially engraved keycaps saying things like 'MENU', 'CROSS-HAIR' and so on. The 'user manual' was a beautifully presented A5 document covered in gold-engraved leathercloth with good thumb-indexing. The Tektronix screens themselves were quite tidy, and not at all out of place in the office of today's executive. All in all, I would say that as well as being easy, the system was also pleasant to use; I shall return to these thoughts in the later discussion on evolution.

Many of the neatest features stemmed directly from the capabilities of the Tektronix screen. In particular one could switch colours in and out virtually instantly, for example in Fig. 2 one could start by showing just last year and budget (actually brown/green), then hit the <PURPLE> key to bring in the year-to-date figures, then hit <BLUE> for the cumulative variation. To check the figures that lay behind the picture, one again just hit an 'OPTION' key and selected 'Table'. Again the switch from figures to picture and back was effectively immediate.

Finally, the use of the 'CROSS-HAIR' key enabled a simple joystick arrangement to move a cross-hair cursor anywhere on the screen. This could be used to read figures from the axis, to compare the relative height of peaks or to show break points in time (such as the year-end). Again, the overall impression was of a system which felt comfortable in daily use.

History and Evolution

The genesis of the system was in July 1981, when a new chairman expressed interest in a top-level information system. By December discussions had led to a prototype, with a small (Superbrain) micro, a 4-colour plotter, and a terminal to a bureau. In early 1982 the OR group was given a new role in supporting head office functions, and a MicroAPL Spectrum microcomputer was acquired. At the same time the bureau service was being developed to drive a Tektronix 4027 screen, and soon demonstrations were held using live data.

A major step forward was taken in July when a Spectrum was acquired specifically to run the graphics system, and the opportunity of cabling the entire London office complex was taken. By August the menu of available graphs had begun to expand. Several similar systems in the USA were seen, and in November a review of possible future developments was made.

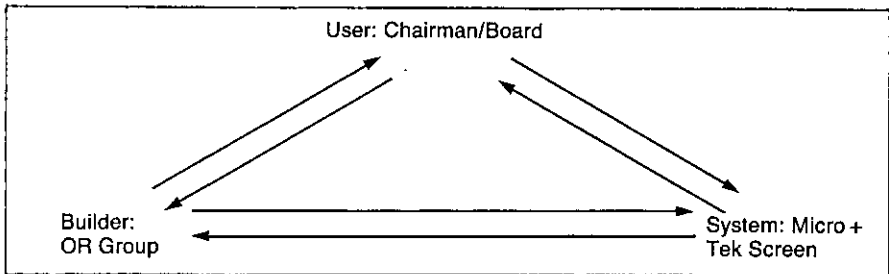
During 1983 the system quietly bedded down, with some output going direct into the board statements as confidence grew. The next stage was to expand from a 'one-terminal' situation to the ideal of one screen per user. The Tektronix 4105 was chosen, and the special keytops designed. By September there were 5 screens, and 200 graphs; by November the number of graphs had grown to 500; by February over 1000. Now there are 12 screens and (as of August 22nd.) 1825 graphs, so one can conclude that the initial growth of the system was very rapid indeed.

1984 has been a period of consolidation, with the purchasing of a third Spectrum for development and a gradual increase in disk space for storage, and in the number of terminals available to users. Many minor improvements have been made, but I think it would be reasonable to say that the several elements of the system have now stabilised.

I would like to close this review with another look at this history in the context of the three-way interaction between 'User', 'Builder', and 'System' which typifies the evolutionary process through which a DSS grows. What is clear from both the literature (1), (2) and (3) and my own experience is that if any of these three communication channels become blocked the growth of the resulting system may be badly stunted, or even aborted. The Imperial Group history is a classic example of the DSS method in action.

Essentially the links are:

Figure 3.



... and the interactions (in cyclic rather than historic sequence) look like this:

User to Builder:

- initial interest.
- requests for new graphs as experience of prototype system grows.
- feedback on usefulness of current range of graphs.

Builder to User:

- presentation of existing data in graphical form.
- attempt to define new views of data to extract the maximum of useful information.
- willingness to respond quickly to ad hoc requests.

Builder to System:

- requirement for inexpensive, neat-looking graphics terminal.
- ability to replace key-caps, and to extract video signal.
- demand for fast response/ease of use.

System to Builder:

- features like cross-hair cursor, easy removal/selection of colours influenced the design of graphs. In particular the full graph could be 'overloaded' with lines because it was easy to cut them in and out.
- fast recall of graphs from file (and quite efficient storage) led to the PRESTEL-type 'pick and choose' approach.

System to User:

- early graphs stimulated new views of existing data, and generated wide interest.
- increasing confidence around screens and keyboards led to quick uptake.

User to System:

- the first impression had to be 'executive'. Presentable manuals, neatly engraved keys etc. were essential.
- initial ease of use was vital. Also ease of re-use after a long period of inactivity. Hence the PRESTEL-like menu-tree.
- a cross-hair cursor was needed so that figures could be accurately read off the axes. Now made largely redundant by the availability of back-up tables.

In summary it is not just the initial 'User to Builder' request that mattered. The system evolved through a learning process in which all the six interactions figured strongly. If anyone who reads this is involved in developing this type of Decision Support System I would strongly recommend them to write down the six headings above, and to treat them as a checklist to ensure that all these channels at least remain open. The undoubted success of the Imperial Group development is to me a real encouragement that the DSS approach does get worthwhile results in areas where traditional systems methods are left hopelessly floundering.

Examples

A selection of typical graphs are illustrated on the following pages.

Figure 4.

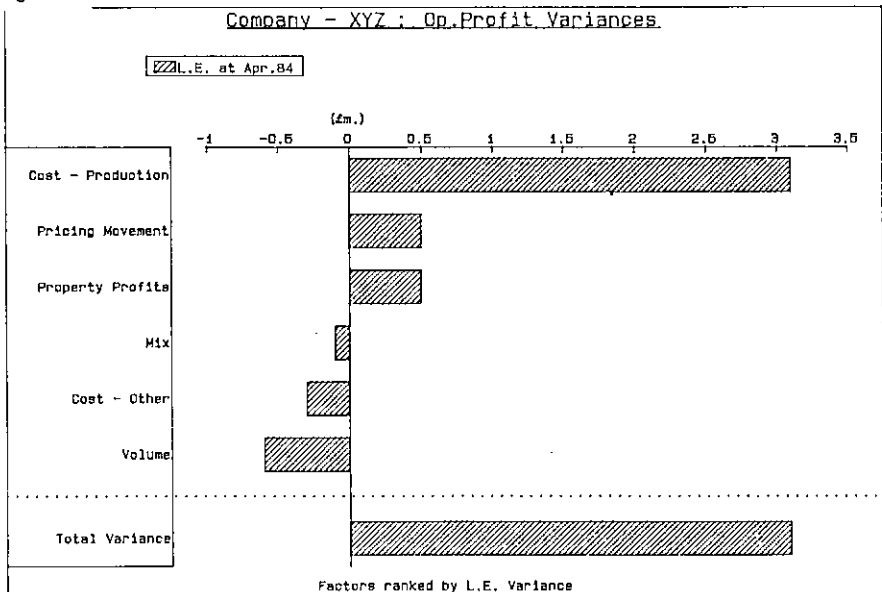


Figure 5.

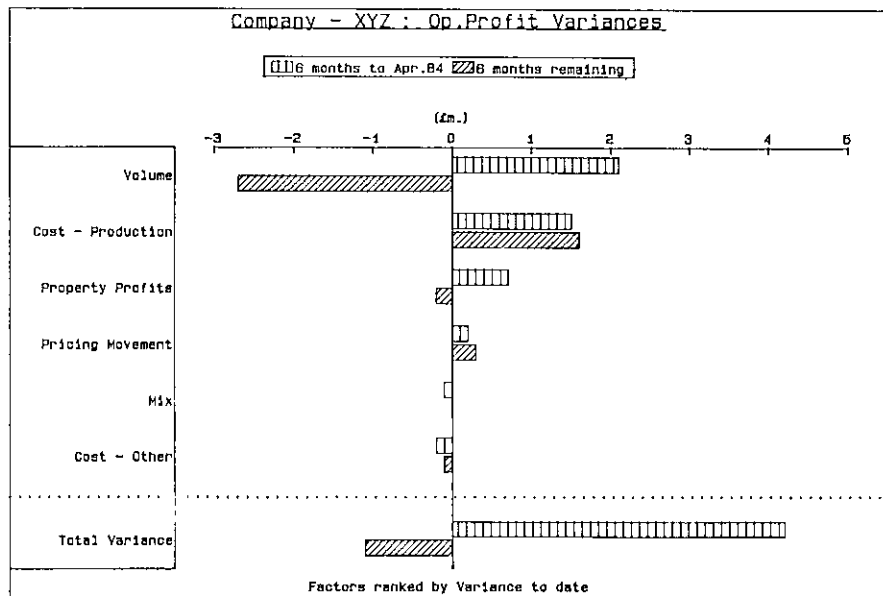


Figure 6.

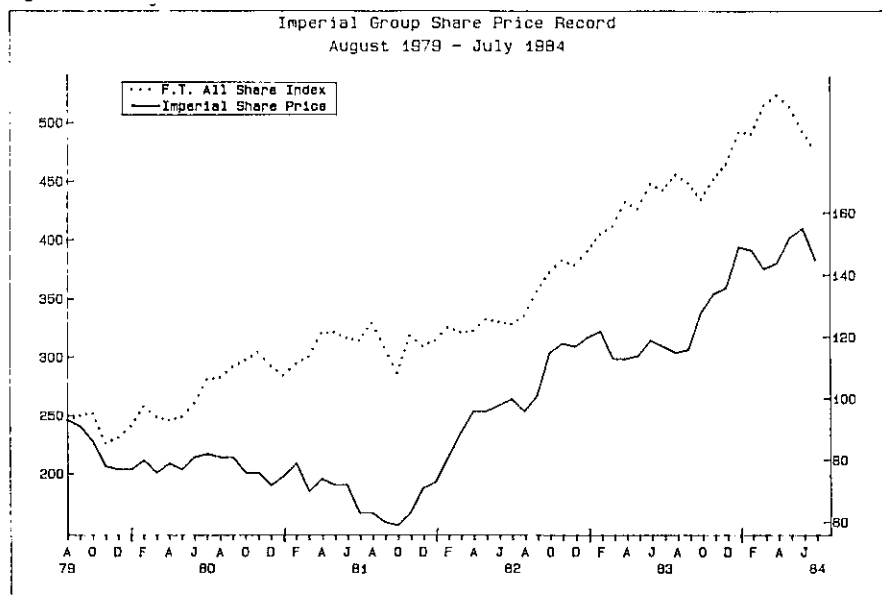
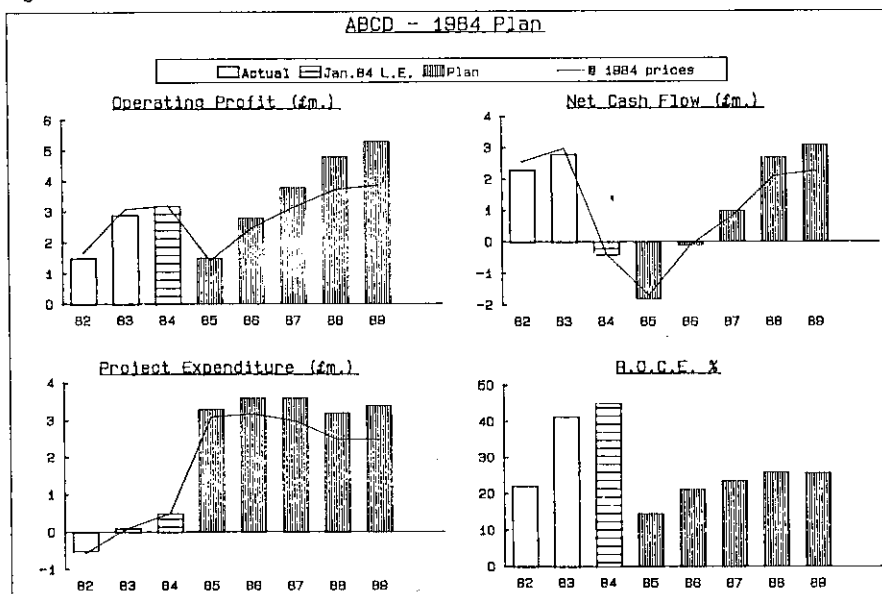


Figure 7



References

- (1) Decision Support Systems: Issues and Challenges
Goran Fick & Ralph H Sprague Jr. (Eds)
- (2) DSS — Research Perspective
P. W. Keen — IIASA Proceedings. Pergamon Press 1980
- (3) Human Interaction with Computers.
H. T. Smith & T. R. G. Green (Eds) — Academic Press, 1982

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL, and will contain items to interest people with differing degrees of fluency in APL.

Imperial Group plc

INFORMATION SYSTEM FOR THE CHIEF EXECUTIVE

Imperial Group employs some 100,000 people. Among its well known names are Wills and Player in tobacco; Courage and John Smiths in brewing; Golden Wonder, HP, Ross and Youngs in food; and the Howard Johnson motels and restaurants in the USA.

A small experienced OR team has implemented and is continuing to develop decision support systems including strategic planning, budgeting, review of monthly performance and investment appraisal. Our clients are the Chief Executive, his immediate subordinates, and the planning and financial functions. The success of this operation depends on speed and flexibility of response to requests and a high level of creativity. The hardware includes multi-user micros and colour screens. The job offers an opportunity to join this team and work at the highest level in a large organisation.

The successful candidate will have several years' experience following a good degree in a numerate discipline. Knowledge of APL, modern planning and management accounting techniques, and modelling are all desirable. However, creative skills, and ability to work at a high technical level and to handle very senior client contact, will be the overriding considerations. The Group Head Office is split between London and Bristol and the job could be based in either location. The starting salary is unlikely to be less than £17,000.

Please send relevant career details to:

Mr. G.E. Wells, Imperial Group plc, Bristol BS99 7JR

0272 666961 ext. 2377

TECHNICAL EDITORIAL

by David Zieman and Jonathan Bermen

We are delighted with the response to the last issue; letters and articles have arrived in sufficient profusion to enable us to base this issue's technical section largely on your own contributions.

This issue introduces a new feature which we hope to continue on a regular basis — the technical letters section. We do not necessarily agree with all the opinions expressed by correspondents, but rather than make editorial annotations, we hope that you will write us letters with constructive criticisms instead.

In the next issue we shall be starting a new feature entitled 'Surely There Must Be a Better Way?' Readers will be invited to submit code where they suspect that there could be a better way of solving a problem, and we will publish any improvements that are received. If you do not know how to solve a particular problem in APL, send it in anyway and see if anyone has a solution. If you feel that you have a beautiful solution to a problem, send that in as well; if no improvements are published you will have the satisfaction of knowing that it may be the best. The problems should be short and capable of being solved with a single APL function. We do not intend to get into the business of writing complete packages.

The distribution of the inaugural issue of VECTOR was delayed, so we have decided to hold over the judging of the competition until the next issue. The responses so far have been numerous and excellent, and include an ingenious solution to beat the rules! Keep the entries coming in; you never know, you might make it.

One of APL's problems is that it has a special character set that cannot be reproduced by most printers, as you have probably noticed whenever one of the computer magazines tries to print any APL. We made valiant attempts in the first issue of VECTOR to mix APL characters with ordinary text, which proved possible, but tricky, so we have now decided to make production easier by keeping any APL text on a separate line from other text. If you can keep to the rules set out below, you will increase the chances of your letters and articles being printed.

VECTOR Publication Standards for APL Code

In order to make the production of VECTOR easier, the following rules should be adopted when preparing letters, articles and other contributions for publication.

Please do not include APL symbols within the text. If an APL symbol needs to be referred to, use its name; most APL textbooks have a list of symbols and their standard names.

Where variable or function names are referred to within the text they will be set in ordinary capitals. This means that sentences have to be carefully worded so that a function or variable name can be easily distinguished from any other words that need to appear in capitals.

APL statements and function listings will be reproduced from the submitted document whenever possible, and will be pasted onto a separate line between the text. This means that the listings should be of high quality; a 'daisy-wheel' printer with a carbon ribbon should be used. If you do not have a quality printer, we are prepared to re-set the APL ourselves if it is not too long, but we would much rather not do this as it is all too easy to make mistakes. Ideally we need two documents; the "readable" copy which shows what the final appearance is likely to be, and the "printer's" copy which we actually send to the printers. The "printer's" copy needs to have all the APL completely separated from the text. The easiest method is to leave three blank lines in the text where the APL is to appear, and to have a separate sheet containing all

the APL; each section also separated by three lines. A simple numbering scheme can then be used to show where the APL block is to appear in the text. These APL characters will be photo-reduced and inserted into the text body.

Drawings, figures and black and white photographs are welcome, as they help to break up the text and make articles more visually interesting. Please prepare each figure on a separate sheet of paper, and give it a number and caption. The caption will be typeset and will appear at the top of each figure, which will be presented in a box as near as possible to the reference in the text. Do not worry about the size of the drawing, as the printers will photo-reduce it to fit the size of the page or the space available. Your text should refer to the drawings strictly by sequential figure numbers.

Footnotes and references will be grouped together at the end of the article, so please avoid writing text that requires a footnote to be on a particular page.

Introduction to Contributed Articles

In this issue we are presenting four contributed articles.

"APLpip — the hard seed of a health-and wealth-giving fruit" was originally sent to us by Mr. Llewellyn-Jones as a letter. We have converted it into a technical article. Program listings were also included, but were unsuitable for reproduction. Mr. Llewellyn-Jones, who works for Reuters in London, tells us the story of how he has attempted to solve the problem of representing APL code using the standard ASCII character set. We would particularly like to hear from others who are working in this important area.

Professor Alan Hawkes of the department of Management Science and Statistics at University College, Swansea has contributed two articles. The first, "Statistical Computing with APL" is reproduced with the kind permission of 'The Professional Statistician'. Although intended for a statistical audience with no knowledge of APL, it also serves as an excellent general introduction to the language.

Professor Hawkes' second contribution, "Numerical integration and probability distribution" presents an APL function that employs Simpson's rule for the numerical integration of any well behaved function. It is implemented in MIPS APL on a PRIME 750 and its use is demonstrated by calculating probabilities from the distributions used in statistics. It is claimed that the results are accurate and efficiently computed. We are most grateful to Professor Hawkes for his articles and look forward with interest to the continuation of the geometrical progression represented by the volume of his contributions.

Finally, in "ISO APL Standard Update", David Ziemann, one of the Technical Editors and a consultant with Cocking and Drury, gives a brief update on the state of the ISO APL standard as a result of the last meeting of the working group in Helsinki just after APL 84.

Technical Correspondence

From Mr. Norman Thomson

26th July 1984

Sir: Following Professor Alan Hawkes' article on "Complex Numbers in APL" in the first issue of "VECTOR", and at the risk of appearing to gloat over those of your readers who are constrained to APL1, I cannot resist pointing out the merits of complex numbers and other features of APL2. Here is the article paraphrased in APL2, and thereby reduced to a single page.

Some assignments:- scalar: $A \leftarrow 7J8$

 one element vector: $B \leftarrow 1pA$

 vector: $C \leftarrow 1J5 \ 2J6 \ 3J7 \ 4J8$

 or $C \leftarrow (14) + 0J1 \times 4 + 14$

 matrix: $D \leftarrow 3 \ 4p(112) + 0J1 \times 12 + 112$

Real and imaginary parts of D are given by using 9 and 11 as the left argument to the Circle function; 10 gives the same result as Modulus. Plus, Minus, Times, Divide, Power, Log and Domino are as for reals, e.g.

$$\begin{bmatrix} 2 & 2p1 & 1 & 0 & 0J1 \end{bmatrix} \leftrightarrow \begin{bmatrix} 2 & 2p1 & 0J1 & 0 & 0J^{-1} \end{bmatrix}$$

and

$$\begin{bmatrix} 2 & 2p1J1 & 2 & 1 & 1J^{-1} \end{bmatrix}$$

is DOMAIN ERROR.

To calculate e^A where A is a matrix, define first

$$\text{EXP: } L \leftarrow \cdot \times / Lp \leftarrow R$$

which raises matrix R to the power L,

$$\text{NORM: } \lceil / + / \lceil R$$

to define the 1-norm, which is applicable both to the real and complex case, and

$$M \leftarrow 1 + \lceil 2 \otimes \text{NORM } R$$

so that M is the smallest non-negative integer satisfying

$$\|A \otimes 2^M\| = \|A\| \otimes 2^M < 1/2$$

The function

$$I \leftarrow T \cdot = T \leftarrow 1 \uparrow pR$$

produces the identity matrix. The exponential function for matrix R is then

$$\text{EXPON: } T \text{ EXP}(I \text{ R}) + \dots + ((113) \text{EXP})^{\sim} \left(R \div T + 2 * M \text{ R} \right) \div \dots 113$$

To paraphrase Professor Hawkes' penultimate paragraph —

"Oh well, it's better than doing it in APL!"

Yours sincerely,

Norman Thomson
IBM United Kingdom Laboratories Limited
Hursley Park
Winchester, Hampshire.

From Mr. Norman Thomson

7th August 1984

Sir: I should like to offer some comments on the article "APL and Partitioned Data" by Jonathan Barman in the first issue of "VECTOR".

In his conclusion, he states rightly that generalised arrays will do much to obviate the need for partition vectors. I believe, however, that there will still be uses for such vectors even when extended APL is available. The appendix (pp. 138—141) suggests naturally a demonstration of another invaluable feature of extended APLs, namely user defined operators. To illustrate, all the 16 little algorithms given can be subsumed in the single user-defined monadic operator:

$$\text{DIFF: } R \text{ LO } L[1] \vee (0[L[1]]) \Phi L[2], R$$

where R and L stand for right and left arguments and LO for left operand.

For example, the first algorithm

$$A \leftarrow 1 + 0 . A$$

becomes

$$\sim 1 \text{ } 0 < \text{DIFF } A$$

I find it helpful to regard the 1's in a partition vector as markers for elements in R, and if the elements of the left argument are referred to as L1 and L2, then the roles of various argument/operand combinations can be summarised as follows.—

L1	L2	LO	Marks:—
1	1	<	Last zeros
~ 1	1	<	First zeros
1	0	>	Last ones
~ 1	0	>	First ones
1	$\sim 1 \uparrow R$	$\neq$	Last in each run
~ 1	$\sim 1 \uparrow R$	$\neq$	First in each run

In each case complementing L2 has the effect of ignoring the trailing or leading run.

Also changing the function L0 to its complement, i.e.

$$\begin{aligned} < \rightarrow \geq \\ > \rightarrow \leq \\ \neq \rightarrow = \end{aligned}$$

obtains the complement of the result.

Further, R need not be boolean. For numeric R,

$\neg 1 \text{ } L < \text{DIFF } R$ marks the starts of monotonic non-strictly increasing subsequences.

$\neg 1 \text{ } 0 > \text{DIFF } R$ marks the starts of monotonic strictly decreasing subsequences.

$\neg 1 \text{ } 1 \neq \text{DIFF } R$ marks non-repeating elements.

For the logical functions R must be boolean and we have:

L1	L2	L0	Extends runs of:—
1	1	$\vee$	1's on the left (including trailing 0)
$\neg 1$	1	$\vee$	1's on the right (including leading 0)
1	0	$\wedge$	0's on the left (including trailing 1)
$\neg 1$	0	$\wedge$	0's on the right (including leading 1)

"Extend" means "increase run length by one by overwriting", and extending say, 1's to the left is equivalent to reducing runs of 0's on the right.

In each case complementing L2 excludes the leading/trailing 0/1.

Yours sincerely,

Norman Thomson
IBM United Kingdom Laboratories Limited
Hursley Park
Winchester, Hampshire.

Editor: APL2 and STSC's Nested Array Research System allow a numeric left argument to the derived functions produced by the reduction and scan operators. The left argument is a scalar which defines the grouping over which the derived function is to take place. The positive Not Equals first difference is:—

$$2 \neq \backslash A$$

where the 2 specifies that a Not Equals reduction of the first element, the first two elements, the second and third elements, and so on up to the last pair of elements, is to be performed. The negative Less Than first difference is:—

$$\neg 2 < \backslash A$$

where the negative-2 specifies that the reduction is applied to reversals of pairs of elements in the right argument.

These are equivalent to the difference operator, but give more interesting results when the left argument is greater than two.

From Mr. D.J. Horton,

Sir: I have greatly enjoyed reading the first issue of VECTOR and am enclosing my entry to your competition.

There is much talk about all the enhancements to APL which are in the offing, and as a convert to the language I welcome any improvements. I feel, however, that as much effort should be put into making the language more "user-friendly". The identification and detection of errors is a particular area where I feel some useful work could be put in. For example, there are several quite distinct reasons for the message DOMAIN ERROR, (a splendid example of jargon), so why cannot the standard for the interpreter lay down greater intelligibility. Again, it is incomprehensible to me why the only division by zero which the interpreter handles is when the numerator is zero. Someone once told me it was an advantage for the machine to collapse in a heap when any other number is divided by zero. On the charitable assumption he was not an isolated case, then at least there should be a switchable option. The mixture of character and numeric data (pre APL2, I agree) is a bar, but how can an interpreter (VS APL) admit

`'1'=1`

as a valid statement (the response is zero) when, if PAGE is a numeric scalar, it would reject

`'PAGE',PAGE`

In face VS APL allows

`=/'PAGE'`

but rejects

`=\'PAGE'`

I am aware that all these niggles can be overcome with simple functions. My concern is that their existence turns APL into an awesome oriental mystery religion whose initiates gleefully converse in magical phrases unintelligible to the plebs. If programming is to be for all then the initial steps must not be too daunting for the timid outsider. Let us then rid the interpreter of gobbledegook and make allowances for "human" ways of thinking.

Yours sincerely,

D.J. Horton, C.A.
Budget Manager
Pfizer Group Limited
Sandwich, Kent.

Editor: With reference to 0 divided by 0, there are many good arguments for returning the result 0, rather than 1. See for example "Zero divided by zero", a paper by Eugene MacDonnell in the APL76 proceedings. There are also convincing reasons for producing a DOMAIN ERROR. See, for example, "The story of 0 divided by 0", by Joseph de Kerf in the APL80 proceedings. The desire to keep interpreters upward-compatible and to conform to the standard means that few implementors would be willing to change in this area. The best they can do is to provide, as you suggest, a switchable option. This was the approach taken in GEC's APL, where a system function is used to control the result of division by zero.

Your points about numeric and character data are worth looking at more closely. The expression

```
'1'=1
```

does indeed produce the result 0, and this is the only reasonable answer if we don't want to report a DOMAIN ERROR. The fact that we do get a result is very useful — it provides VS APL with a mechanism for determining the type of an array. (The expressions

```
0=0\0/,W
```

or

```
0=1↑0pW
```

will yield a 1 if W is numeric, otherwise a 0). Catenation of a character array with a numeric array does cause a DOMAIN ERROR, and this is reasonable in a language which does not support heterogeneous arrays. The definitions of "equal" and "catenate" are different, and so we should expect different results. To understand the reason for the result of

```
=/'PAGE'
```

we must recall the informal definition of reduction — the function is inserted between each element of the argument. Therefore the result is

```
'P'='A'='G'='E'
```

The first comparison is between two characters ('G' = 'E'), which yields a numeric result. The second and subsequent comparisons are then always of the type 'character = numeric', which must give a 0 (recall '1' = 1). The final result must always be 0 for character vectors containing more than two elements. For two element vectors the result is 1 or 0 dependent on whether the characters are the same or not. For scalars or one element vectors the function is not applied and the result is the scalar character, because a dyadic function can be applied between only two arguments. The result of

```
=\'PAGE'
```

now follows from the definition of scan, which composes its result by performing a set of reductions on windows of its argument. i.e.

```
=\'PAGE'    ↔    (='P'),(='PA'),(='PAG'),='PAGE'
               ↔    'P',0,0,0
```

and the DOMAIN ERROR comes indirectly, because character and numeric data cannot be catenated in VS APL. In APL2 this restriction is lifted, and the indicated result would be returned — a vector whose first element is the character '1' and whose final three elements are numeric 0s.

Any improvement to APL's "user friendliness" would indeed be welcome, particularly in the area of error messages. (A recent estimate by Joseph de Kerf put the total number of different error messages in 50 dialects of APL at 1500!). In fact, the message DOMAIN ERROR has a very specific meaning, which has been borrowed from mathematics; if a function argument is outside the domain of that function then the error is reported. Thus division by zero, catenation of a numeric with a character and square-rooting a negative number all report DOMAIN ERROR in VS APL, because the arguments lie outside the domains of the respective functions.

The ISO APL standard sensibly uses a new error class — LIMIT ERROR — to report errors which are not strictly errors of domain, but limitations of the system. Attempting to evaluate by multiplication a number whose magnitude is larger than the machine limit, for example, is an error caused by the physical limits of the machine (or APL interpreter) rather than a theoretical deficiency of the multiply function.

PRIZE COMPETITION: MATRIX NUMERIC VALIDATION

by David Ziemann

For the poor underprivileged souls who are forced to use APL interpreters that do not boast system functions such as 'quad-VI' and 'quad-FI' the problem of numeric validation is an important one. It is also relevant to those who wish to write standard-conforming code — the ISO APL standard will not include such niceties as these two system functions.

Typically, the problem is to verify a character vector which purports to contain valid numbers, where each potential number is separated by one or more blanks. The function VALIDNUMSV validates such a vector by returning a 2 column numeric matrix, as follows:

```

      VALIDNUMSV '3 1.1 -1.1  1A3 0 1E2 '
1      3
0      0
1      -1.1
0      0
1      0
1      100

```

Column one contains a 1 when the corresponding number has been validated successfully, and a 0 otherwise. (See VECTOR Vol.1 No.1 p.33 for clues on how to write such a function.)

However, with the increase in demand for full-screen applications the more natural approach is towards the validation of character matrices rather than character vectors. This is because the user is often required to enter numeric data into multi-row screen fields. It is normal to restrict the user to entering only a single number into each of the field rows.

Our competition this issue to to write an APL program that will perform numeric validation on a character matrix. The minimum requirements for the program are as follows:

1. It must correctly validate a character matrix, each row of which contains the representation of a single number.
2. It must provide the correctly converted numbers as well as an indication of which numbers were acceptable.
3. It must be possible to specify a validation code that restricts the set of acceptable numbers.
For example:

1	—	0 or 1 accepted only
2	—	whole number input only
3	—	whole or decimal numbers only
4	—	whole, decimal and E format numbers accepted
4. The program must conform to the first draft proposal for an ISO APL standard (don't worry too much — if you assume VS APL and you're not too indiscreet, the result should be acceptable).

Respondents should not feel dissuaded however, from exceeding these minimum requirements in any way.

The judges will consider the following criteria in making their decisions: usefulness, robustness, ease of use, generality, efficiency and elegance. The standard of the supplied descriptions of the facilities provided will also be taken into consideration.

A first prize of £30 (or equivalent) will be awarded to the author who, in the judges' opinions, submits the best entry. Two commendation prizes of £10 each will additionally be awarded. The closing date for entries is 31st December 1984.

Competition Rules

- Entries must be in legible English or APL as appropriate and should preferably be machine produced.
- The date and your full name and address should appear on each sheet of your entry.
- Entries should be physically separate from other contributions such as letters, and should be clearly marked 'Competition Entry'.
- All submissions should be sent to the editor.
- Those on the committee, activities group or journal group of the British APL Association are ineligible.

Prize Competition: This is your Life

Due to late distribution of the last issue, the winners of Prize Competition 1 will be announced in the next issue.

APLpip — THE HARD SEED OF A HEALTH AND WEALTH GIVING FRUIT

by L. Llewellyn-Jones

I was very interested in the matter of an ASCII-based character set for APL which was the subject of Mr Bowman's letter as well as being a topic of numerous articles elsewhere in VECTOR Vol. 1 No. 1. I became so moved by the force of the arguments in its favour that I felt impelled to sit down and try to design one which would lose less in brevity than FGL, because brevity is not only the soul of wit, it is also one of APL's greath strengths.

Now APL has a set of about 140 characters whereas ASCII, a 7-bit code, has only 128 made up of:— 32 characters used for controlling communication and printing; space; DEL; and 94 graphics. If you set aside two of these for use as set selectors (like FIGURE shift and LETTER shift in BAUDOT code) you can have two sets of 94 codes which together are large enough to encompass APL's 140. However, I was less concerned about this than with trying to squeeze a workable APL scheme into a single sheet. This task that I set myself was successfully completed, and now — with the benefit of hindsight — performing it seems to have been fairly straightforward. Here's how it went.

Trying to fit 140 characters into a set of 94 is like trying to put a quart into a pint pot — you can't do it without spilling some. The characters I thought were eminently spillable are those which are not presently in common use, and along with them went the UNDERBAR, UNDERBAR DELTA, and the UNDERSCORED ALPHABETICS — relics from the days of backspacing and overstriking — nice to keep as souvenirs, but not essential, and therefore a luxury that I just couldn't afford. This brought the number of characters required for a one-to-one mapping down by 35, but it still wasn't enough.

I was aware of two commonly known multi-character substitutes for SLOPE BAR and SLASH BAR, so I knew I could save another two codes by using those instead. Further savings, I thought, could come from substituting compounds such as NOT AND and NOT OR for NAND, NOR etc, but then I remembered that NOT can only be used monadically whereas NAND and NOR can also be used dyadically, so I had to think again. I finally settled on three compounds culled from BASIC to substitute for NOT GREATER etc, and I made up three others which I find aesthetically pleasing as replacements for NAND etc. Unlike the first two, the last six would cause SYNTAX ERROR in conventional APL, but in a purpose-built interpreter they would be tokenised and stored internally as single codes which would function in precisely the same way as the APL symbols which they represent. So, by eliminating 43 APL symbols in all, I had just enough ASCII codes to do the job, and the task of mapping one set to another could begin.

With APL and ASCII sharing a common subset of 56 characters I was off to a flying start and more than half way there. I then looked at the remaining ASCII symbols and was greatly surprised by the ease with which each seemed to find its APL counterpart. It was almost as if someone up above was smiling on me! Eleven of them are imitative, one is symbolically synonymous, and DOLLAR, which looks nothing like DEL(ta) does at least have a name which sounds similar. It also carries with it an air of authority which makes it very suitable for the important role of function definition. I very much prefer this restrained usage of the symbol to the profligate way it is used in BASIC to make program listings look cluttered and data names unpronounceable. And now, with 69 character assignments made, I was nearly three-quarters of the way there, and had only to find 25 equivalences from the lower-case alphabet which held all the remaining ASCII characters.

The characteristic which I most prized in carrying out this task was mimicry. Nine of the lower-case alphabetics possess this attribute, in degrees ranging from excellent to fairly good. Four of them are also mnemonic in the sense that they prefix common names for the APL symbols they represent, another valuable attribute, and the only one available for deciding the rest of the assignments. Six more were matched on this basis alone, but for the last ten the best I could

do was to look for words or phrases which began with the available letters and which were synonymous with the action or purpose of the APL symbols to which they would be assigned. You might not like the words that I came up with, but they will all be remembered by their initial letter mnemonics. The unassigned letter is k. Together with the ASCII DEL it provides a pair of set selectors which will enable a second set to be defined later on. Finally, there remained only two more things to do: what to call this creation; and what to do with it.

I chose the name APLpip because it encapsulates my hopes for its future — a vision of growth and proliferation for my favourite programming language. I am donating the copyright to the British APL Association because I think it will be the best body to promote and regulate its usage.

Here are some of the things it can provide:—

- (1) A means of representing APL code in print.
- (2) A means of transmitting and listing APL workspaces of all sorts under ASCII protocol.
- (3) A basis for alternative interpreters enabling:—
 - (a) APL software to run on unmodified ASCII hardware and yet be maintainable 'in situ'.
 - (b) APL to be learned and used on very low cost hardware by the school and hobbyist communities.

I will discuss each one briefly.

APLpip for publishing

APLpip can be used to overcome the typesetting problem referred to by the Editor on page 5 of VECTOR Vol.1 No.1

The atomic vector of VS APL — and many others, as well, I suspect — contains most of the ASCII symbols as well as all those of APL. It is easy in APL to write a function whose argument is the name of any function in the workspace and get it to convert the function's canonical representation to APLpip format. Thus contributors can create copy for typesetting without any risk of mistranslation. Similarly, recipients can enter the APLpip format and use a complementary function together with quad-FX to create a ready-to-run orthodox APL program.

Adoption of this procedure would not only overcome the typesetting problem, but also make available to schools and hobbyists using APLpip interpreters high quality software which would otherwise have to be translated before being entered.

Function listings look hardly any different from normal APL code because 95% of the graphics used to represent the functions are either part of the APL set or ASCII lookalikes.

ASCII protocol

The availability of a pair of set selectors permits one more set of 94 characters to be used to accommodate the 43 APL codes excluded from the first whilst still leaving plenty of spare room for expansion. This means that APL workspaces of all sorts could be transmitted and printed under ASCII protocol. Second set characters are so infrequently used that the overhead in characters to switch from one set to another would not be very significant. A printing convention of underlining characters of the second set would enable them to be visually distinguished from those of the first.

APLpip Interpreter Advantages

APL software is often run at non-APL terminals, but the lack of APL symbols on their keyboards can sometimes leave programmers helpless. A STACK FULL condition can prevent them from doing anything but using system commands or calculator mode, for a CLRSTACK function is rendered inoperable when its name cannot be pushed onto the stack. APLpip would enable them to overcome this problem by simply entering "g" and then CLRSTACK. It could also enable any software maintenance to be carried out 'in situ'.

Because demand is elastic and competition has been fierce the schools and hobbyist sectors of computing have become huge. Prices are keen, buyers fastidious, and accustomed to getting enormous value for money. And yet, with a few notable exceptions, APL has made no impact on these markets whatsoever. Robert Bittlestone has said, and he should know, (*Ed. — should he?*) that one cause of this is the inordinately high cost of using APL symbols. So if you remove this obstacle, APL should flourish, shouldn't it?

Three of the Association's sustaining members have great experience of developing and marketing APL interpreters written for Z80 machines, but because of the move towards more powerful 16-bit processors their Z80 products, for the most part, are languishing. Here is an opportunity to refurbish them and bring them to buyers with suitable machines who know a bargain when they see it. "Suitable machines", in my definition, include those with cassette storage, and are exemplified by the Amstrad CPC 464 (first year's sales forecast of 200,000 units) which goes on sale in September 1984 at £239 for a complete hardware kit. But there are a great many others including the Sharp MZ711, RM390Z, Coleco Adam, Lynx, Spectravideo, the various MSX machines, and even that evergreen, updated, old-timer Apple IIe. Eventually they will all have disk drives, but in the meantime many owners would be thankful for an interpreter containing even a subset of APL which will allow them to)SAVE a 10-20kb workspace on a separate cartridge.

Learning APL with an APLpip interpreter presents difficulties because every text-book on the subject shows the primitives in orthodox symbols. This is a real problem, but it shouldn't be exaggerated. A good half of APLpip's 38 ASCII substitutes are lookalikes and those that aren't are mnemonic. At worst there is an additional burden of recognising a dozen different symbols and learning a new alphabet, something we all managed with ease in our kindergarten days. I believe the way to tackle this problem is to start "bilingually" with decals bearing the orthodox symbols affixed to the alphabetic keys. There should also be a "bilingual" introductory text. An orthodox one already exists priced at \$2.95. Perhaps Messrs. Smith and Camacho would use their considerable communicative skills to create a "bilingual" version. Oh, and don't forget to provide a PF (programmed function) key causing the unshifted alphabetic keys to be read as upper-case in APL mode and vice-versa in TEXT mode.

APLpip — APL/ASCII Character Mappings

		CHARS USED	
(1) INESSENTIALS NOT INCLUDED		35	0
A-B I α ω _ n u Δ ↵			
(2) EXACT ASCII EQUIVALENTS			
space		1	
A-Z		26	
0-9		10	
. , : ; ! ? ' - + * = < > () [] / \		19	56
(3) IMITATIVE AND OTHER SYMBOLS			
Λ ‡ + □ ▣ ▤ ▥ ▦ ▧ ▨ ▩			
& % _ ! @ \$ ^ ~ " { }		13	13
(4) LOWER CASE ALPHABETIC SUBSTITUTES			
IMITATIVE ONLY			
•orvx			
copvx		5	
IMITATIVE AND MNEMONIC			
ε		e	psilon
ι		i	ota
l		l	ower style
t		t	ake
MNEMONIC ONLY(Conventional names)			
d		d	rop
j		j	ot
m		m	atrix inverse/divide
n		n	ot
q		q	uote quad
u		u	pgrade
MNEMONIC ONLY(Unconventional names)			
a		a	ctuate
b		b	igger/biggest
f		f	alling sequence
g		g	oto
h		h	ead-over-heels
r		r	eflect diagonally
s		s	plit (scalars and vectors into vectors and matrices whilst decoding)
w		w	eave (vectors and matrices into scalars and vectors whilst encoding)
y		y	aw (nautical term)
z		z	eroes in decimals etc.
		10	25
(5) COMPOUND SUBSTITUTES			
\		\	[1]
/		/	[1]
<=		<=	
<>		<>	
=>		=>	
<v>		<v>	
<A>		<A>	
<\$>		<\$>	
		6	
		8	0
Reserved as Set Selectors ASCII DEL and k		94	
ASCII codes 32 to 127		2	
		96	
		=	

STATISTICAL COMPUTING WITH APL

by Alan G. Hawkes

The programming language APL has been around since the mid sixties. It has been slow in gaining widespread acceptance, especially in the UK, being thought of as the province of 'mathematicians, statisticians and nutcases'. This is because it is quite unlike the other major languages, with a lot of funny symbols (mainly Greek) which make the code very concise and mathematical-looking. This tends to put people off at first sight and means that, if you want to stay sane, you need special terminals. In operation it tends to be greedy in CPU time, making the user unpopular with the computer manager, but the days of cheap independent processors are doing away with that problem.

In the last three years or so the use of APL has started to grow rapidly in this country, with the membership and activities of the British APL Association expanding fast. Several small, lively companies specialising in APL hardware and software are active. And funnily enough the vast majority of users are not mathematicians or statisticians, many of whom have never heard of it, but accountants, management services departments and industrial and commercial research people. The essential reason is that it is cost effective; they want answers today or tomorrow, not in three months time. They want the flexibility to make changes and try out new ideas quickly, not packages that have a set routine and take ages to alter. It is typically much faster to develop programs (sets of functions) in APL than in other languages. Today it is people-time that is expensive, not machine-time. An added bonus is the elegance and logical simplicity of the language which makes it fun to work with, in a way that slogging through a hundred lines of FORTRAN can never be. And if you do write a package in APL, it can easily be arranged that the end user doesn't have to see those funny symbols and need only use English language.

Language of Statistics

In my view, of all the computer languages currently available, APL is the natural language of statistics. I would urge all statisticians, who have not already done so, to spend two or three days giving it a serious try, say with a copy of Gilman and Rose (1); then they can make up their own minds. There are many reasons for this view of which some are as follows:—

- a) It is flexible, interactive and very powerful. Often only a few minutes are required between having an idea and getting the results back, making it easy to experiment or try out different forms of analysis depending on the way the data looks. Data is easily manipulated.
- b) There are several built-in functions which are very useful to statisticians. For example !R is factorial R, if R is an integer, or, more generally, the gamma function $\Gamma(R+1)$. The dyadic form of the same symbol used as R/N produces the combinatorial

$$\binom{N}{R}$$

A random sample without replacement of R objects from a finite population of data stored in a vector X can be taken simply by

$$Y \leftarrow X[R ? \rho X]$$

which assigns the sample values to a new vector Y (what, no dimension statement?). What are the properties of the sample? You might fancy something a bit unusual like the mean of all the sample values excluding the two largest and two smallest (assuming there are more than four); that can be found simply by

$$(+ / Z) \div \rho Z + 2 + ^2 Y[4 \downarrow Y]$$

Yes I know it looks queer, but it is really very easy when you know how and only takes about a minute to think of it, type it and get the answer back. How long would it take in FORTRAN or BASIC? In the above expression

$$Y[\Delta Y]$$

gets the data into numerical order,

$$2+^{-}2+$$

drops two items off each end and the rest of the statement calculates the mean

$$pZ$$

tells you how many objects there are and

$$+ /$$

adds them up. Note that APL goes from right to left.

- c) Like Glim, GENSTAT or MINITAB the language is array oriented in data structure and computation, but easier to use and more versatile. For example ! 2 3 4 yields the result 2 6 24 being the factorials of each value in the vector 2 3 4. A nice example

$$(R \text{ IN}) \times ((1-P) * N - R) \times P * R + 0, 1N$$

calculates all the probabilities $p(0), p(1) \dots p(N)$ for Binomial distribution Bin (N,P).

- d) The language is modular in structure with the natural modules being user defined functions. Each function has a name and optional right and left arguments. For example, one of mine, NORMTAIL X calculates the upper tail probability of the unit normal distribution corresponding to each element of any array X you supply, e.g. NORMTAIL -1.96 0 2.576 elicits the immediate response

$$0.975 \ 0.5 \ 0.005$$

It is very easy to make use of functions you have previously written for another purpose. Even better, with a group of people working on the same system a significant increase in productivity is possible by using functions others have already written, or perhaps will write for you during the course of the afternoon if problems are freely discussed among the group.

- e) Matrix algebra is very simple. Consider the following matrix expression

$$S = P(I - SAB)^{-1}LU$$

If the matrices on the right already exist, then S can be computed by

$$S \leftarrow P + . \times (I - S + . \times A + . \times B) + . \times L + . \times U$$

Apart from a slight change of notation, this has the great advantage that it is transparently obvious that it represents the above matrix expression, and you get an immediate answer.

$$\boxtimes X$$

computes the inverse of X. There is also a more general dyadic use of this symbol. Thus if Y is a vector and X a matrix of predictor variables, then

$$B \leftarrow Y \boxtimes X$$

supplies the vector of regression coefficients obtained by least squares fit.

$$R \leftarrow Y - X + . \times B$$

then finds the residuals and

$$+ / R * 2$$

calculates the residual sum of squares.

Why APL?

There are a number of good non-APL statistical packages widely available which have been refined over many years. Some are better than others, but most have a restricted range of what you can do. If you have a lot of routine analyses to do you will probably be quite happy with three or four, say SAS, CLUSTAN and GLIM, depending on your field of application. However, with APL you can soon build up a package of standard functions and leave yourself free to tie them together how you like, or slot in new functions as required; in other words leaving yourself in total command of the situation, not restricted to what the package writer will allow you to do. If you want ready made APL software, there are good published functions in Anscombe (2) and some in the periodicals VECTOR and APL Quote Quad. Packages such as STATGRAPHICS (from the USA) and various specialist ones from the Applied Statistics Research Unit at the University of Kent, from Swansea University, and elsewhere, are starting to become available.

Thus you have the ability to use other people's packages or functions as well as your own, tie them up together with the use of very powerful raw APL (as indicated in the previous section) in a flexible mode of operation.

Similar advantages occur in using APL for teaching statistics. In Swansea, apart from service teaching, all undergraduate statistics courses now assume the student has a basic fluency in APL and the ability to use collections of functions relevant to each course, e.g. time series functions, multivariate functions etc. Students begin at the terminal the first week of the first year. They can be eased gently into computing in easy stages, yet are very quickly able to do significant calculations, using the machine like a super calculator. Within five weeks students cope quite happily with all the examples of the previous section, except matrix algebra and regression which they meet at the end of the first year. Students are able to analyse much more data and the calculations can be organised so that the process of carrying them out enhances the student's understanding of the theory. With most packages it is a question of 'numbers in, numbers out' with little idea of what went on between. We find that students take to APL very readily, being much easier to learn than BASIC. For further details see Hawkes (3).

Possible disadvantages have been mentioned before. You need the equipment and the interpreter, but these are now quite cheap. Secondly, APL can be a bit slow in machine-time on large scale number crunching (not always by any means). The primitive functions are very fast, but if you write loops which are executed many times (fortunately this is often not necessary) the interpretive overheads can be quite high. Compilers have been developed, but are not widely available. However, one or two versions of APL are said to be semi-compiled and should offer a considerable improvement in performance. But if you value people-time more highly this may not be at all important. APL*PLUS have just introduced a limited compiling facility, while IBM have the production of an APL compiler as one of their research priorities.

Which APL?

Fortunately, unlike BASIC, there has for some time been a high degree of standardisation between different interpreters. I call this 'standard' APL. In the last two years or so there have been introduced at least four 'second generation' APL interpreters which enhance and enrich the language significantly. To some extent each has developed in its own way, but there are encouraging signs that the suppliers are talking to each other, so that a new standard may emerge. All embrace the idea of 'nested arrays' implemented in various ways. These allow a flexible and convenient data structure allowing numeric and string arrays of various shapes to be mixed together in a tree-like structure, together with powerful functions and operators with which they can be manipulated. IBM's APL2 and I.P. Sharp's APL use complex numbers as the basic field for arithmetic; other interpreters are likely to follow this lead in the future. The above interpreters run on IBM mainframes, and an I.P. Sharp version recently became available for the IBM PC

micro, allowing the possibility of access to their worldwide information services. There are also a number of standard APL's for the PC including IBM and APL*PLUS. The latter is due also to be released on the DEC Rainbow in August 1984.

Dyadic Systems, a British company, have a second generation interpreter called DYALOG. This is written in 'C', runs under the UNIX operating system and incorporates a lot of UNIX facilities within the language. This makes it particularly easy to interface with other processes which run under UNIX. For example you could write a special routine in the very efficient compiled language 'C' which could be called directly from APL and the results returned to APL. A statistical package written in FORTRAN, such as SPSS, might be called in a similar way. APL is very good at glueing things together in this way, especially in a multi-tasking environment like UNIX. APL*PLUS also has a second generation interpreter which runs on IBM mainframes and has recently produced a version written in 'C'. Like DYALOG this gives it great portability over a wide range of machines run under UNIX. As it is said to be semi-compiled, it should run quite fast. Both of these interpreters should now be available on a wide variety of machines including VAX, Perkin Elmer, Hewlett Packard, PERQ and many based on the currently popular MC68000 16-bit chip such as Cadmus, Bleasdale, Fortune, Wicat, Cifer and Torch. A Cadmus running DYALOG or APL*PLUS appears to be a particularly powerful system at a reasonable price.

Standard APL's, in addition to those already mentioned for the IBM PC and Rainbow, include VAX APL by Digital and APL68000 which runs on Sages and on the Spectrum (produced by MicroAPL, not Sinclair!). These are proven, reliable tools at the minimicro level. Standard interpreters are also available on PRIME and on most mainframes. For 8-bit micros a good standard APL is VIZ::APL, which runs under CP/M. Some 8-bit APL's tend to lack the more useful features such as matrix inversion, so look carefully at the specification.

Earlier this year there was the announcement of Acornsoft APL, developed by Inner Product Ltd., which will run on a BBC micro fitted with the new Z80B second processor and a suitable 80-track disk drive. This incorporates ready access to the excellent BBC colour graphics. As it is not likely to be in the shops before August 1984, I am unable to comment on performance. The specification sounds good, and if it comes up to expectation, it should provide a cheap APL machine that will find its way into many schools, colleges and homes. Furthermore, I understand that a good sixteen bit British APL micro is likely to be on the market at under £1,000 within the next year. Then the growth in APL usage should be really fast.

Finally, if you can afford at least £60,000, try The APL Machine being developed by Analogic. It is a small array processing machine for an array processing language which can execute APL with impressive speed.

(This paper first appeared in the Professional Statistician)

REFERENCES

- L. GILMAN and A.J. ROSE (1984), *APL an interactive approach*, 3rd ed., Wiley.
F. ANSCOMBE (1981), *Computing in statistical science through APL*, Springer.
A.G HAWKES (1983), The role of APL in statistics teaching, *APL Business technology 1983*, UK APL User Group, 123-132.

NUMERICAL INTEGRATION AND PROBABILITY DISTRIBUTION

by Professor Alan Hawkes

At school we learned a lot of clever tricks to integrate a wide class of functions $f(x)$ to get a definite integral

$$\int_a^b f(x)dx$$

Unfortunately, there is an even wider class of functions which occur frequently in all branches of science and engineering for which no algebraic solution can be found in terms of known simple functions. This is certainly the case in statistics, where if $f(x)$ is a probability density function the above integral represents the probability of an observation falling in the interval (a,b) . In such cases one must resort to numerical integration techniques to get numerical answers. Often the results are tabulated for users to look up, such as the normal, t , chi-squared, and F distributions which are so commonly used in statistics and found in e.g. Biometrika Tables by Pearson and Hartley (1958).

A lot of ingenious methods were developed to get good approximations with as little calculation as possible, using analytical techniques to place error bounds on the results. Biometrika and other tables were computed to very good accuracy, with great skill and dedication, using the famous Brunsviga hand calculating machines. With modern computers, especially using APL, it is so easy to calculate the function over a large grid of points that simple techniques will be perfectly adequate for most purposes, and accuracy may be controlled simply by repeating the calculation over a finer grid to see if results differ by less than some prescribed accuracy; if so, stop, otherwise make the grid finer again.

The simplest well known technique, the trapezium rule, approximates the function by series of straight lines. Very nearly as simple, and usually quite a bit more accurate for the same grid size is Simpson's rule. If the interval (a,b) is divided up into $2M$ intervals with division points at

$$a = x_0 < x_1 < x_2 \dots < x_{2M} = b$$

the function $f(x)$ is approximated by quadratic curves, rather than straight lines, over each pair of intervals (x_0, x_2) (x_2, x_4) This leads to the approximate integral

$$\int_a^b f(x)dx \doteq f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \dots + 4f(x_{2M-1}) + f(x_M)$$

The APL function $Y \text{ NUMINT } X$ enables one very easily to carry out numerical integration using Simpson's rule for any well behaved function (one which is unbounded, for example, would need special treatment). The full specification of the right argument is a three element vector $X \times A, B, N$. The result is then an N -vector containing the integral of the function over each of N equal subintervals

into which the interval (A,B) is divided; e.g. if $X+1\ 5\ 2$ the result consists of the integral over $(1,3)$ and the integral over $(3,5)$. If N is omitted, so that $X+A,B$, it is automatically assumed that $N+1$ and the result is simply the integral over the whole range (A,B) . If X contains a single item only, $X+B$ we assume $A+0$ and $N+1$ so that the result is the integral over $(0,B)$.

The function that is to be integrated is specified by the left argument Y in the form of a string vector. The usual form taken by Y will be the name of a function with an optional left argument to specify the shape if required. The function is expecting a vector as its right argument and returns as a result a vector of the same shape, containing the value of the function at each point of the input vector. The string is catenated with an appropriate 'GRID' in line [21] and executed, $\downarrow$; this is also done with a different value for GRID in line [23]. In general one thus has a statement of the form 'A functionname', NUMINT X, with A a shape parameter vector.

2 EXAMPLES USING THE NORMAL DISTRIBUTION

The probability density function, pdf, of the standard normal distribution is given by

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

This is calculated for each element of the vector X by the function *NORMALPDF* X. As an example the probability attached to the interval $(1.5,2.4)$ say is

$$\int_{1.5}^{2.4} f(x) dx \text{ which is calculated by}$$

```

)DIGITS 4
'NORMALPDF' NUMINT 1.5 2.4
0.0586                                while
'NORMALPDF' NUMINT 1.96
0.475

```

is the probability over the interval $(0, 1.96)$. The probabilities for the three intervals $(0,1)$, $(1,2)$, $(2,3)$ can be found from

```

'NORMALPDF' NUMINT 0 3 3
0.3413 0.1359 0.0214

```

For the normal distribution the probability of the random variable X being less than a given value $x > 0$, can be written as $P(0 < X < x)$. This can be computed for each $x = 1, 2, 3, 4$ say by

```

)DIGITS 7
0.5 + + \ 'NORMALPDF' NUMINT 0 4 4
0.8413447 0.9772499 0.9986501 0.9999683
- (1)

```

which agrees exactly with results in Table 1 of the Biometrika Tables.

The accuracy of integration is controlled simultaneously for all intervals over which the integral is required by a global variable $\Delta INTERR$ which was set at $1E^{-7}$ for all the above examples. Suppose now we go right up into the tail and require the probability for each of the intervals (14,16), (16,18) and (18,20)

```
)DIGITS 4
'NORMALPDF' NUMINT 14 20 3
1.833E-44 1.712E-57 2.933E-72
```

Now these are very much smaller than the accuracy setting, so we should not be surprised if these were not too good. If now we let

```
 $\Delta INTERR = 1E^{-50}$  and try again
'NORMALPDF' NUMINT 14 20 3
7.794E-45 6.389E-58 9.741E-73 (2)
```

That's much better, these are known to be correct to four figures.

In principle it would be a good idea to make $\Delta INTERR$ smaller still to cope with some of the very small values. However, with $\Delta INTERR = 1E^{-60}$ this soon leads to *WS FULL*! The reason is that at this level of accuracy the grid has to be very fine indeed over intervals for which the integral is relatively large. The moral of this is you should not simultaneously use intervals over which the values of the integral vary by a very large amount, and that $\Delta INTERR$ should not be made much smaller than you really need it. In isolation, with $\Delta INTERR = 1E^{-80}$

```
'NORMALPDF' NUMINT 18 20
9.741E-73 works perfectly well. The WS FULL problem previously was obviously due to the first interval.
```

These calculations were done on a PRIME 750. If you are working on a much smaller machine you would run out of space quicker. In that case you may prefer to take a different approach and approximate the integral by a suitable algebraic function. In the case of normal distribution this is done in Hawkes (1982) and is implemented in the function *NORMTAIL X*. This is known to be accurate to at least four significant figures, so for example compute

```
(-1+Z) - 1+Z+ NORMTAIL 14 16 18 20
7.793E-45 6.388E-58 9.741E-73
)DIGITS 7
1 - NORMTAIL 14
0.8413451 0.9772489 0.9986502 0.9999683
```

These should be compared with results (2) and (1) respectively which are accurate to the number of figures shown. The approximation in *NORMTAIL* is good enough for most purposes and requires less workspace than using *NUMINT* (although it is not faster).

3 SOME MORE DISTRIBUTIONS

The shape of the probability density of student's *t* distribution depends on a parameter known as the degrees of freedom, *DF*. This is calculated at a vector of points *X* by the function *DF TPDF X*. Then for the *t* distribution with 5 degrees of freedom say we can find

```
)DIGITS 5
0.5 + \5 TPDF NUMINT 0 4 4
0.81839 0.94903 0.98495 0.99484
```

which agrees exactly with the results in Biometrika Table 9. Again exact agreement with Biometrika Table 7 is found from the chisquared distribution with 12 degrees of freedom.

```
1 - + \ '12 CHISQPDF' NUMINT 0 24 4
0.91608 0.44568 0.11569 0.02034
```

which are the probabilities of exceeding 6, 12, 18, and 24 respectively.

The chisquared distribution is a special case ($\alpha = DF/2$, $\lambda = 1$) of the gamma distribution which has pdf

$$f(x) = \lambda^\alpha x^{\alpha-1} e^{-\lambda x} / \Gamma(\alpha), \quad x > 0.$$

This is computed in *A GAMMAPDF X* where *A* is a two element vector containing values of α and λ .

Note that if $\alpha < 1$, you need to be careful not to include zero in any interval to be integrated over because the function tends to infinity. If $\alpha = 1$ we have the well-known negative exponential distribution

$$f(x) = \lambda e^{-\lambda x}, \quad x > 0.$$

Of course, we do not really need to use numerical integration on this one, but it can be done very simply, e.g.

```
'0.5*-0.5*' NUMINT 0 8 4
0.63212 0.23254 0.08555 0.03147
'1 0.5 GAMMAPDF' NUMINT 0 8 4
0.63212 0.23254 0.08555 0.03147
```

of course gives exactly the same result. The former version, however, is interesting because it is done with only raw APL in the left argument.

Our range of basic probability distributions is completed by the *F* distribution and the Beta distribution, each having a left argument which is a vector of 2 elements.

```
'6 10 FPDF' NUMINT 5.39.
0.99003
'325 577 BETAPDF' NUMINT .36 .38
0.39577
```

in agreement with p.44 of Soper (1921), evaluates the integral

$$\int_{.36}^{.38} \frac{x^{a-1} (1-x)^{b-1}}{B(a,b)} dx$$

for $a = 325$, $b = 577$. Again note that there may be difficulties at $x = 0$ and/or $x = 1$ if a and/or b is less than unity

Of course a, b are not restricted to integers

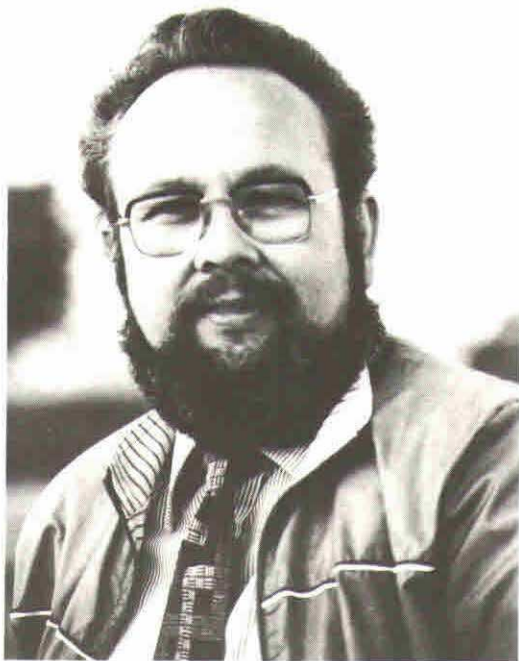
'16.1 5.2 BETAPDF' NUMINT 0.6
0.05671

agrees with the result on p.17 of Soper.

REFERENCES

- E. S. PEARSON and O. HARTLEY (1958), *Biometrika Tables for Statisticians Vol.1*, Cambridge University Press.
- A. G. HAWKES (1982), Approximating the normal tail, *The Statistician*, 31, 231-236.
- H. E. SOPER (1921), *Tracts for computers No. VII*, Cambridge University Press.

Professor Hawkes



```

V R+Y NUMINT X;N;RANGE;RT;DX;GRID;FO;FE;M;S
[1] A NUMERICAL INTEGRATION. <Y> IS A STRING OF FORM 'A FNNAME'
[2] A THE GLOBAL <AINTERR> SPECIFIES ACCURACY
[3] A RESULT IS INTEGRAL OVER <N> SUBINTERVALS OF X[1],X[2]
[4] XCHECK:→((1≥ρX)^(3≥ρ,X))/XOPTIONS
[5] XMESS:'X SHOULD BE VECTOR OF SHAPE 3. X[3] IS NUMBER OF '
[6] 'INTERVALS INTO WHICH (X[1],X[2]) DIVIDED'
[7] 'DEFAULT X[3] IS 1 IF NOT SPECIFIED'
[8] →0,0ρ□+'IF ONLY SCALAR X GIVEN THEN IT IS TAKEN AS X+0,X,1'
[9] XOPTIONS:→(1<ρ,X)/TWO
[10] X+0,X
[11] TWO:→(3=ρX)/XCHECK2
[12] X+X,1
[13] XCHECK2:→(X[3]≠1[|X[3]|])/XMESS
[14] M+1
[15] R←(N+X[3])ρ0
[16] RANGE+X[2]-X[1]
[17] LOOP:RT+R
[18] M+M×2
[19] DX+RANGE/S/M×N
[20] GRID+X[1]+DX×0,1S
[21] FO+Y, ' GRID'
[22] GRID+X[1]+DX×(1S)-0.5
[23] FE+Y, ' GRID'
[24] R←(DX+6)×+/(N,M)ρ(4×FE)+(1+FO)+-1+FO
[25] →(AINTERR<[|R-RT|]/LOOP
V

```

```

V R+NORMALPDF X
[1] R←((O2)*0.5)×*-0.5×X*2
V

```

```

V R+DF TPDF X
[1] R←((!0.5×DF-1)÷(1-1+DF÷2)×(ODF)*0.5)÷(1+(X*2)+DF)*0.5×1+DF
V

```

```

V R←DF FPDF X
[1] R←(1+0.5*+/DF)×((#/DF)*0.5×1+DF)÷×/!1+DF÷2
[2] R←R×(X*1+0.5×1+DF)÷(1+X×#/DF)×+/DF÷2
V

```

```

V R←A BETAPDF X
[1] A PDF OF BETA DISTRIBUTION
[2] R←((1++/A)÷×/!A)×(X*1+A)×(1-X)*1+A+A-1
V

```

```

V R←A GAMMAPDF X;L
[1] A <A> IS ALPHA, LAMDA
[2] R←(X*A)×(*-L×X)÷(!A*1+A-1)÷(L+1+A)*1+A
V

```

```

V R←DF CHISQPDF X
[1] R←((DF÷2),0.5)GAMMAPDF X
V

```

```

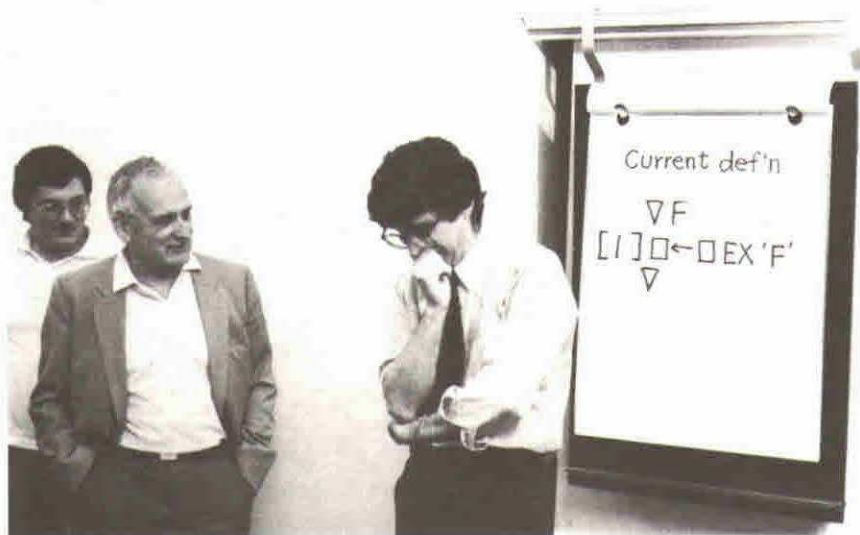
V R←NORMTAIL X;Z;T;Q1;Q2
[1] A UPPER TAIL N(0,1) FOR ARRAY <X>. HAWKES 1982-STATISTICIAN
[2] Z←|X
[3] T←Z×Z≤2.2
[4] T←T×1+T×T×7.5166E-3+T×T×3.174E-4-2.965E-6×T×T
[5] Q1←0.5×1-(011-×-T×T÷0.5)*0.5
[6] Q2←(1.184+Z)×(*-Z×Z÷2)÷((1.209+Z×1.176+Z)×(02)*0.5)
[7] T←Z≤2.2
[8] R←(T×Q1)+(1-T)×Q2
[9] T←X≥0
[10] R←(T×R)+(1-T)×(1-R)
V

```

APL ISO STANDARD MEETING

"Your two minutes is up, Mr. Morrow."

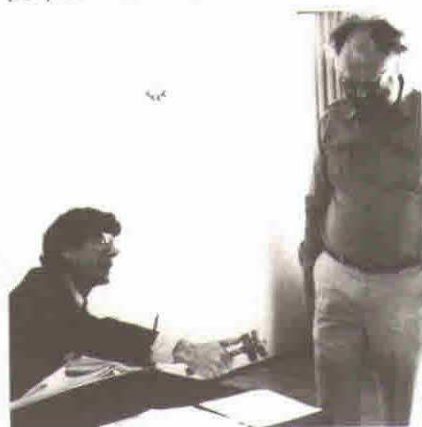
Raymond Tisserand, the convenor of the ISO meeting and Alex Morrow, the editor of the standard, consider a tricky problem.



The location of the standards meeting.



"...and thats the last time you suggest we put)RUN into the standard."



ISO APL STANDARD UPDATE

by David Ziemann

The ISO meetings for the development of the APL standard have a strange habit of cropping up in proximity with the annual APL conferences. It should be no surprise then, to learn that the last such meeting was held in Helsinki, Finland, just prior to this year's international APL conference.

The four day meeting was attended by fifteen delegates from eight countries; Belgium, Canada, Finland, France, Germany, Sweden, UK and USA. It represented the culmination of work carried out by people of these and other countries in studying the first Draft Proposal of the ISO APL Standard (DP1). The fruits of this work consisted of over 190 comments submitted by 10 countries, including comments from Australia, Hungary, Japan and the Netherlands. The objective of the technical work of the meeting was to process and resolve these comments with a view to producing the 2nd Draft Proposal. Each comment was either accepted, accepted with modification or rejected, and in the last two cases a reason for the modification or rejection provided. Some comments uncovered problems in the standard, but failed to address their solutions adequately, in which case the response was an acceptance with modification. Comments proposing solutions that were not adopted (by vote or consensus) were flagged or rejected. Generally, most decisions were made by consensus, with voting for the less clear cut and more important questions.

Comments can be split into two categories; the typographical/editorial type, which are presumably non-contentious, and the substantive comments. The latter group divides into comments which don't intend to change the meaning of the standard, and those that do. It's this last group which is the more interesting one, especially when the comment is accepted and a substantive change is made to the standard. At this point in the life of the document, we wouldn't expect to see many such substantive changes being made, and this was the case at Helsinki. There were, however, a few such modifications, as we shall now see.

As explained in the last issue VECTOR, DP1 required that the catenation of two empty vectors should produce an empty result of type determined by the type of the left argument. For example, where C is an empty character vector, and N an empty numeric vector, the expression C,N would yield an empty character vector, according to DP1. This has now been changed. The original comment that sparked off the discussion at Helsinki pointed out that most APL interpreters in fact produced the right argument as the result, rather than the left, and that therefore the standard was not documenting existing practice in this area. A motion to change the definition in DP1 was made and narrowly passed, and the nature of the change was discussed. It was proposed that the way out of the problem would be to have the standard report a DOMAIN ERROR for this case. A vote was taken and the proposal resoundingly accepted. What this means in practice is not that an implementor must report a DOMAIN ERROR in this case, but that either type of result may be produced for the implementation to conform to the standard. In this way, the consistent-extension mechanism (of replacing error reports by other behaviour) allows an interpreter to return either result. The disadvantages of this approach are twofold. First, it may encourage implementors to report an error in such a case. Second, the change reduces the set of programs (rather than interpreters) that conform to the standard. That is to say, any program that employs catenation of empty vectors of different types will NOT conform to the standard.

Another example is a change made to the definition of 'erasable'. This is an operation used in the formal model that describes the standard interpreter. In DP1 the description of 'erasable' was such that any pendant functions on the SI stack could not be successfully expunged: in other words, the result of applying quad-EX to the name of such a function was always zero. In DP2 the definition will be rewritten to permit an implementor either to allow or disallow a pendant function to be expunged.

The status of the workspace interchange convention (WIC) was also finally decided at Helsinki. This is a convention intended to facilitate the exchange of APL functions and variables between

APL installations in a manner that is independent of specific APL implementations and operating systems. Provision is also made for installations to cooperate in defining local extensions to the convention so that the exchanges need not be limited to functions and variables.

Work on WIC has proceeded in parallel with the work on the APL language standard, and although WIC will not be a part of the standard, it will now be published as an annexe to it.

As you can see from these somewhat esoteric examples, the standardisation process is slowly drawing to a close — or at least the technical aspects of it are. There is still much ISO bureaucracy to get through! It is hoped that the ISO working group will not need to have another formal meeting and that we will have an international APL standard some time in 1985. How important the standard is, will of course, depend on you.

INDEX TO ADVERTISERS

APL People Ltd	Page	82
Cocking & Drury	Page	40
Dyadic Systems Ltd	Page	28
E & S Associates	Page	48
H.M.W. Programming Consultants	Page	58
Imperial Group plc.	Page	98
MicroAPL Ltd.	Pages	51,60
I.P. Sharp Associates	Page	4
John Wiley & Sons Ltd	Page	46

Potential advertisers are reminded that advertisements for the next issue of VECTOR should be booked by the third Friday in October. For further details please contact David Preedy at Metapraxis Ltd., Hanover House, Coombe Road, Kingston-upon-Thames, KT2 7AH, or phone 01-541-1696.

BRITISH APL ASSOCIATION
Membership Application Form

Please read the membership information in the inside front cover of VECTOR before completing this form. Existing members who have not already done so should send in an application to update our records; this will be credited pro-rata for any advance membership fees already paid. Use photocopies of this form for multiple applications. The membership year runs from 1st May 1984 — 30th April 1985.

Name: _____

Department: _____

Organisation: _____

Address line 1: _____

Address line 2: _____

Address line 3: _____

Address line 4: _____

Post or zip code: _____

Country: _____

Telephone number: _____

Membership category applied for (tick one):

Non-voting student membership	Free
UK private membership	£ 6
Overseas private membership	£ 10
Corporate membership	£ 50
Sustaining membership	£250

For student applicants:

Name of course: _____

Name and title of supervisor: _____

Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required.

Send the completed form to the Treasurer at this address:
Mel Chapman, 12 Garden Street, Stafford, ST17 4BT, UK.

THE BRITISH APL ASSOCIATION

The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. It is administered by a Committee of eight officers who are elected by the vote of Association members at the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1984 COMMITTEE

Chairman:	Philip Goacher 01-637 0471	The British Computer Society, 13 Mansfield Street, London W1M 0BD
Secretary:	Anthony Camacho St. Albans 60130	2 Blenheim Road, St. Albans, Herts AL1 4NR.
Treasurer:	Mel Chapman 0785-53511	12 Garden Street, Stafford, ST17 4BT.
Activities:	Dick Bowman 01-634 7639	CEGB, 85 Park Street, London SE1.
Publicity:	Chris Beatty 01-675 2964	Vantage Computer Consulting Ltd. 220 Balham High Road, London SW12.
Journal Editor:	Robert Bittlestone 01-541-1696	Metapraxis Ltd. Hanover House, Coombe Road, Kingston KT2 7AH.
Education:	Vacant	
Technical:	Vacant	

ACTIVITIES WORKING GROUP

David Allen
Dick Bowman
Dominic Murphy
David Preedy
Roy Tallis
Stan Wilkinson

JOURNAL WORKING GROUP

Jonathan Barman	01-493-6172
Robert Bittlestone	01-541-1696
David Preedy	01-541-1696
Adrian Smith	0904-53071
David Ziemann	01-493-6172

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. APL stands for "A Programming Language" — an interactive computer programming language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL*Plus Ltd.

Aston Science Park, Love Lane,
Birmingham B7 4BJ.
Tel. 021-359 5096

Cocking & Drury Ltd.

16 Berkeley Street, London W1X 5AE.
Tel. 01-493 6172

Dyadic Systems Ltd.

30 Camp Road, Farnborough, Hants.
Tel. 0252 547222

Inner Product Ltd.

Eagle House, 73 Clapham Common Southside,
London SW4 9DG.
Tel. 01-673 8354

MicroAPL Limited

Unit 1F, Nine Elms Industrial Estate,
87 Kirling Street, London SW8 5BP.
Tel. 01-622 0395

I.P. Sharp Associates

132 Buckingham Palace Road,
London SW1W 9SA.
Tel. 01-730 4567

